



河南大學
HENAN UNIVERSITY

第5章 中央处理器

<u>5.1 CPU的功能和组成</u>	——掌握
<u>5.2 指令周期</u>	——掌握
<u>5.3 时序发生器和控制方式</u>	——了解
<u>5.4 微程序控制器</u>	——掌握
<u>5.5 硬连线控制器</u>	——了解
<u>5.6 流水线技术与流水处理器CPU</u>	——理解

5.1 CPU的组成和功能

5.1.1 CPU的功能

5.1.2 CPU的基本组成

5.1.3 CPU中的主要寄存器

5.1.4 操作控制器与时序发生器

>>> 5.1.1 CPU的功能

➤ CPU（中央处理器）

- 控制程序按设定方式执行；

➤ CPU的主要功能：

- 指令控制 **顺序寻址、跳跃寻址**
 - ◆ 控制程序的执行顺序；
- 操作控制 **对指令操作码译码后产生控制信号**
 - ◆ 产生和发送各部件的操作控制信号；
- 时间控制 **指令各个操作实施时间的定时**
 - ◆ 控制指令、或操作的实施时间和先后顺序；
- 数据加工 **由ALU完成具体的运算**
 - ◆ 对数据进行算术逻辑运算；

>>> 5.1.2 CPU的基本组成

➤现代的CPU的组成

- 运算器、控制器、片内Cache;

冯·诺依曼机的定义

➤控制器

- 性质：控制部件
- 组成：程序计数器、指令寄存器IR、指令译码器ID、时序发生器和操作控制器
- 功能
 - ◆取指：从内存中取出一条指令，并指出下条指令的存放位置；
 - ◆分析：对指令进行译码，产生相应的操作控制信号；
 - ◆执行：控制CPU、内存和输入/输出设备之间数据流动；

>>> 5.1.2 CPU的基本组成

➤ 运算器

- **性质：** 执行部件
- **组成：** 算术逻辑运算单元(ALU)、寄存器组、状态条件寄存器
- **功能：** 数据加工
 - ◆ 执行所有的算术运算；
 - ◆ 执行所有的逻辑运算，并进行逻辑测试。

>>> 5.1.3 CPU中的主要寄存器 (1/3)

➤ 中央处理器CPU = 运算器 + cache
+ 控制器

➤ 运算器

- ALU

- 通用寄存器：R0 ~ R3（所用模型中）

- 暂存器：DR

- 程序状态字寄存器：PSWR

➤ cache

- 指令cache：PC，IBUS

- 数据cache：AR，DBUS

➤ 数据缓冲寄存器（DR）

- 暂时存放运算器与外界传送的数据。

- 作用：

- ① 作为CPU和内存、外部设备之间信息传送的中转站；

- ② 补偿CPU和内存、外围设备之间的速度差异；

➤ 指令寄存器（IR）

- 保存当前正在执行的一条指令。

- 其对应的操作码字段直接作为指令译码器的输入。CPU中的通用寄存器可多达16个，32个，甚至更多。

>>> 5.1.3 CPU中的主要寄存器 (2/3)

➤ 程序计数器 (PC)

- 始终存放下一条指令的地址，对应于指令Cache的访问；
- 其内容变化分两种情况
 - ◆ 顺序执行： $PC+1 \rightarrow PC$
 - ◆ 转移执行： (指令OPR) $\rightarrow PC$

计数自增功能

➤ 地址寄存器 (AR)

- 保存当前CPU所访问数据的内存单元地址；
- 上图中显示AR中的地址用于寻址数据Cache中的单元；
 - ◆ 对于数据Cache未命中的情况下，AR中的内容将用于访问CPU之外的存储器。

➤ 通用寄存器

- 功能：暂时存放ALU运算的数据或结果。

➤ 程序状态字寄存器/状态条件寄存器 (PSWR)

- 保留算术逻辑运算指令或测试指令的结果而建立的各种状态信息
- 保存各种状态和条件控制信号；
 - ◆ 进位标志(C)，溢出标志(V)，零标志(Z)，符号标志(N)
- 每个信号由一个触发器保存，从而拼成一个寄存器。

>>> CPU中的主要寄存器

1. 数据缓冲寄存器（DR）

- 暂时存放运算器与外界传送的数据。
- 补偿CPU和主存、外围设备的速度差；

2. 通用寄存器

- 暂时存放ALU运算的数据或结果。
- 一般包含16/32个，甚至更多寄存器；

3. 状态条件寄存器（PSW）

- 保存各种状态和条件控制信号；
 - ◆进位标志，溢出标志，零标志，符号标志；
 - ◆保存中断和系统工作状态等信息

4. 地址寄存器（AR）

- 保存当前CPU要访问的主存单元地址；

5. 程序计数器（PC）

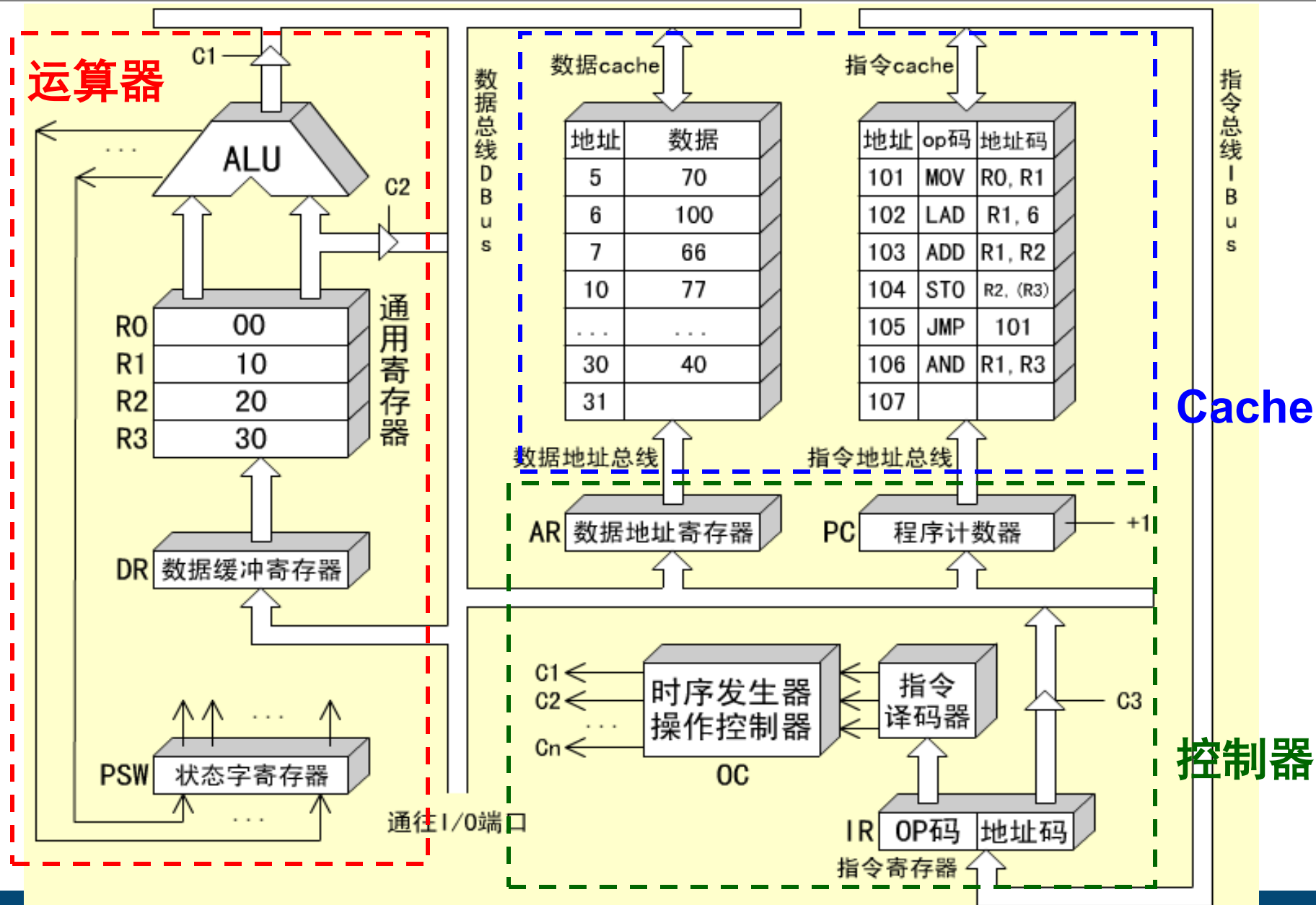
- 始终存放下一条指令的地址，对应于指令Cache的访问；

6. 指令寄存器（IR）

- 保存当前正在执行的一条指令。
- IR的高位为操作码字段，作为指令译码器的输入。

>>> CPU模型图

动画演示: CPU基本结构.swf



下列存储器中，汇编语言程序员可见的是（ ）

- ☐ A 存储器地址寄存器（MAR）
- ☒ B 程序计数器（PC）
- ☐ C 存储器数据寄存器（MDR）
- ☐ D 指令寄存器（IR）

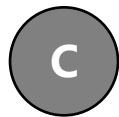
提交

(2018) 减法指令“sub R1,R2,R3”的功能为“(R1) — (R2) → (R3)”,该指令执行后将生成进/借位标志CF和溢出标志OF,。若(R1) = FFFF FFFFH, (R2) = FFFF FFF0H, 则减法指令执行后, CF与OF分别为_____。



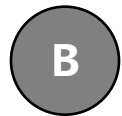
A

CF=0, OF=0



C

CF=0,OF=1



B

CF=1,OF=0



D

CF=1,OF=1

提交

>>> 5.1.4 操作控制器与时序发生器

➤ 数据通路

- 两个部件之间传送信息的通路(1)。具体流向由操作控制器的部件完成。
- CPU内各个寄存器以及系统总线之间的信息传送通路(2)。

➤ 操作控制器

- 根据指令操作码和时序信号，产生各种操作控制信号；
- 建立正确地数据通路，从而完成指令的执行。

➤ 根据设计方法不同，操作控制器可分为

- ① 硬布线控制器：采用时序逻辑技术实现；
- ② 微程序控制器：采用存储逻辑实现；
- ③ 前两种方式的结合；

➤ 时序发生器

- 提供定时和时序信号，对各种操作实施时间的控制。
- 时序发生器保障了操作控制器定时发出控制信号，对各种控制信号实施的严格控制

>>> 5.1.4 操作控制器与时序发生器

➤ 根据设计方法不同，操作控制器可分为

- ① **硬布线控制器：** 采用时序逻辑技术实现；
- ② **微程序控制器：** 采用存储逻辑实现；
- ③ **前两种方式的结合；**

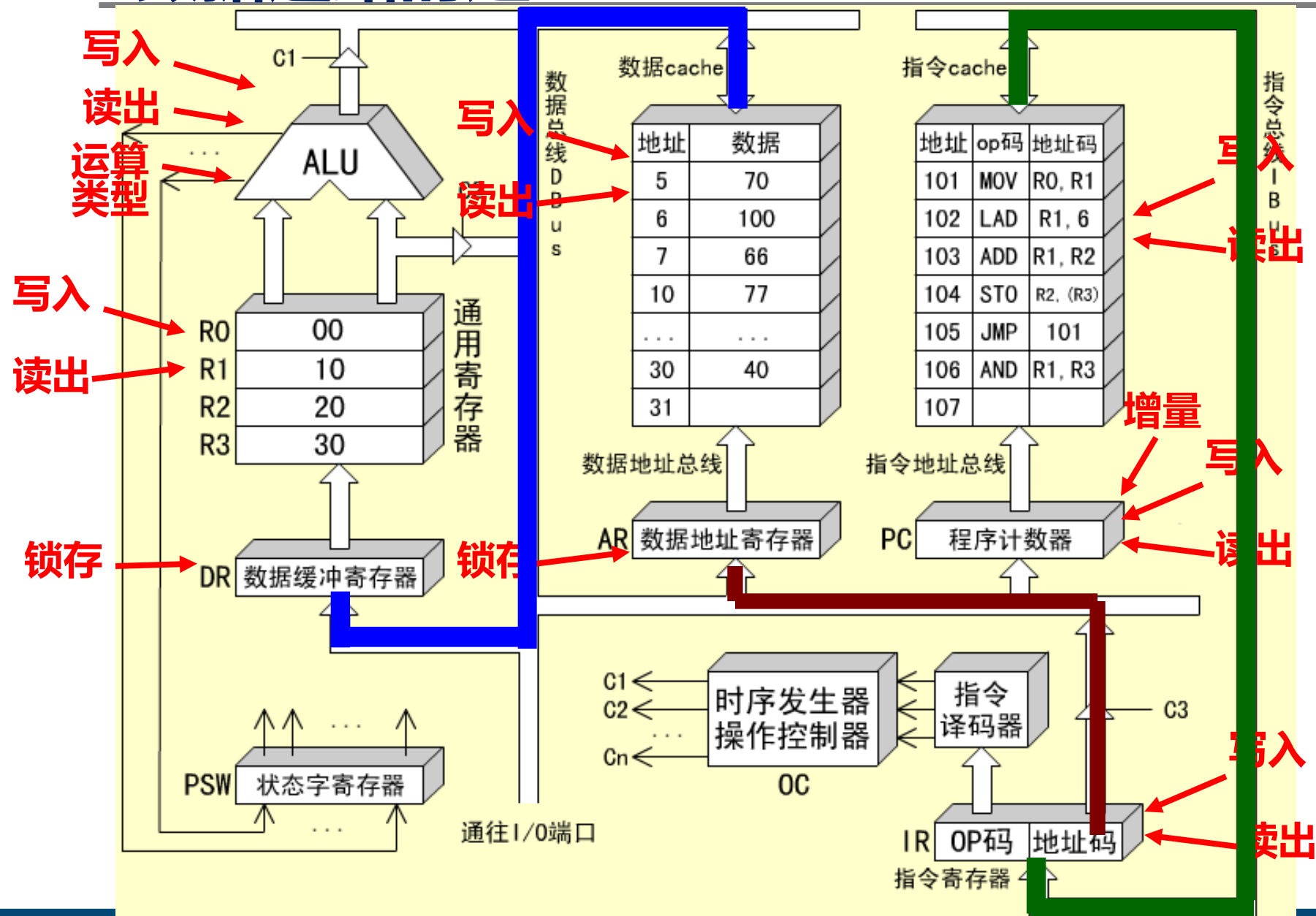
➤ 硬布线控制器（组合逻辑控制器）

- 用组合逻辑电路产生固定时序控制信号
- 控制信号形成部件是由门电路组成的复杂树形网络
- 优点：速度快
- 缺点：时序控制信号形成部件的结构不规整，设计、调试、维护困难，难以实现设计自动化

➤ 微程序控制器（存储逻辑控制器）

- 采用存储逻辑实现
- 微操作信号代码化，将每条机器指令转化为一段微程序并存入控制存储器中，微操作控制信号由微指令产生
- 优点：设计规整，调试、维护及更新、扩充指令方便，易于实现自动化设计
- 缺点：指令执行速度比硬布线控制器慢

>>> 数据通路的建立



数据通路

- 指令的执行过程就是在CPU内各部件之间以及CPU与外部部件之间，依次、依条件传递信息并进行必要的判断和数据加工的过程
- CPU内部的信息传送通过CPU内部总线进行；CPU与外界交换信息通过系统总线进行

>>> 5.2 指令周期

5.2.1 指令周期的基本概念

5.2.2 MOV R0 , R1指令的指令周期

5.2.3 LAD R1 , 6指令的指令周期

5.2.4 ADD R1 , R2指令的指令周期

5.2.5 STO R2 , (R3)指令的指令周期

5.2.6 JMP 101指令的指令周期

5.2.7 用方框图语言表示指令周期

>>> 5.2.1 指令周期的基本概念

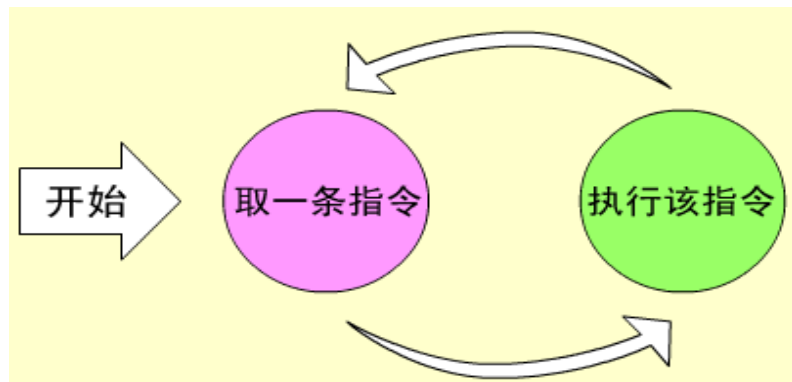
➤ CPU执行程序是一个“取指令—执行指令”的循环过程。

➤ 指令周期

- CPU从内存中**取出**一条指令，并**执行完毕**的时间总和；
- 包含若干个CPU周期

➤ CPU周期（**机器周期**）

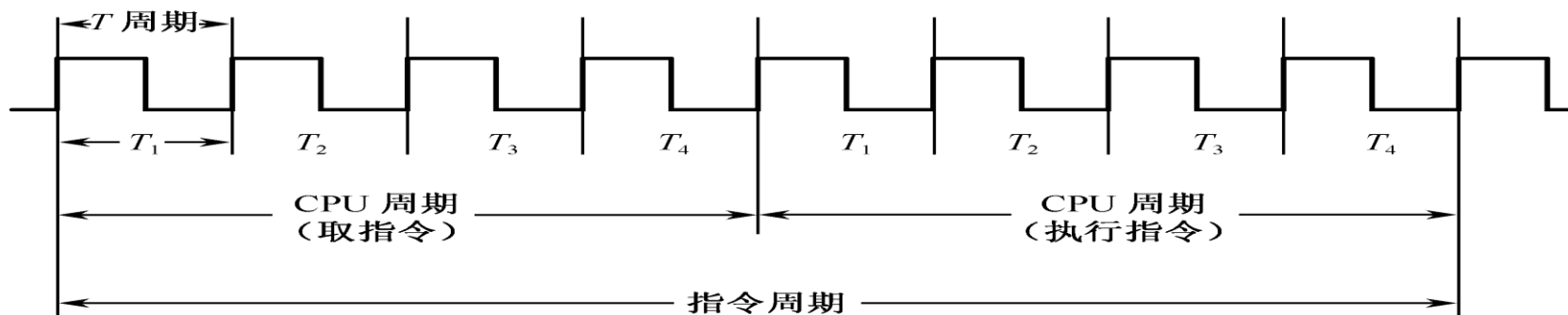
- 指内存中读取一个指令字的最短时间；
- CPU周期通常包含若干个时钟周期
- 时钟周期，又称节拍脉冲或T周期，处理操作的最基本单位。



>>> 5.2.1 指令周期的基本概念

➤ 时钟周期(节拍脉冲、节拍周期或T周期)

- 在一个CPU周期内，要完成若干个微操作。这些微操作有的可以同时执行，有的需要按先后次序串行执行。因而需要把一个CPU周期分为若干个相等的时间段，每一个时间段称为一个节拍脉冲或T周期。
- 时钟周期通常定义为机器主频的倒数。
- 注意：不同的计算机系统定义的术语未必相同，例如：在不采用三级时序的系统中，时钟周期相当于T周期。



>>> 单周期、多周期和流水线处理器

- **单周期CPU**：在一个时钟周期内完成从指令取出到得到结果的所有工作
 - 所有指令执行时间都以最长时间的指令为准（执行步骤相对固定）
 - 效率低，较少采用。
- **多周期CPU**：把指令的执行分成多个阶段，每个阶段在一个时钟周期内完成
 - 时钟周期短
 - 不同指令所用周期数可以不同
- **流水线处理器**：在多周期CPU设计的基础上，让各个阶段的执行在时间上重叠并行执行
 - 允许功能部件在一条指令执行过程中被重复使用

>>>关于指令周期

➤一个完整的指令周期由若干机器周期：

○**取指周期**——**间址周期**（取操作数阶段）——**执行周期**——**中断周期**

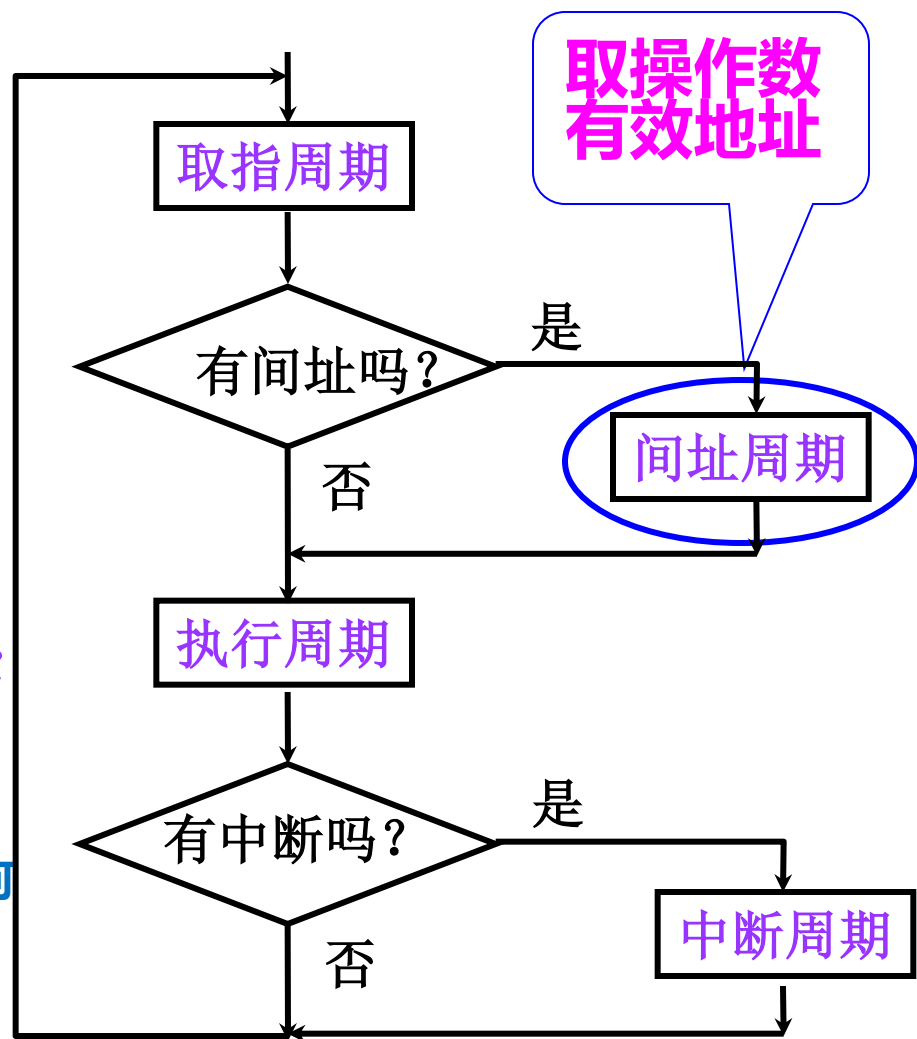
○任何指令周期最少两个机器周期：**取指周期+执行周期**；

➤不同指令的指令周期长度不一定相同。

○所有指令的**第一个机器周期**必为**取指周期**；

- ◆从内存指定单元取出当前指令送至CPU内的指令寄存器IR
- ◆修改程序计数器PC，以便取下一条指令
- ◆对指令的操作码字段进行译码测试，以便确定下一步进行何种操作

○指令的执行周期可包含1~多个机器周期；



>>>关于指令周期

➤ 一个完整的指令周期由若干机器周期：

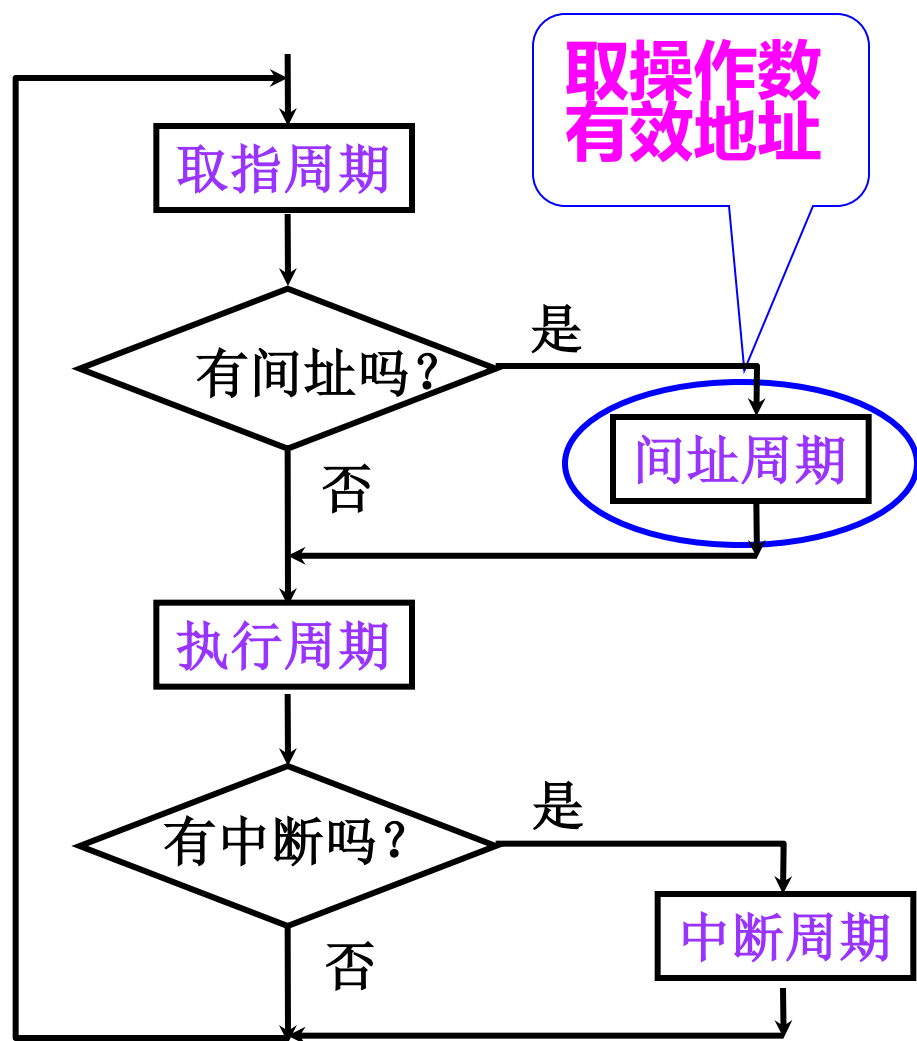
○ **取指周期**——**间址周期**（取操作数阶段）——**执行周期**——**中断周期**

➤ 取操作数阶段

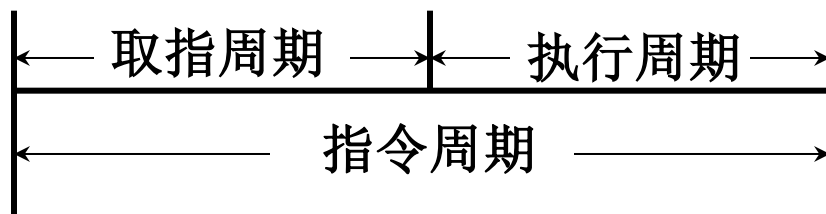
- ◆ 由指令中的形式地址计算实际地址并访问内存取得操作数
- ◆ 只有在需要到内存取操作数的指令中才有该阶段
- ◆ 直接寻址方式和间接寻址方式的取数阶段不相同

➤ 执行周期

- 因各条指令的操作不同而复杂程度各异
- 占用时间相差比较大

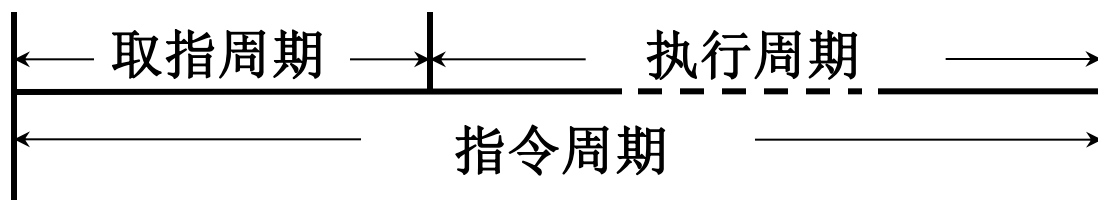


>>> 不同指令的指令周期不同 (指令的执行过程)



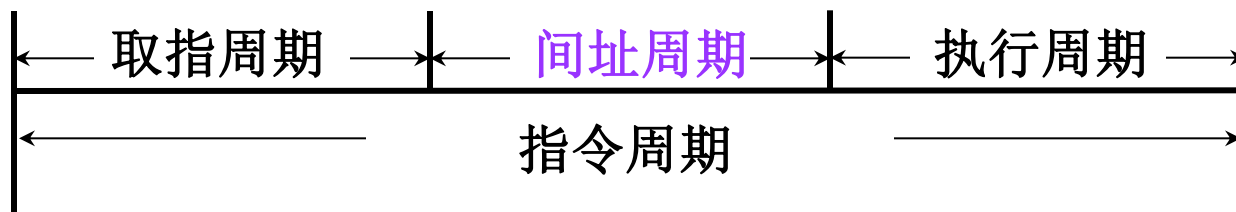
`MOV AX, 0`

不需访存



`ADD AX, BX`

不需访存



`ADD AX, 10[BX]`

需访存

>>> 一个简单的程序

地址	数据
5	70
6	100
7	66
10	77
...	...
30	40 (120)

八进制地址	指令	说明
100		程序执行前设置(R0)=00, (R1)=10, (R2)=20, (R3)=30
101	MOV R0 , R1	传送指令MOV执行(R1)→R0
102	LAD R1 , 6	取数指令LAD从6号单元中取数100→R1
103	ADD R1 , R2	加法指令ADD执行(R1)+(R2)→R2, 结果为(R2)=120
104	STO R2 , (R3)	存数指令STO用(R3)间接寻址, (R2)=120写入30号单元
105	JMP 101	转移指令JMP改变程序执行顺序, 转到101号单元
106	AND R1 , R3	逻辑与指令AND执行(R1) · (R3) →R3

>>> 5.2.2 MOV R0, R1指令的指令周期

➤ **MOV**是一条**RR**型指令，它需要两个**CPU** 周期：（不需访存，**RR**型指令）

○ 取指周期

- ① 从存储器中取出指令；
- ② 程序计数器PC加1；
- ③ 译码或测试指令操作码，发出控制信号；

○ 执行周期

- ◆ 在控制信号的作用下，将R1中的数据经过ALU送入R0；

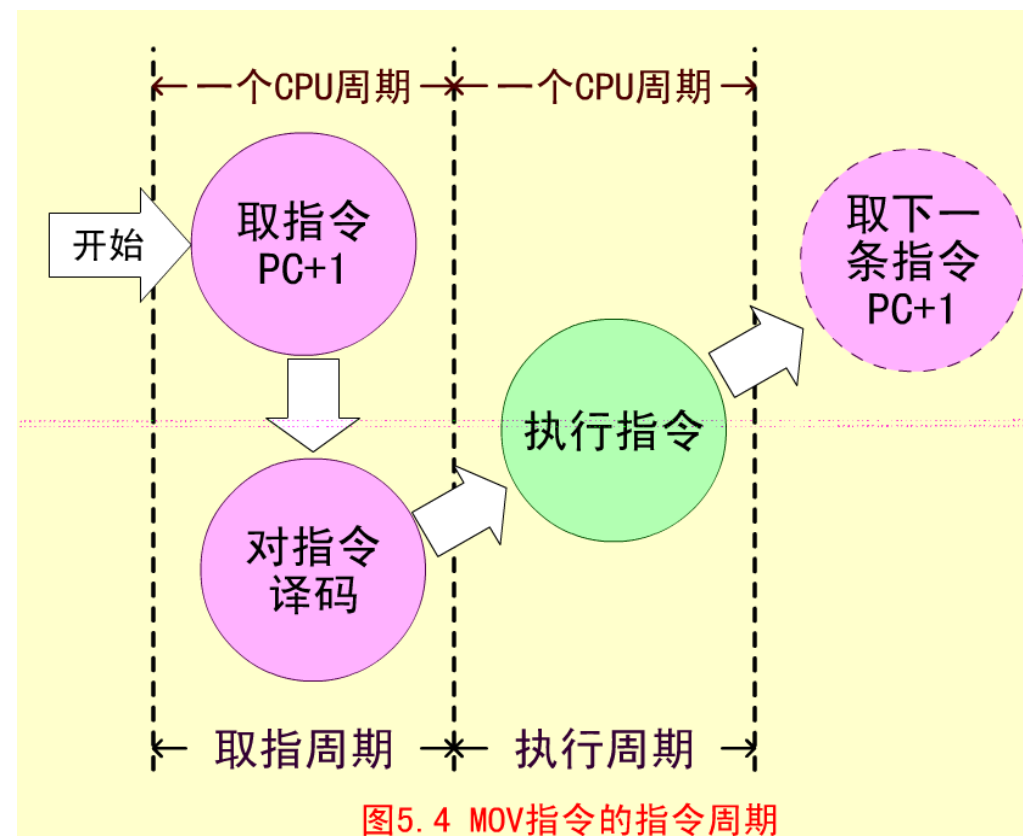
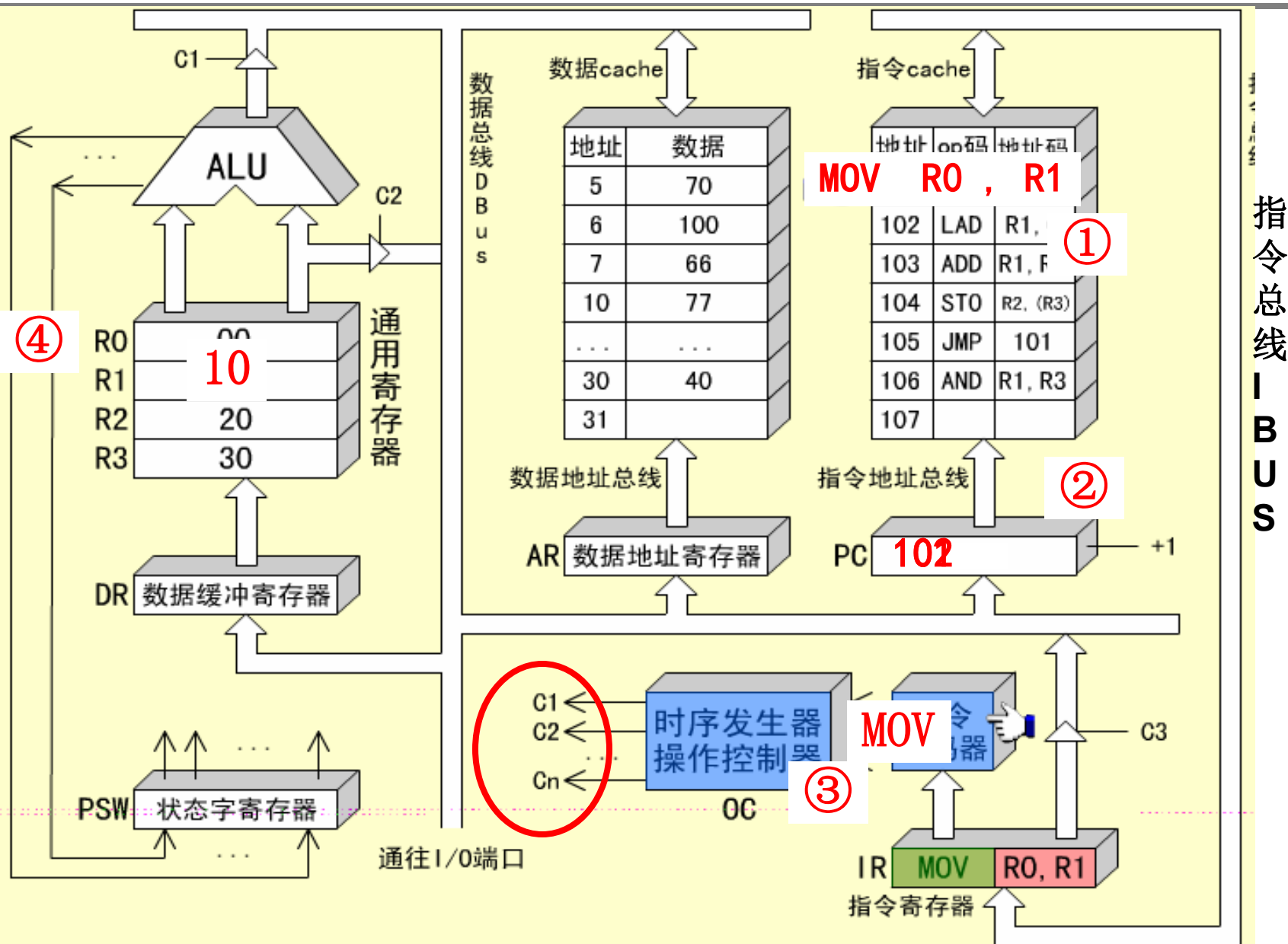


图5.4 MOV指令的指令周期

>>> MOV R0, R1指令的执行过程演示



>>> MOV R0, R1指令周期中的控制信号

1. 取指周期

- ① PC → **ABUS** → 指令Cache, 译码并启动;
- ② 指令Cache → IR;
- ③ PC+1 → PC, 为取下条指令做好准备; **PC增量**
- ④ IR中的操作码被译码或测试, CPU识别出是指令MOV。

2. 执行指令周期

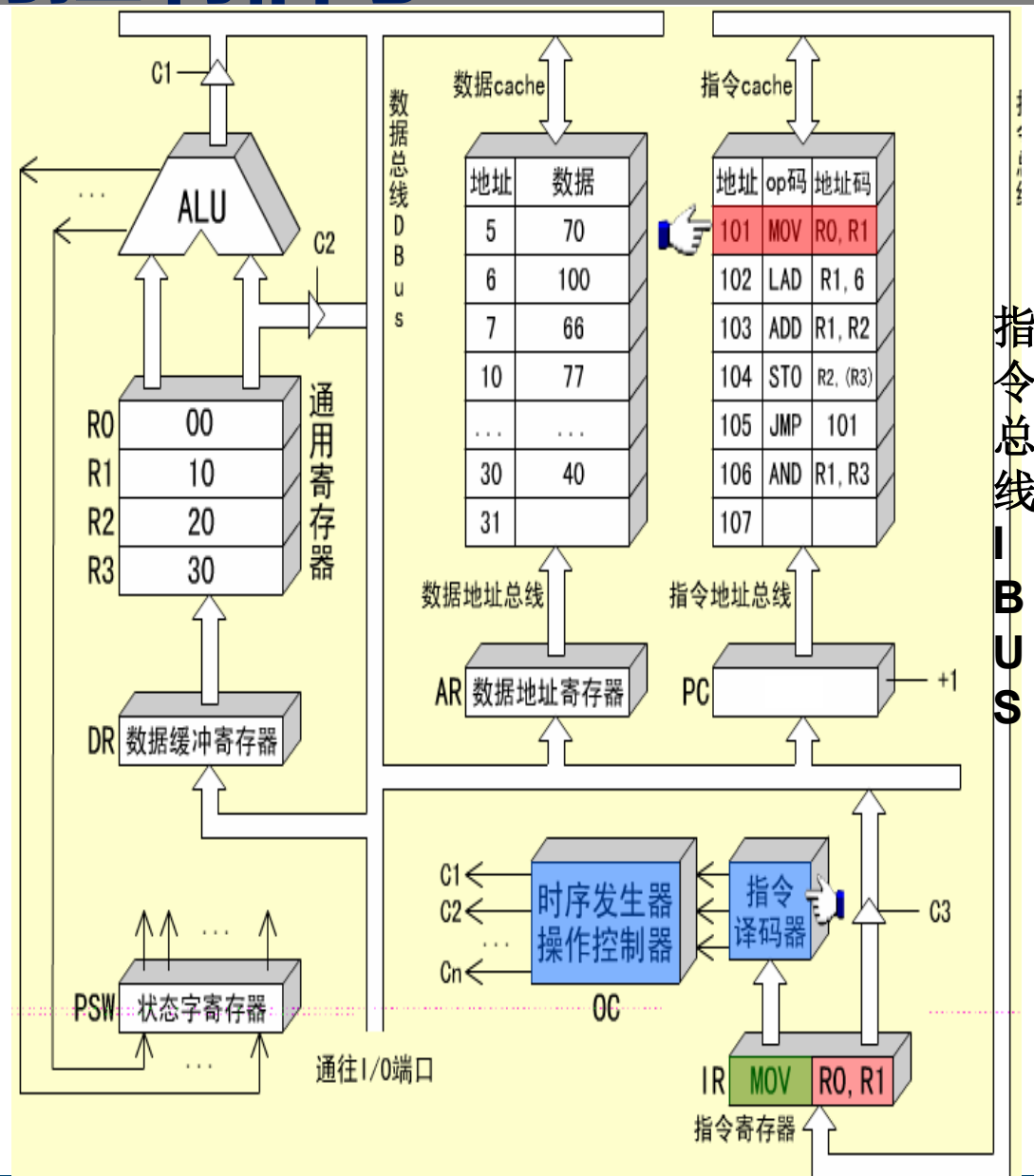
- ① R1 → ALU, R1中数据通过ALU传送;
- ② ALU → **DBUS** → DR → R0;

PC读
指令Cache启动

指令Cache读
IR写

R1读
ALU传送控制

ALU输出
DR锁存
R0写



>>> 5.2.3 LAD R1, 6指令的指令周期

➤ **功能：**将内存给定单元的数据送至通用寄存器

LAD R1, 6: (6) → R1 内存直接寻址

➤ **LAD指令是RS型指令，需要访存获取操作数，共包含三个CPU周期：**

① **取指周期**

② **执行周期 (2个CPU周期)**

a) **使用直接地址访问数据Cache，获得源操作数；**

b) **将源操作数送入通用寄存器R1；**

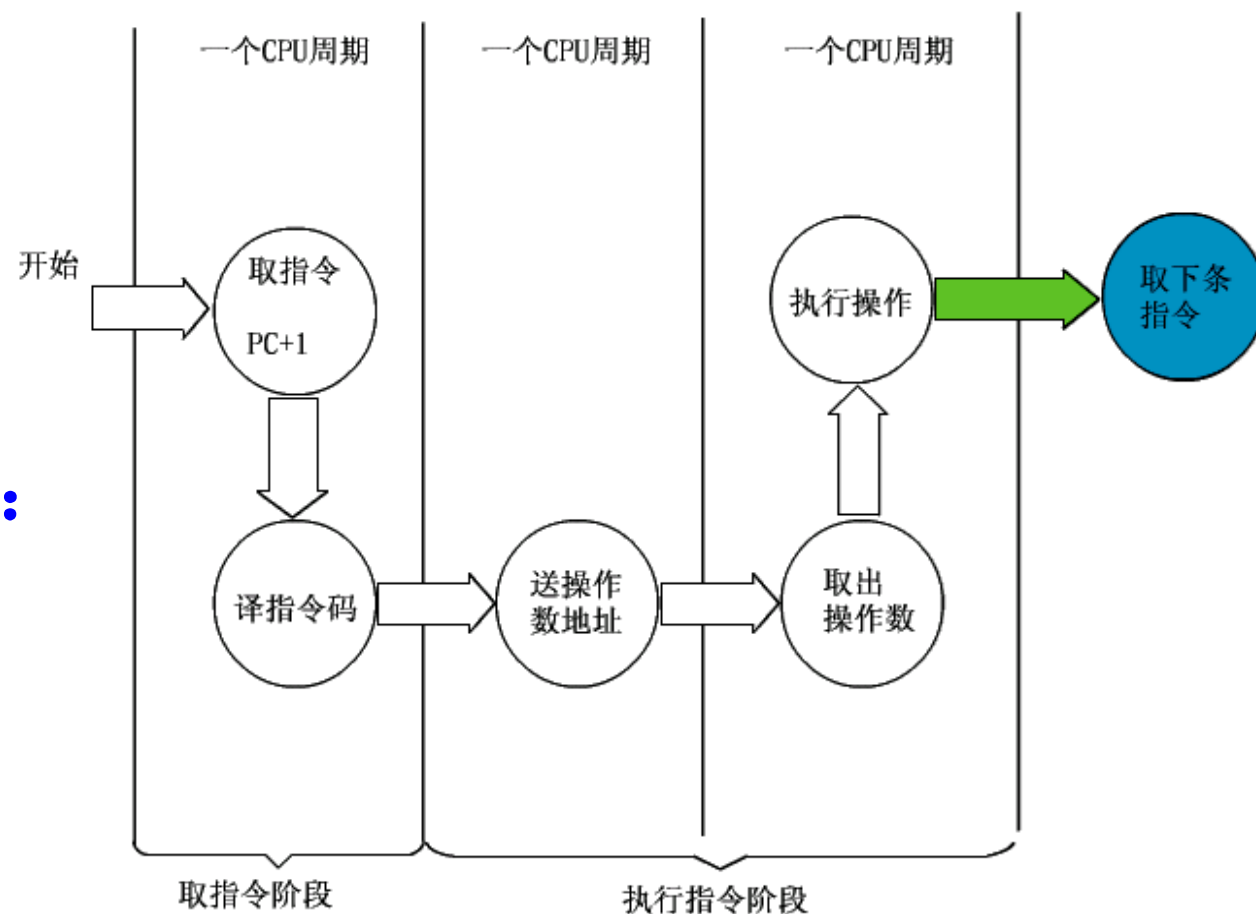
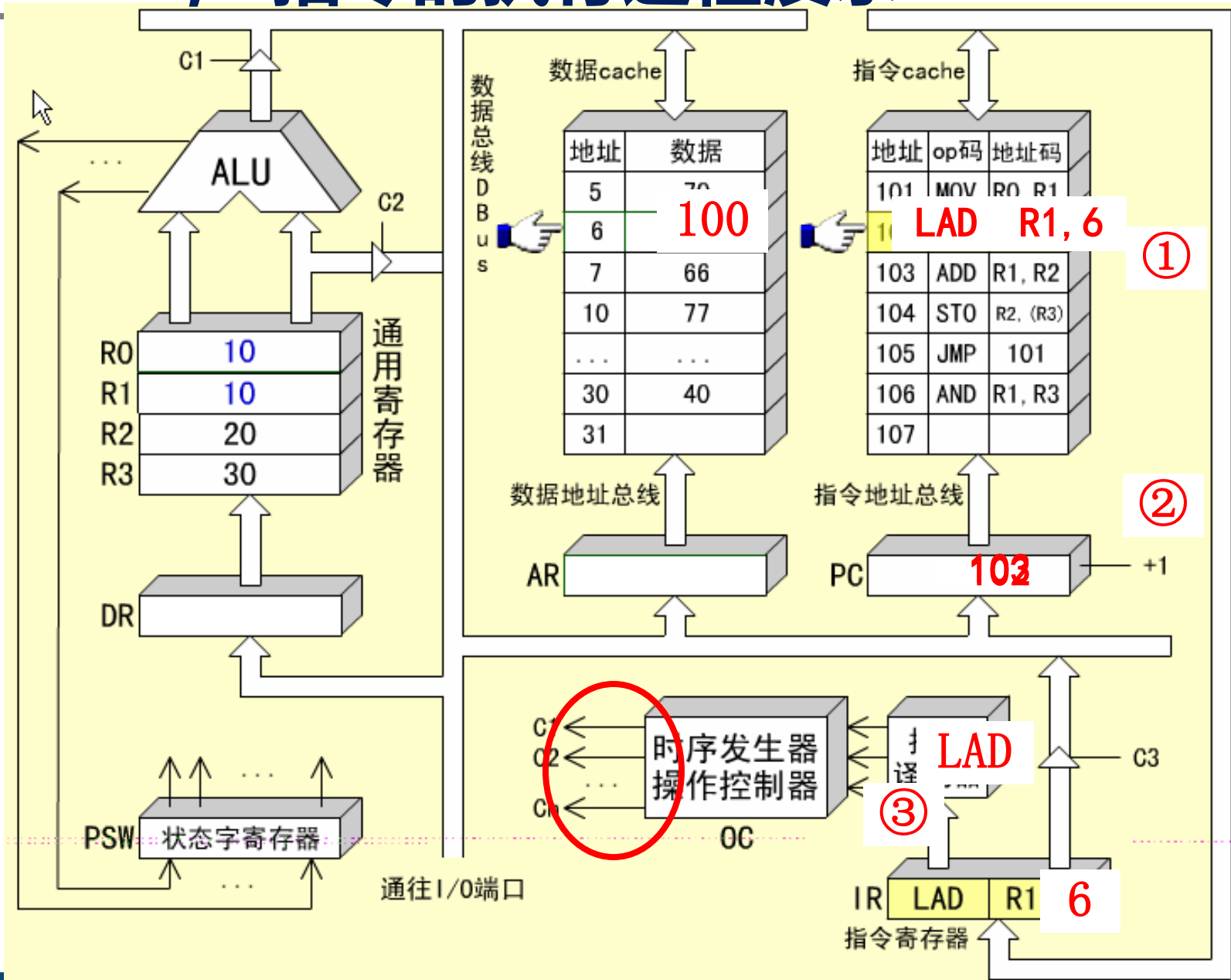


图5.7 直接访内指令的指令周期

>>> LAD R1, 6指令的执行过程演示



>>> LAD R1, 6指令周期中的控制信号

1.LAD取指周期

○与MOV指令的取指周期一样。

2.LAD指令的执行周期

○IR → DBUS → AR;

IR读
AR锁存

◆ AR → ABUS → 数据Cache, 译码并启动;

○数据Cache → DBUS → DR → R1;

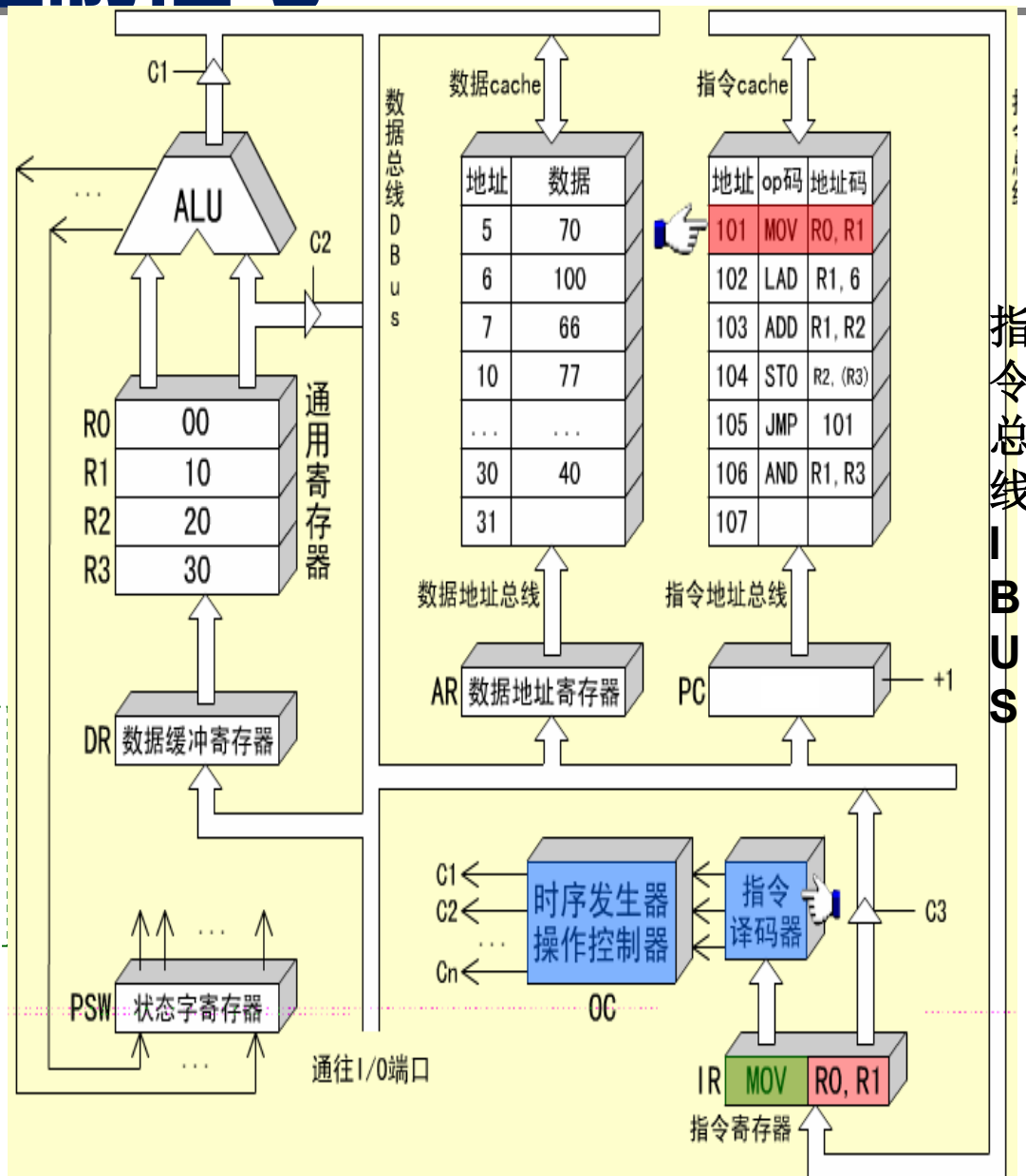
数据Cache启动

数据Cache读
DR锁存
R1写

执行周期需要两个CPU周期的原因是:

源操作数的地址和源操作数都需要经过

DBUS传送;



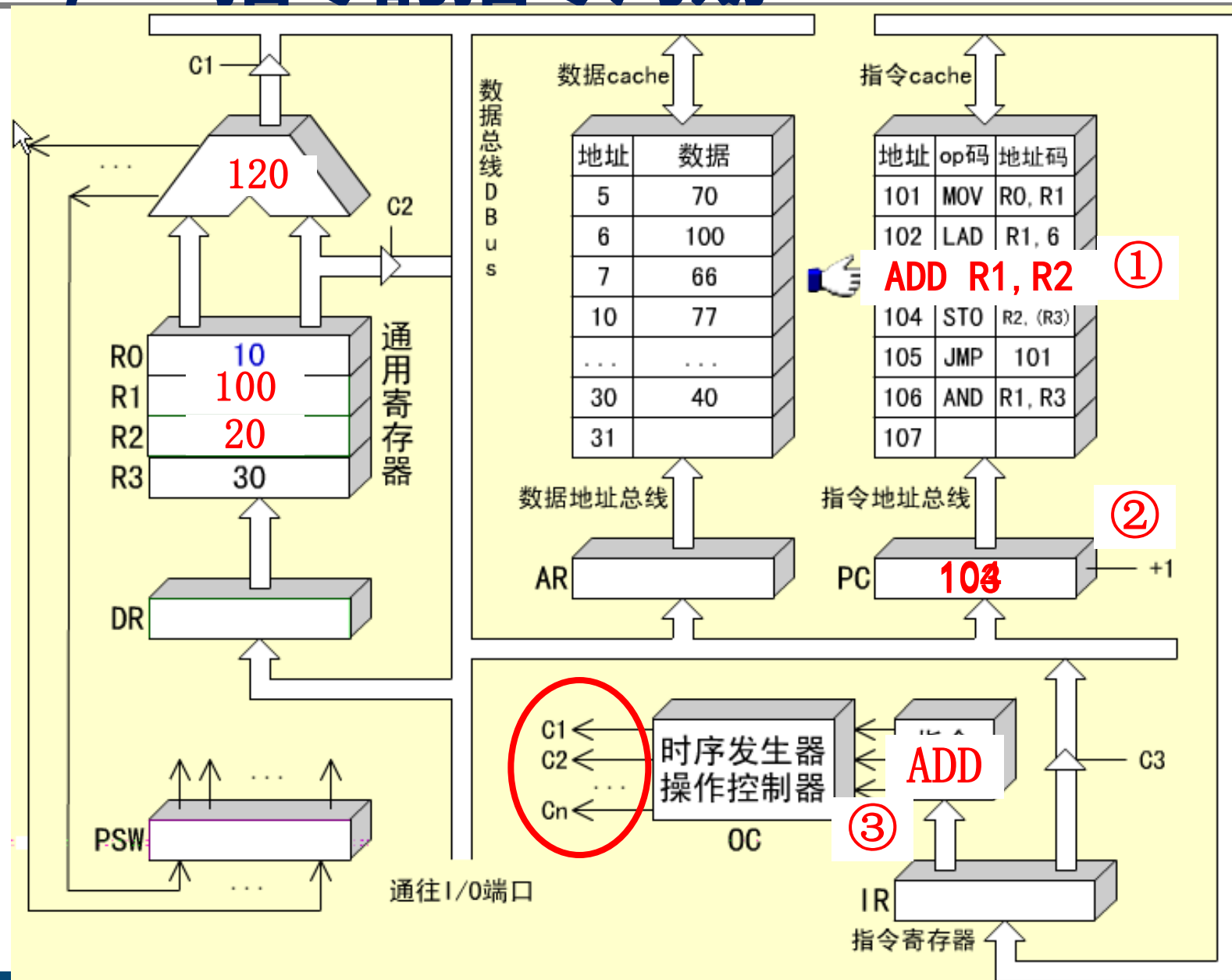
>>> 5.2.4 ADD R1, R2指令的指令周期

➤ **ADD指令的指令周期由两个CPU周期组成。**

① **取指周期（略）**

② **执行周期**

- ◆ **从寄存器R1、R2中取出数据，作为源操作数；**
- ◆ **将两数据送往ALU，并使ALU进行加运算；**
- ◆ **结果保存到R2中。**



>>> ADD R1, R2指令周期中的控制信号

①取指周期（略）

○ 与MOV指令的取指周期一样。

②执行周期

○ R1、R2→ALU;

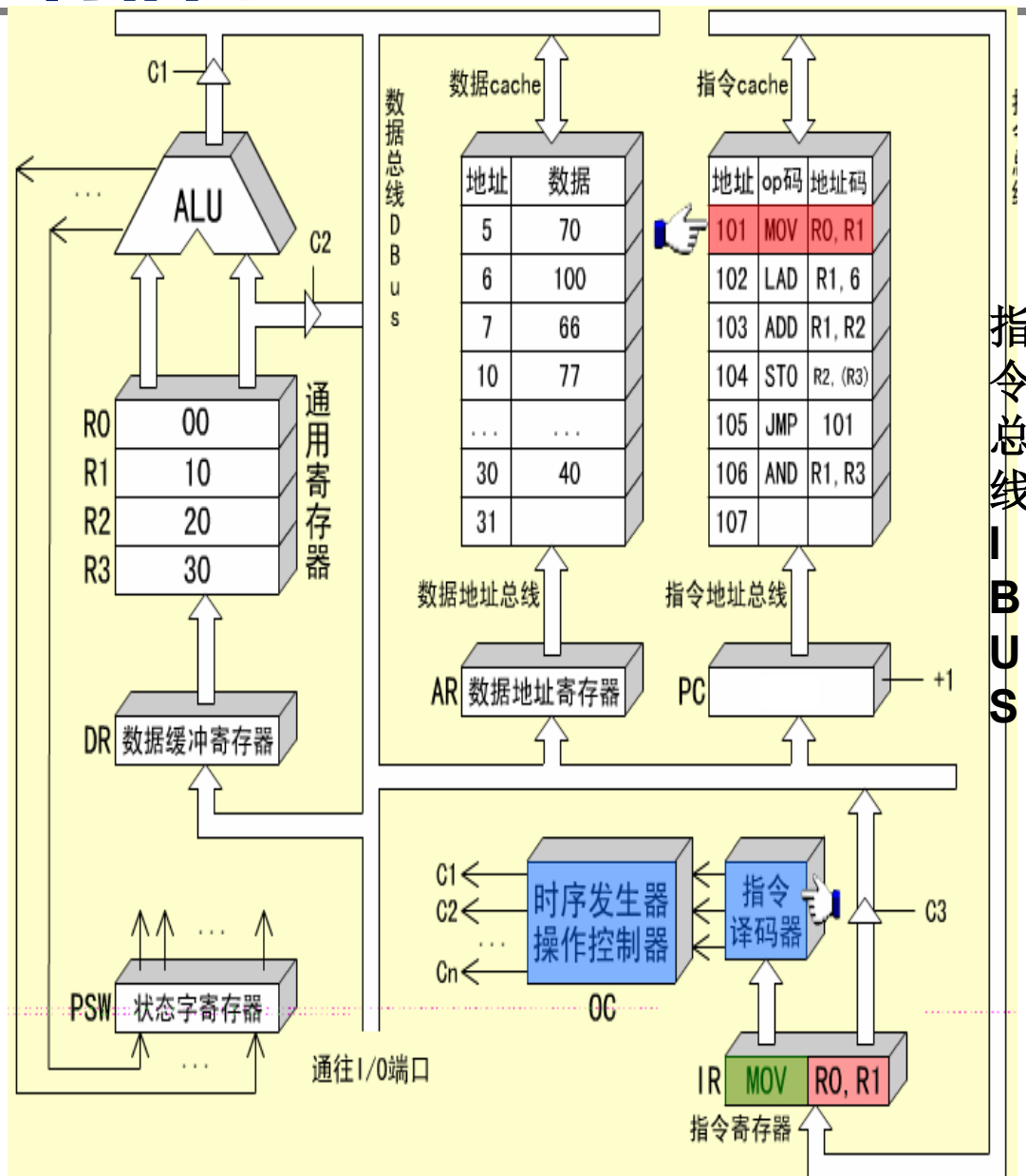
○ ALU做加运算，将两数相加;

○ ALU→DBUS→DR→R2, 保存结果;

ALU加

ALU输出
DR锁存
R2写

R1读
R2读



指令总线 I BUS

>>> 5.2.5 STO R2, (R3)指令的指令周期

➤ **功能：**将寄存器内容送入由寄存器指定的内存单元。
STO R2, (R3): R2→(R3)

➤ **STO指令是RS型指令，需要3个CPU周期。**

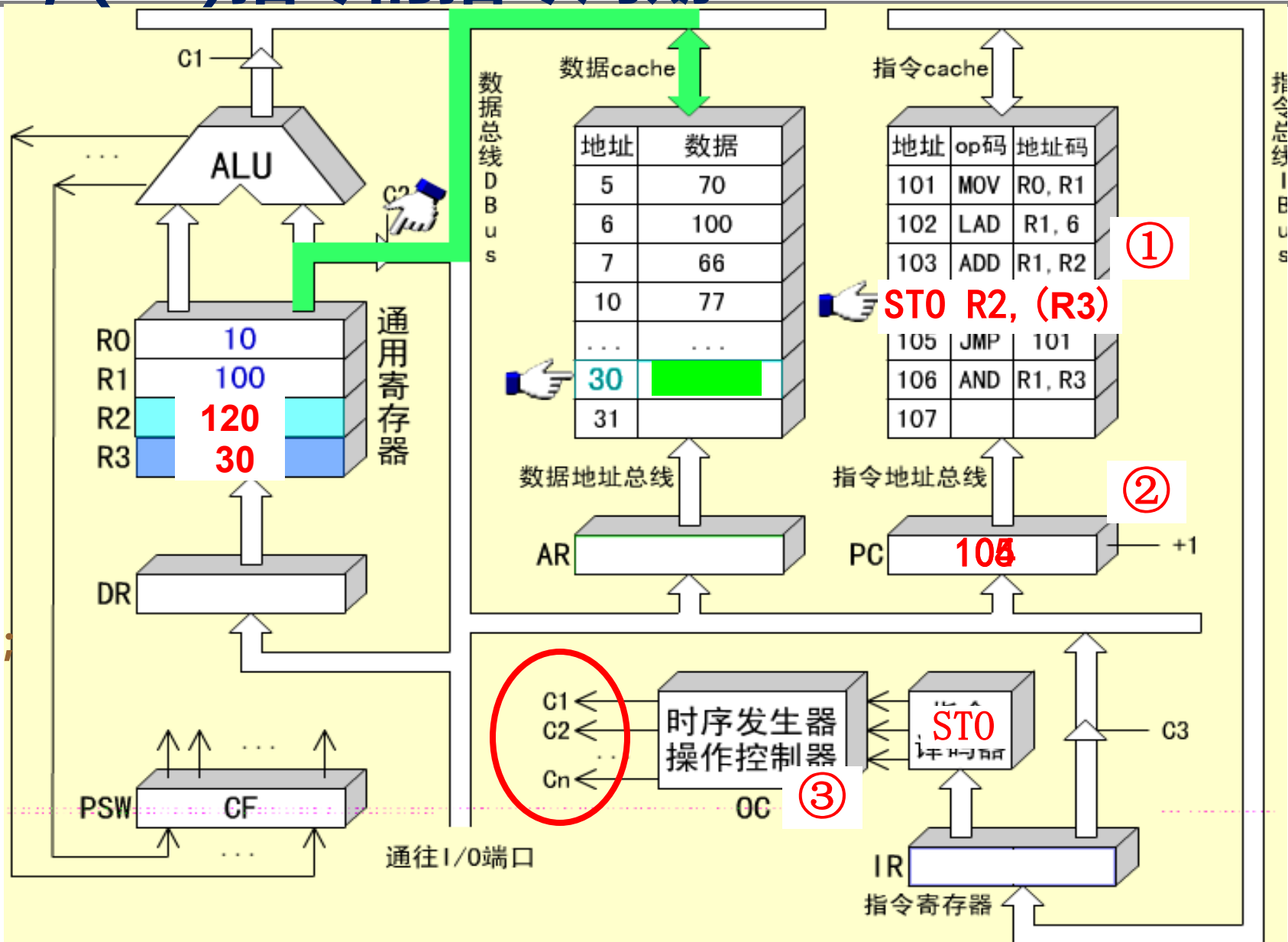
① **取指周期（略）**

② **间址周期**

◆ 根据R3中的地址寻址存储单元；

③ **执行周期**

◆ 将寄存器R2中的数据送入选定的存储单元；

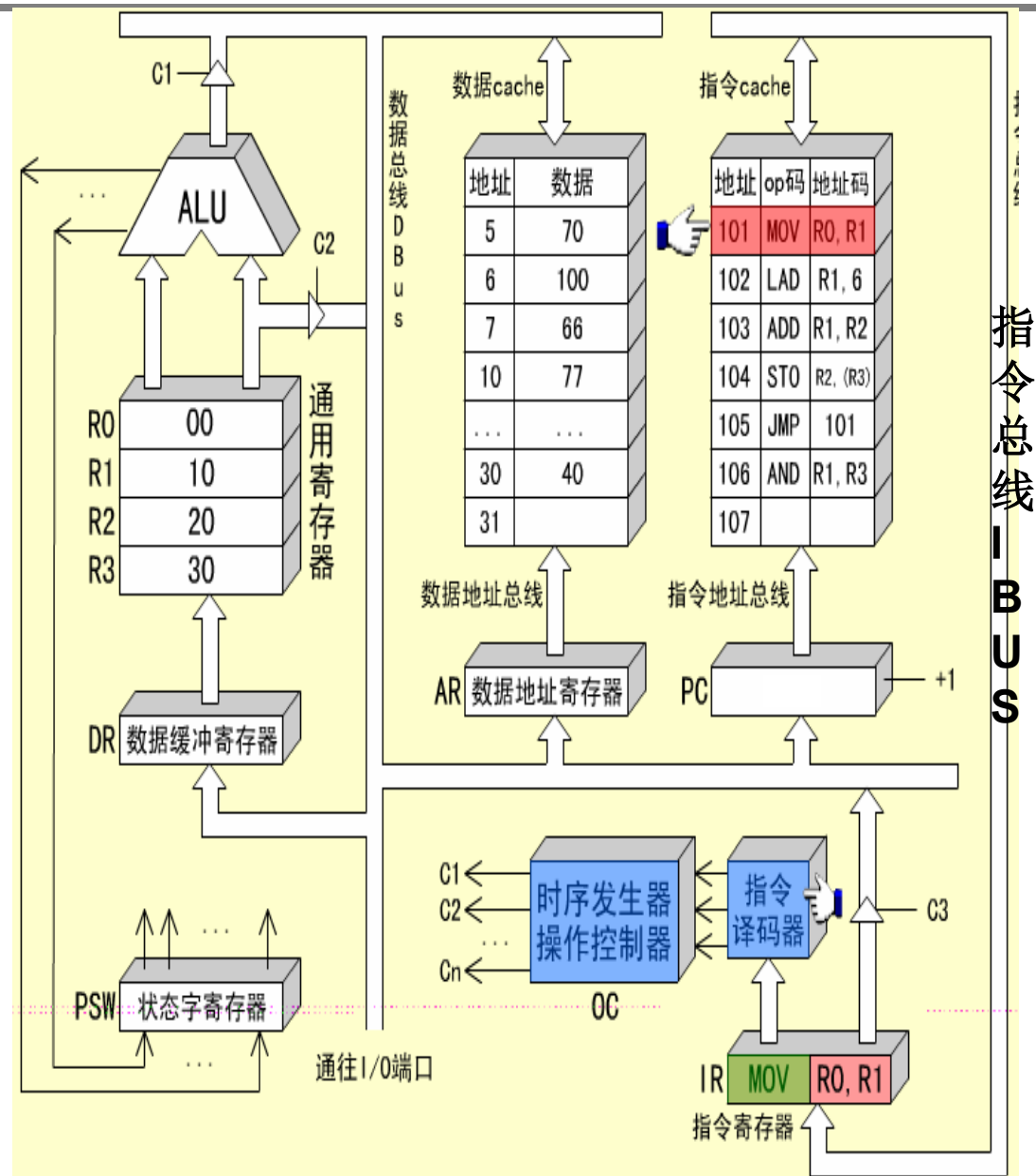


>>> STO R2, (R3)指令周期中的控制信号

1. 取指周期（略）

2. 执行周期

- **R3→DBUS→AR**, 发出地址启动数据Cache;
◆ 该过程为间址周期;
○ **R2→DBUS→数据Cache**;
R3读AR锁存
R2读数据Cache写



>>> 5.2.6 JMP 101指令的指令周期

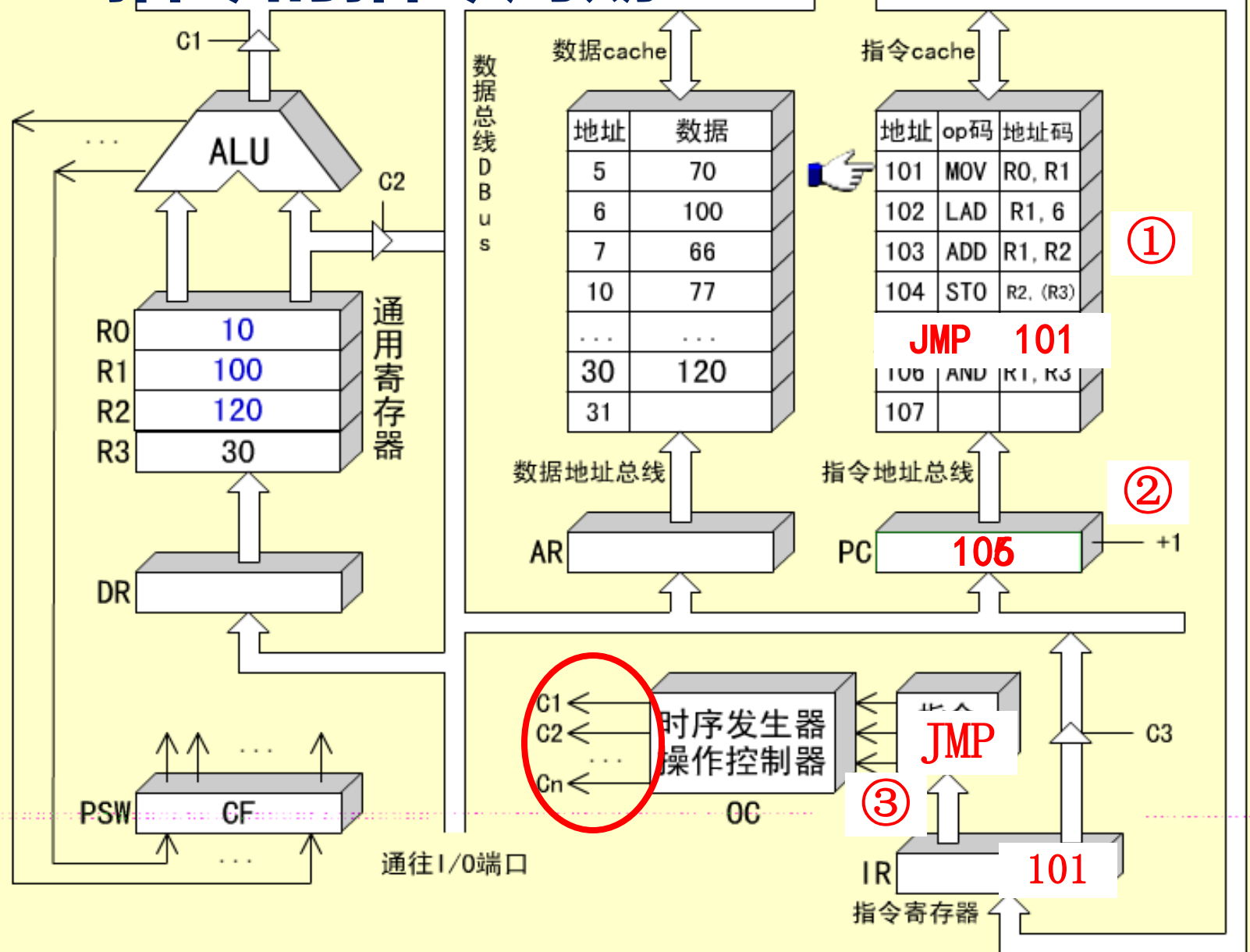
JMP 101指令的执行过程演示

- **JMP**指令是一条无条件转移指令，用来改变程序的执行顺序；
- **JMP**指令的执行需要两个**CPU**周期：

①取指周期（略）

②执行周期

- ◆ 使用**JMP**指令中的直接地址为PC赋值；



>>> JMP 101指令周期中的控制信号

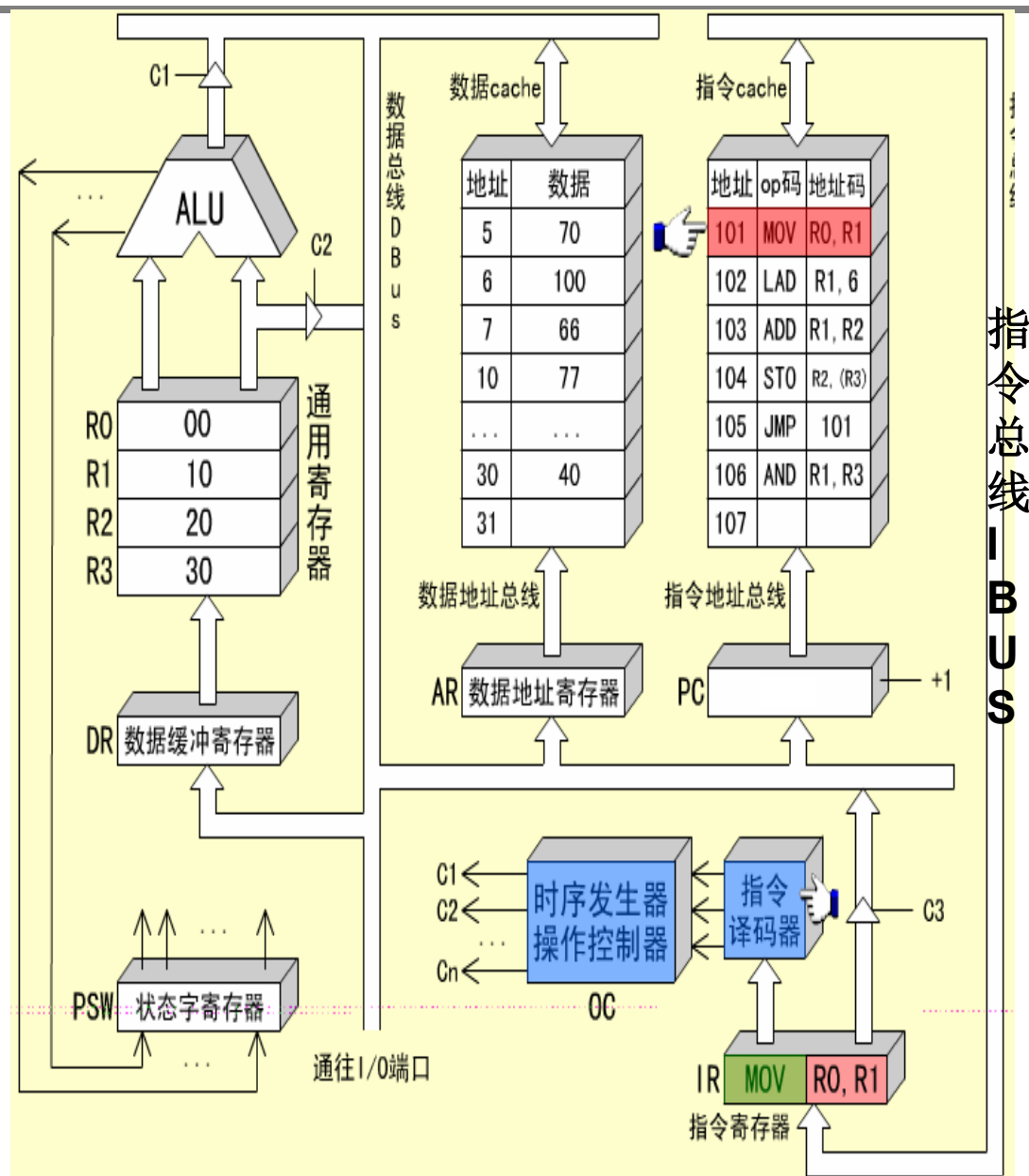
1. 取指周期（略）

2. 执行周期

○ IR → DBUS → PC;

IR读
PC写

5条指令执行过程的动画演示



>>> 5.2.7 用方框图语言表示指令周期

➤在进行中央处理器设计时可以用方框语言来描述一条指令的指令周期

➤方框

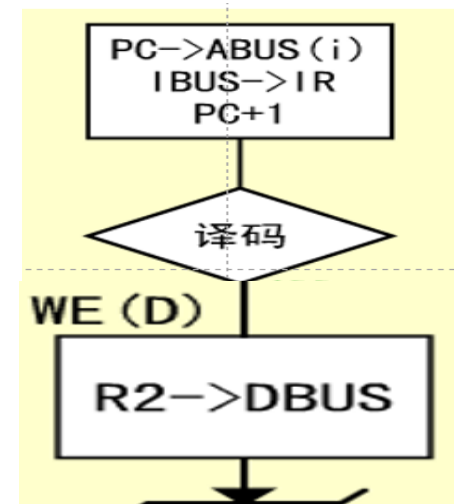
- 一个方框代表一个CPU周期;
- 方框中的内容表示数据通路的操作或某种控制操作。

➤菱形

- 通常用来表示某种判别或测试; 例如指令译码
- 时间上依附于之前一个方框的CPU周期, 而不单独占用一个CPU周期;

➤~ (公操作符号)

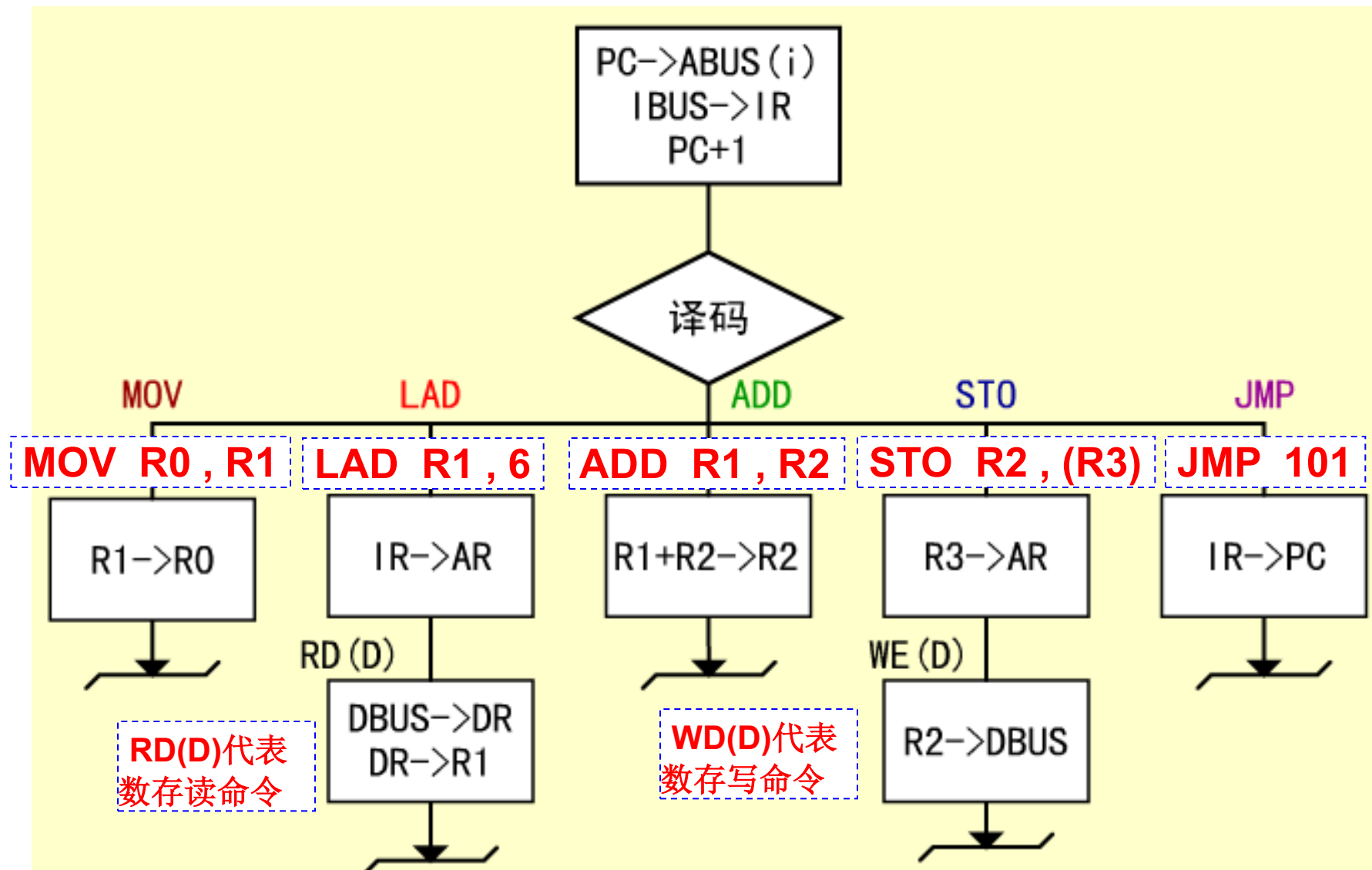
- 表示一条指令已经执行完毕, 转入公操作。
- 所谓公操作就是一条指令执行完毕后, CPU所开始的一些操作, 比如对外围设备请求的处理等。



动画演示:

[方框图语言描述指令周期.swf](#)

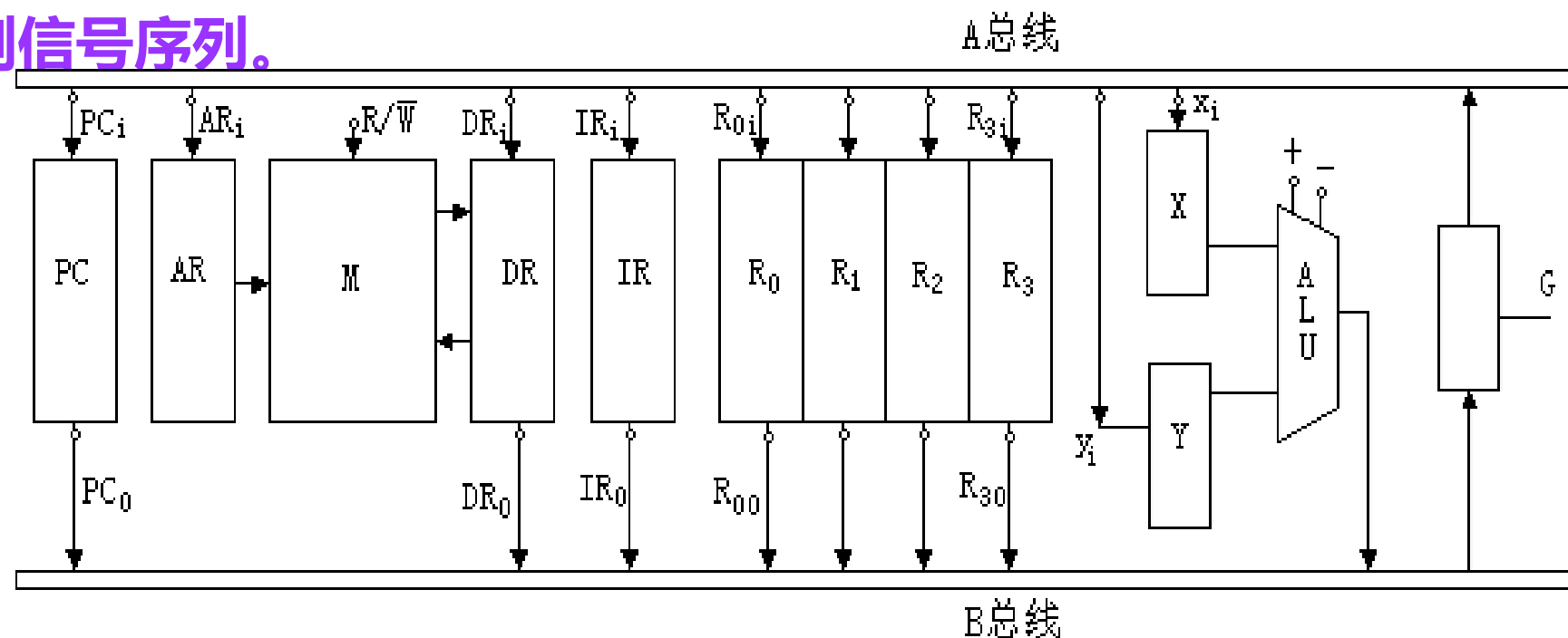
>>>P162 图5.15方框图语言表示的指令周期



>>> 课本P161 【例5.1】

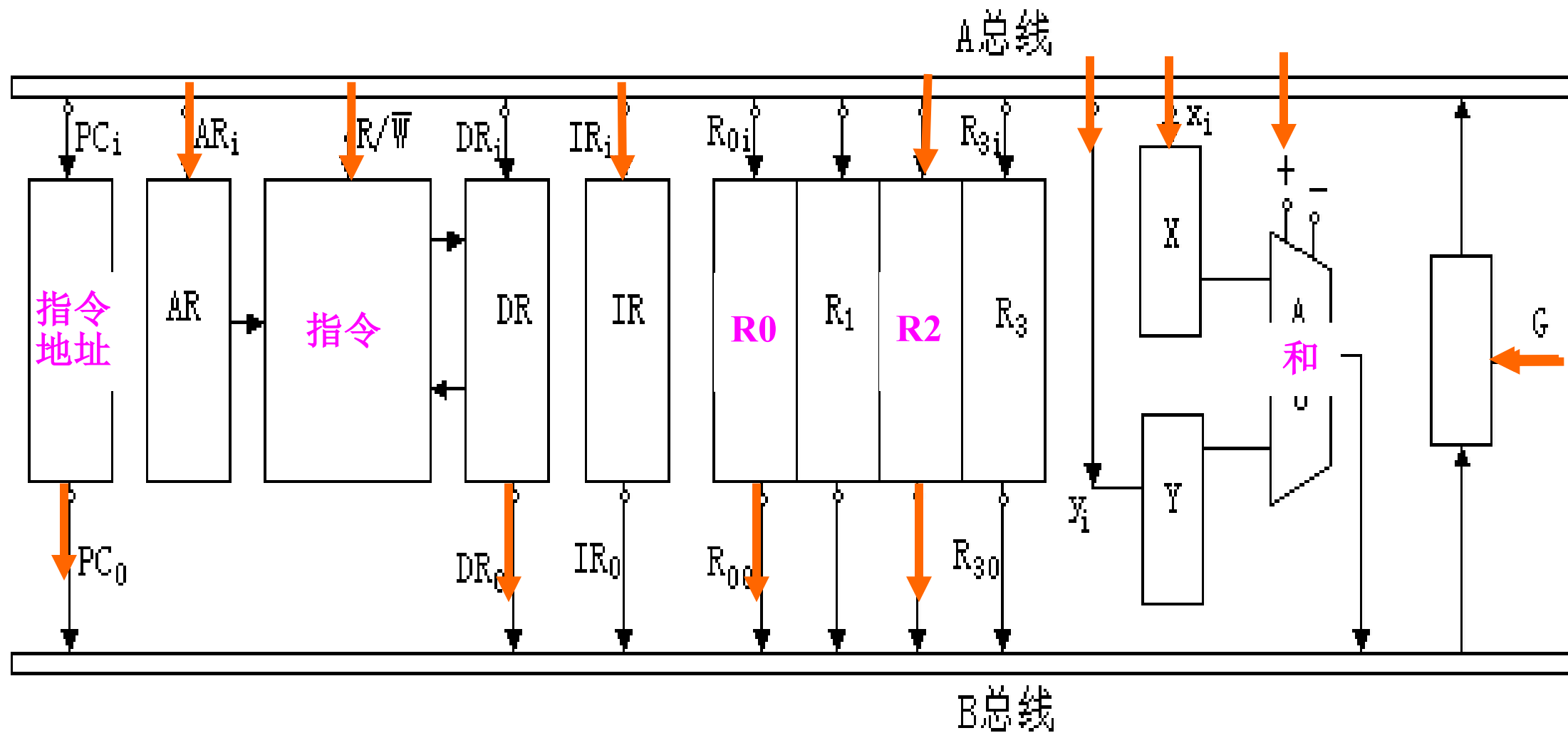
➤ 图5.15所示为双总线结构机器的数据通路，各构成部件如图，线上标注有小圈表示有控制信号，未标字符的线为直通线。

- ① “ADD R2, R0”指令完成 $(R0)+(R2) \rightarrow R0$ 的功能操作，画出其指令周期流程图，并列出相应的微操作控制信号序列。
- ② “SUB R1, R3”指令完成 $(R3)-(R1) \rightarrow R3$ 的操作，画出其指令期流程图，并列出相应的微操作控制信号序列。



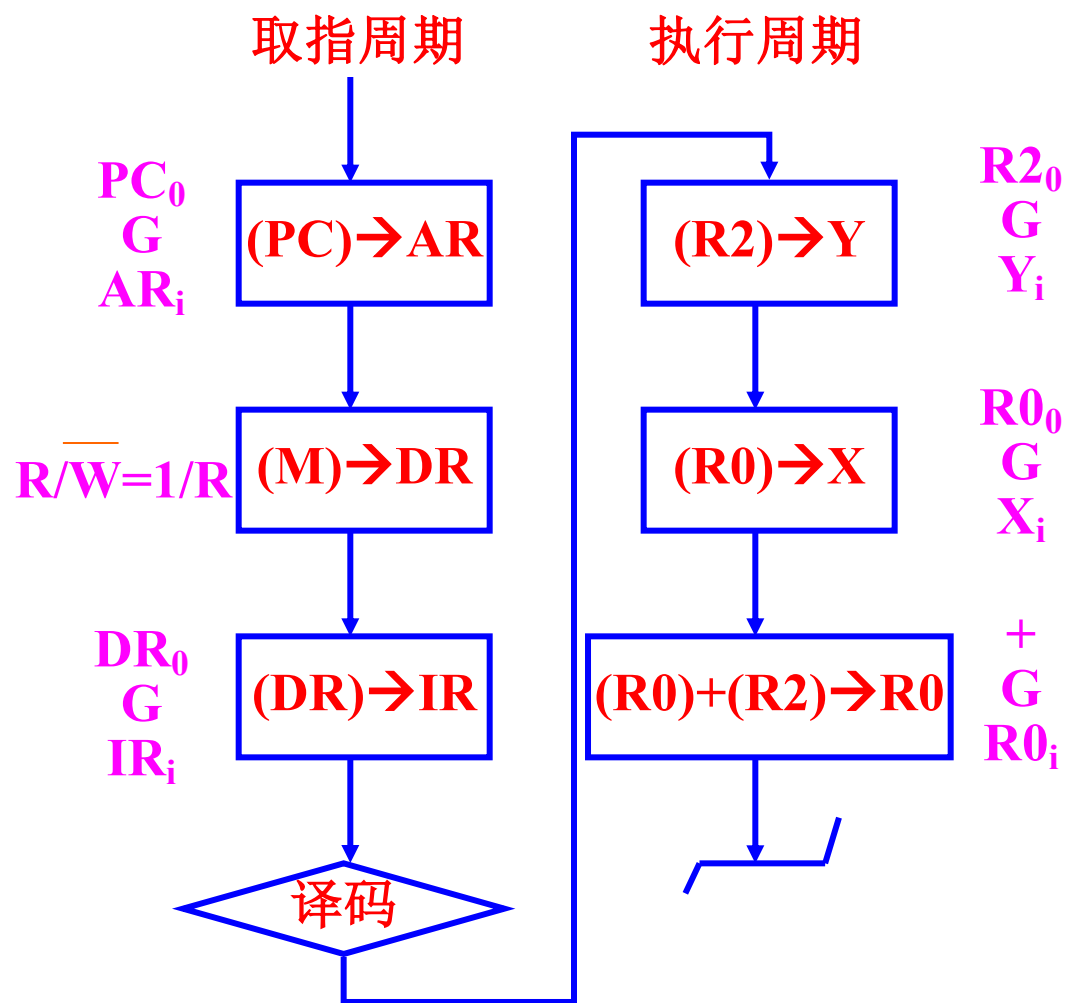
IR为指令寄存器
PC为程序计数器（具有自增功能）
M为主存（受 R/W 信号控制）
AR为地址寄存器
DR为数据缓冲寄存器
R0~R3为寄存器组
ALU由加、减控制信号决定完成何种操作
控制信号G控制的是总线旁路器

>>> (1)分析指令ADD R2, R0的执行过程



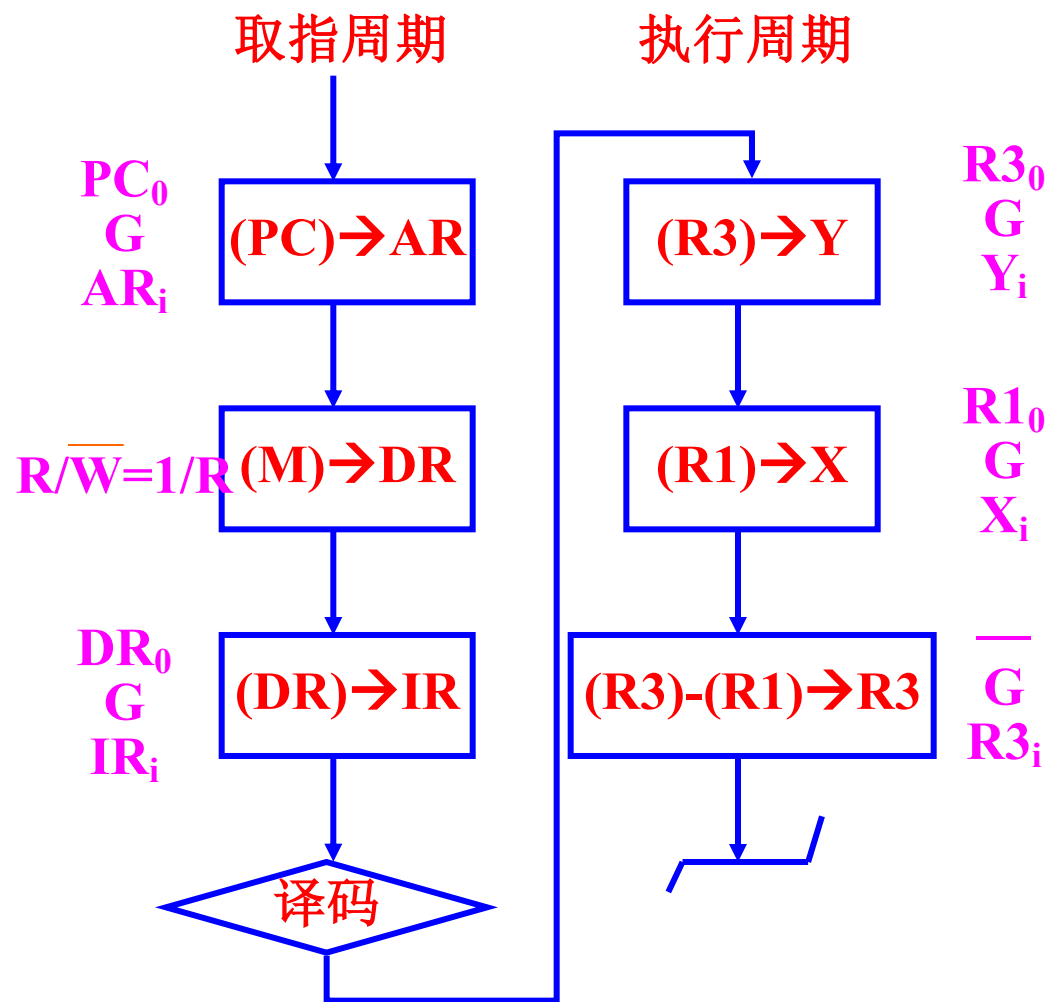
>>> (1) “ADD R2, R0” → (R0)+(R2)→R0

- 指令周期应包括取指周期和执行周期；
- 执行周期中应首先将R0、R2两寄存器的内容送入ALU的操作数缓冲器中，再执行加法运算；



>>> (2) “SUB R1, R3” → (R3) - (R1) → R3

- 指令周期应包括取指周期和执行周期；
- 取指周期与ADD指令完全相同；
- 执行周期与ADD指令不同之处在于ALU的控制信号为“—”





5.3 时序发生器和控制方式

➤ 5.3.1 时序信号的作用和体制

➤ 5.3.2 时序信号发生器

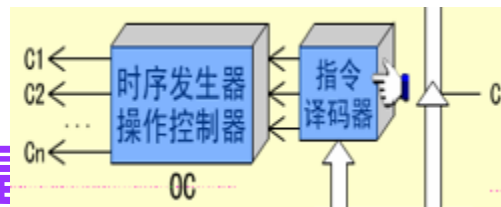
➤ 5.3.3 控制方式

>>> 5.3.1 时序信号的作用和体制

➤ 时序信号的作用

时序信号相当于CPU的时间表

- CPU中的控制器用它指挥计算机准确无误地工作
- CPU通过时序控制(时序信号/周期信息)来识别从内存中取出的是

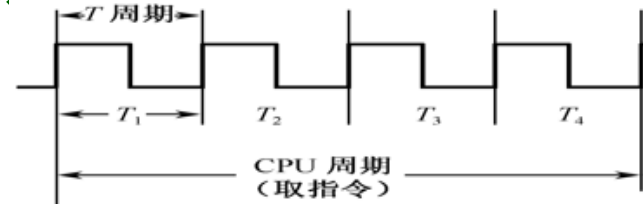


◆ 指令周期的第一个CPU周期——**取指周期**；（从时间判断）

◇ 从存储单元中取出的一定是指令，需要送往IR指令寄存器；（从空间判断）

◆ 指令周期的其他CPU周期——**执行周期**；（从时间判断）

◇ 从存储单元中取出的一定是数据，需要送往运算器。（从空间判断）



- 一个CPU周期中时钟脉冲对CPU的动作有严格的约束（例如4个T周期分别完成取指、中读指令cache、写入IR、 $pc+1$ 、译码）
- 操作控制器发出的各种控制信号是时间（时序信号）和空间（部件操作信号）的函数。

>>> 5.3.1 时序信号的作用和体制

➤ 时序信号包括

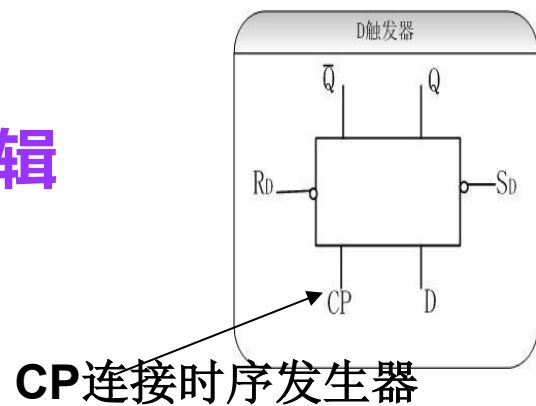
- 时钟信号、使能信号、复位信号等，共同组成数字系统的时序逻辑

➤ 时序信号特点

- 周期性、顺序性、同步性

➤ 时序信号的基本体制（信号怎么组织搭配） --- 电位、脉冲制

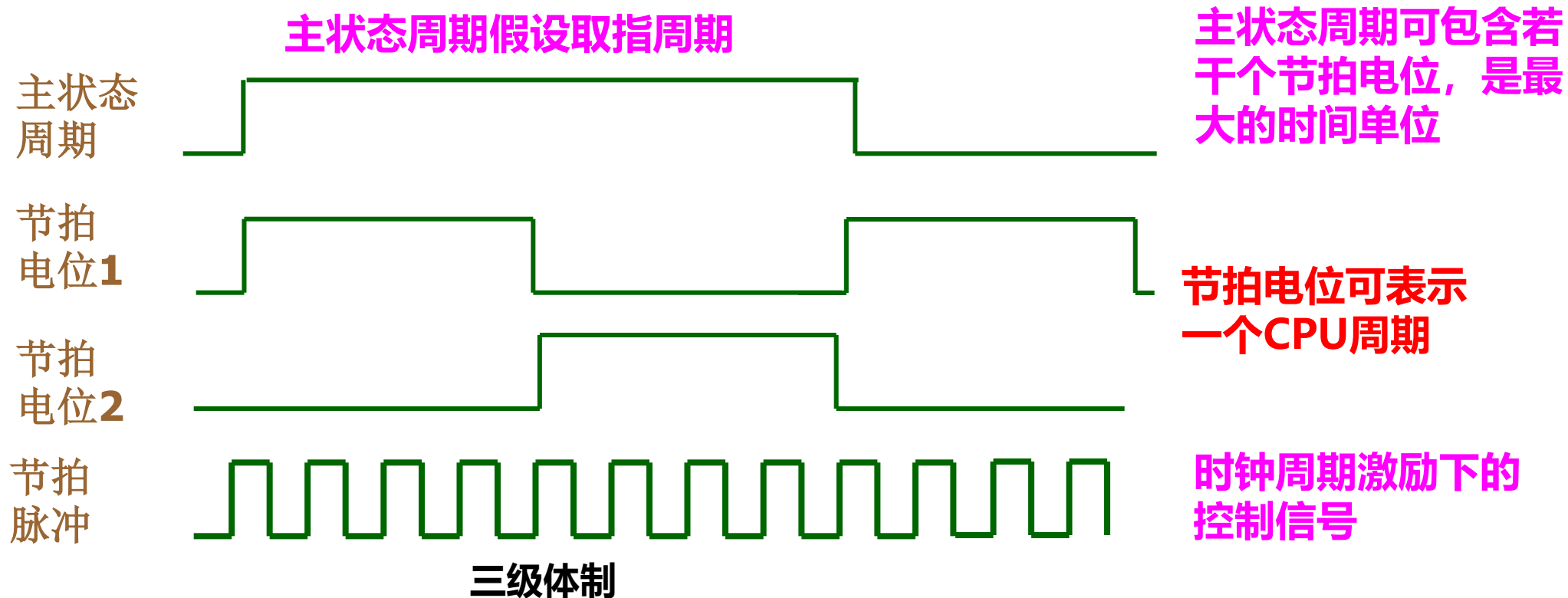
- 电位：用电平的高低进行控制；脉冲：用信号的边沿进行控制
- 计算机硬件的器件特性决定了时序信号的基本体制是电位—脉冲制（以触发器为例）
- 脉冲到达之前，电平信号必须要稳定；



>>> 控制器的时序信号

➤ 硬布线控制器的时序信号

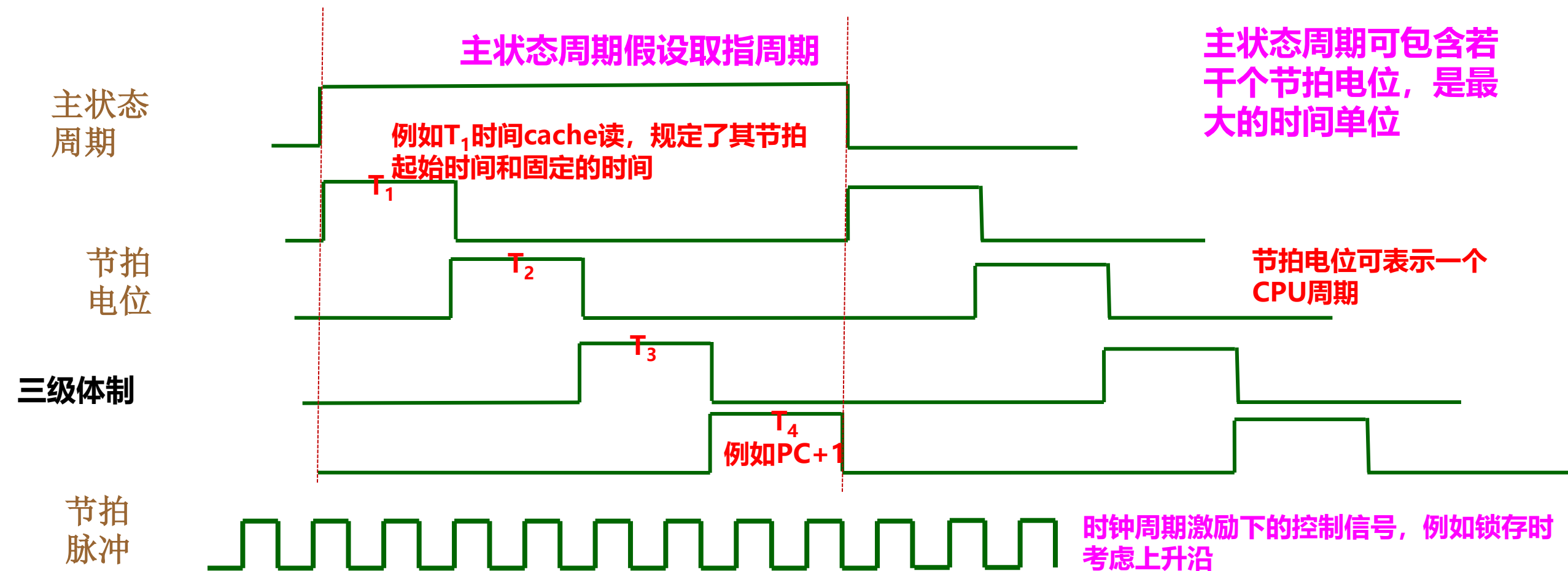
○ 常采用**主状态周期 - 节拍电位 - 节拍脉冲**三级体制。节拍电位=CPU周期



>>> 控制器的时序信号

➤ 硬布线控制器的时序信号

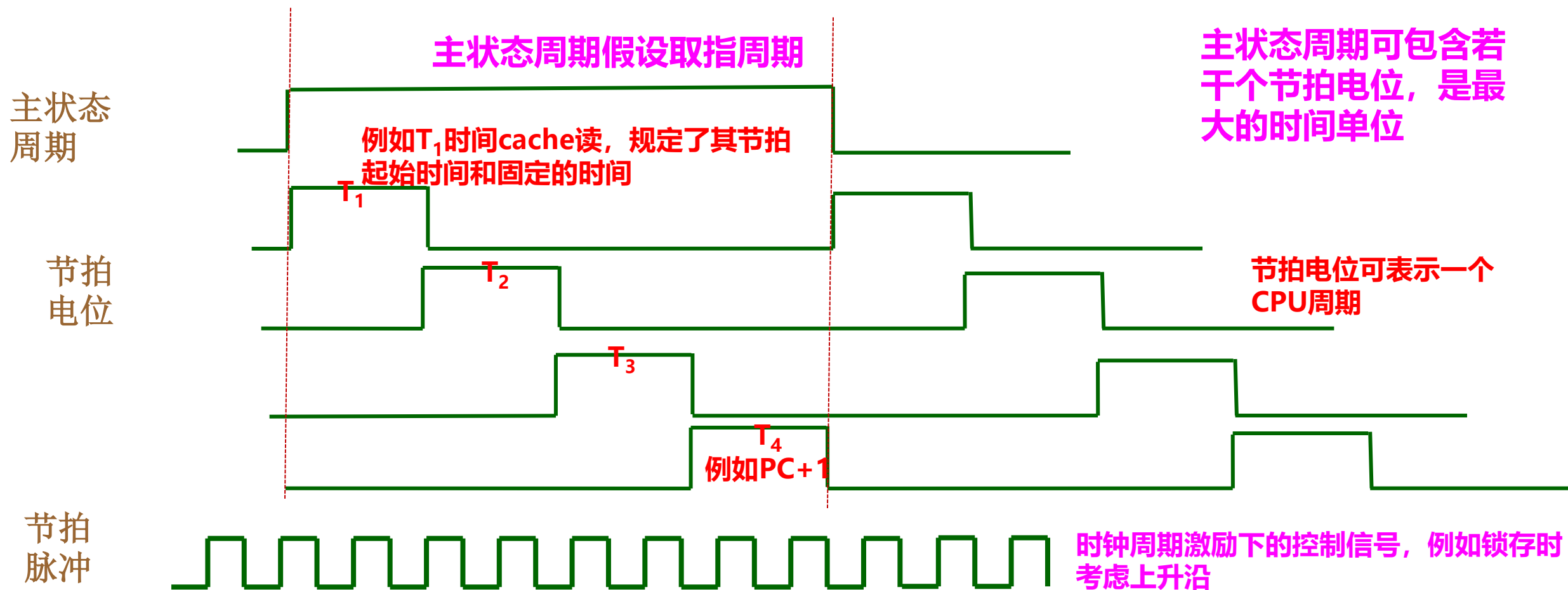
○ 常采用主状态周期 - 节拍电位 - 节拍脉冲三级体制。节拍电位=CPU周期



>>> 控制器的时序信号

➤ 硬布线控制器的时序信号

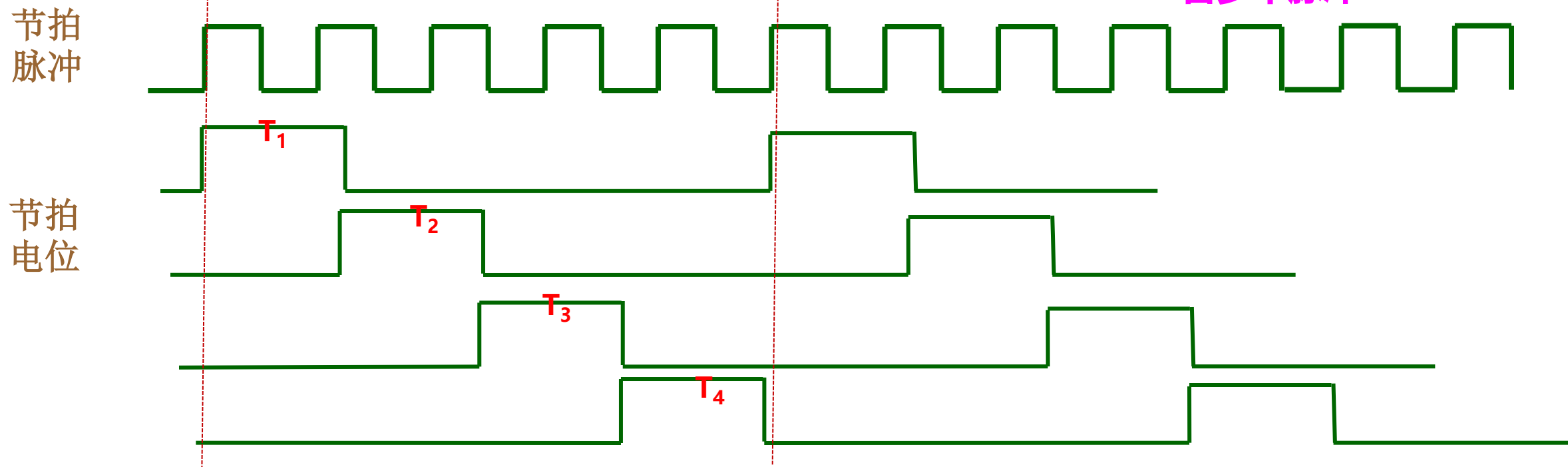
○ 常采用主状态周期 - 节拍电位 - 节拍脉冲三级体制。节拍电位=CPU周期



>>> 控制器的时序信号

➤ 微程序控制器的时序信号

- 一般采用**节拍电位 - 节拍脉冲**二级体制。
- 利用微程序顺序执行来实现微操作。



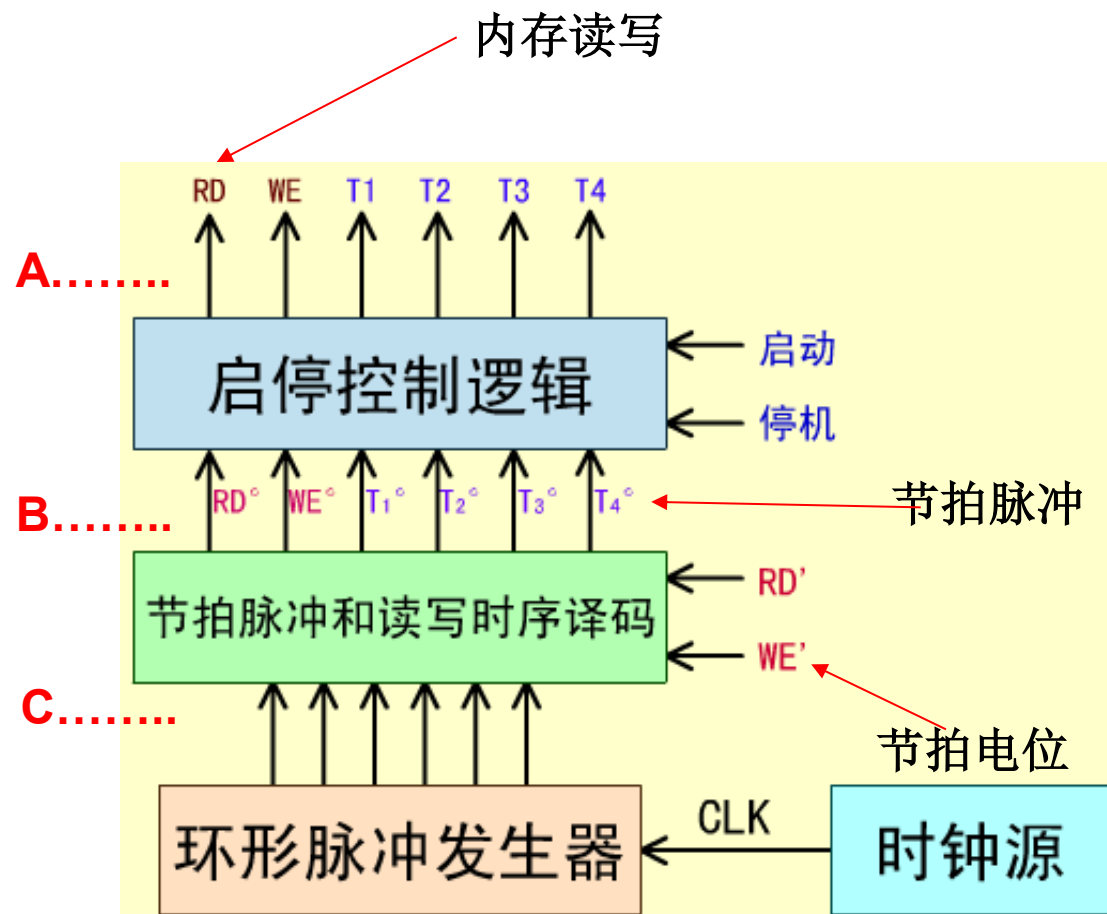
>>> 5.3.2 时序信号发生器

➤时序信号发生器功能：产生时序信号，用逻辑电路实现时序的控制；

- 各型计算机产生时序电路不相同
- 大、中型计算机时序电路复杂，微型计算机的时序电路简单

➤微程序控制器中时序信号发生器的组成：

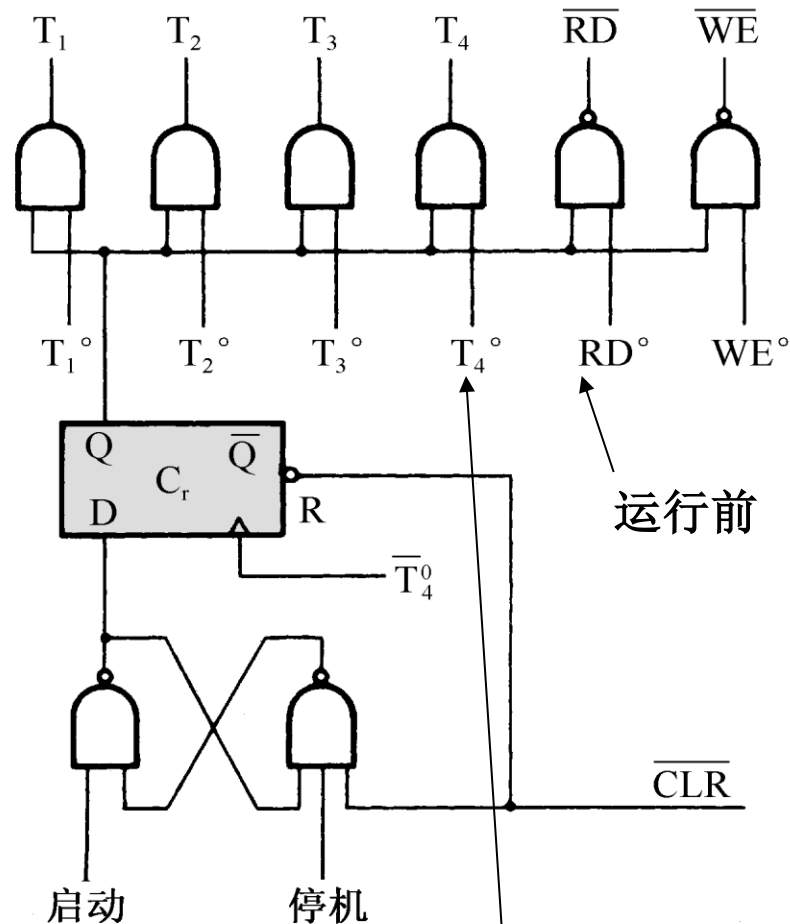
- **时钟源：** 提供稳定的基本方波时钟信号（信号时间固定、定时）；
- **环形脉冲发生器：** 产生一组有序的间隔相等或不等的脉冲序列，以便通过译码电路产生需要的节拍脉冲的部件；
- **节拍脉冲和读写时序译码逻辑**
环形脉冲与节拍脉冲共同作用产生各控制信号的时序（有先后顺序-时序）；
- **启停控制逻辑**



5.3.2 时序信号发生器

► 启停控制逻辑

- 机器一旦接通电源，就会自动产生原始的节拍脉冲信号 $T_1^0 \sim T_4^0$ ，但是只有在启动机器运行的情况下，才允许时序发生器发出CPU工作时所需的节拍 $T_1 \sim T_4$ ，因此需要由启停控制逻辑来控制原始信号 $T_1^0 \sim T_4^0$ 的发送
- 由于启动、停机是随机的，对读/写时序信号也需要由启停逻辑加以控制

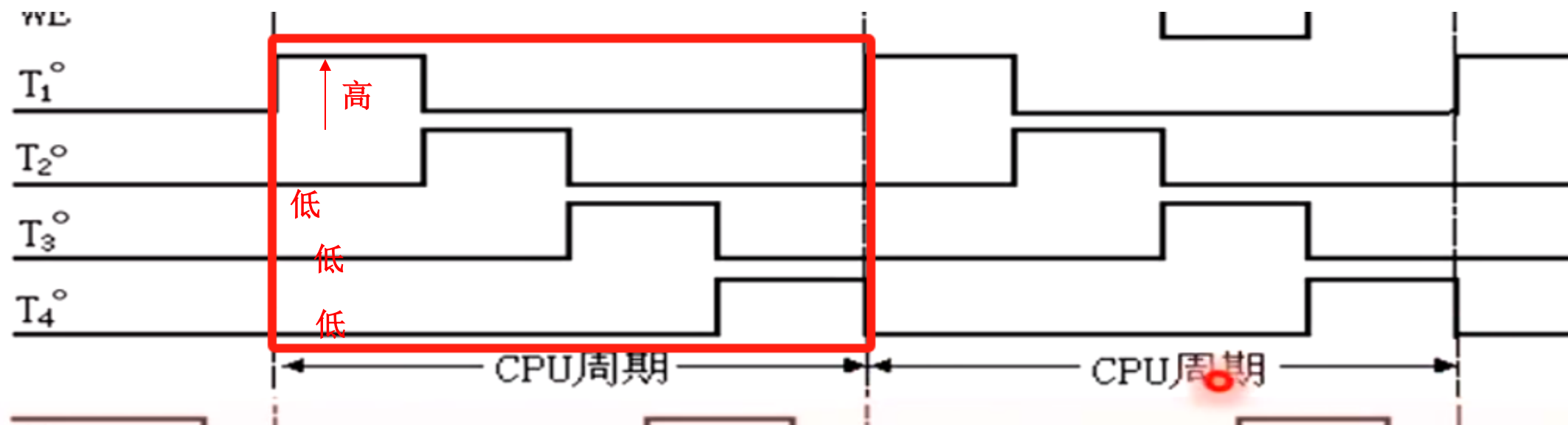


原始节拍脉冲，一加上电就自动产生

>>> 5.3.2 时序信号发生器

➤ 启停控制逻辑

- 由于启动、停机是随机的，启动控制逻辑必须符合：启动时，一定从第1个节拍脉冲前沿开始工作，而停机时一定要在第4个节拍脉冲结束后关闭时序发生器。只有这样才能发送出去的脉冲都是完整的脉冲。





CPU时钟信号

A 处输出信号图

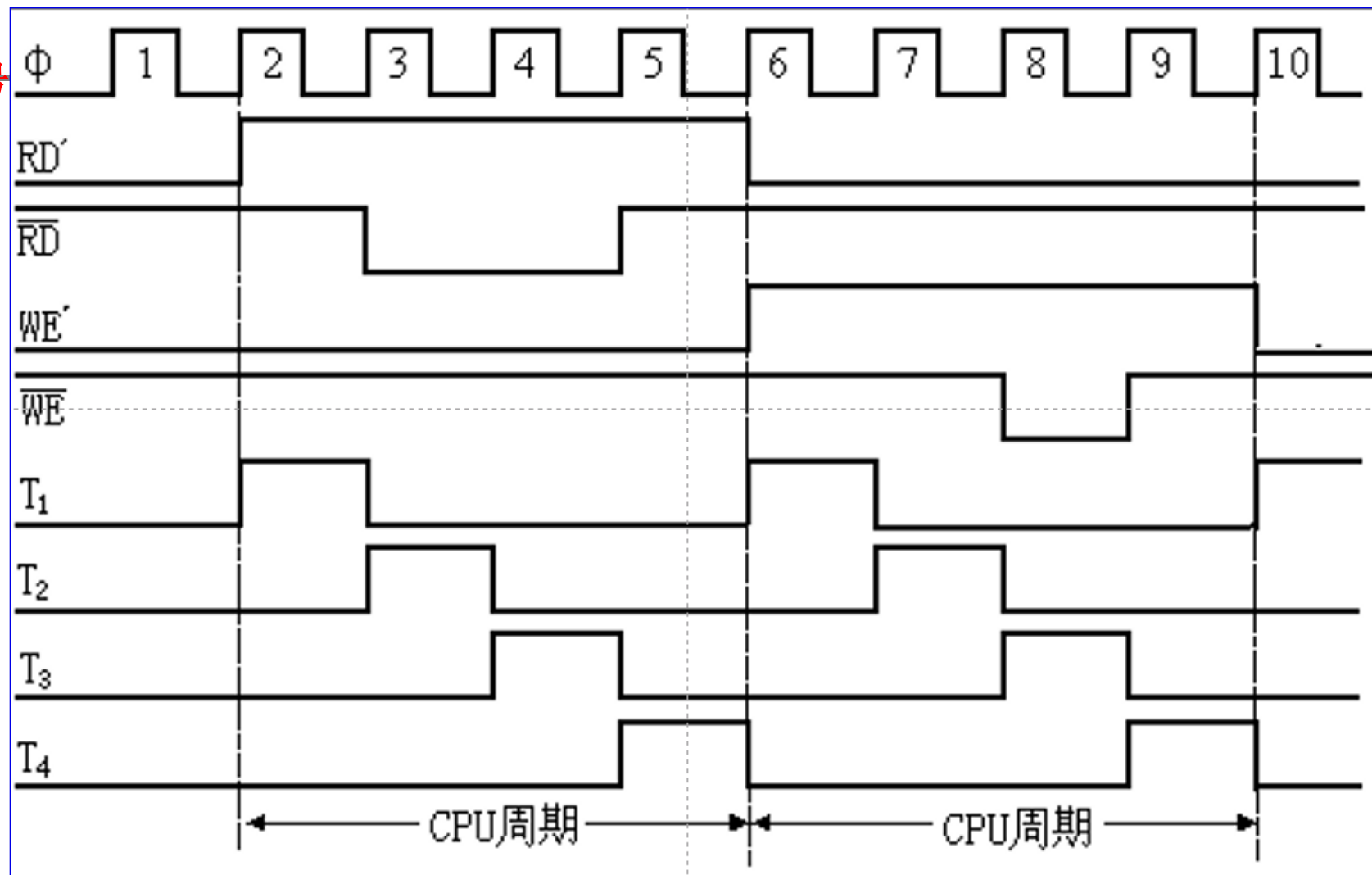
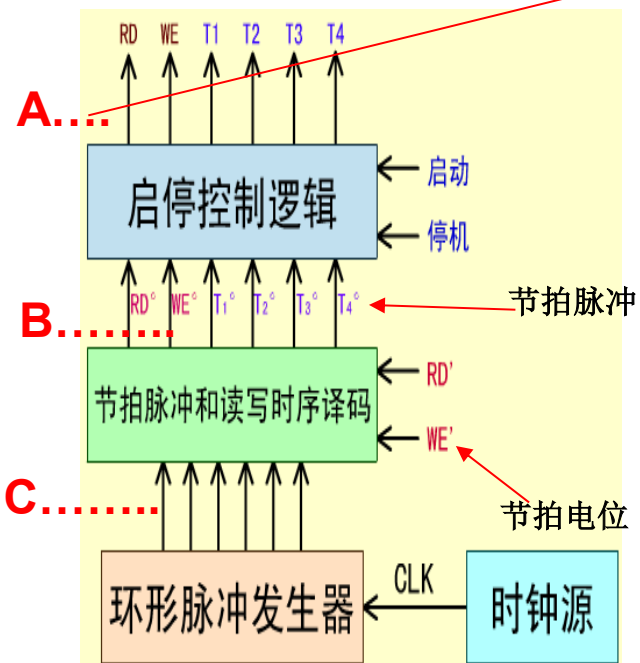


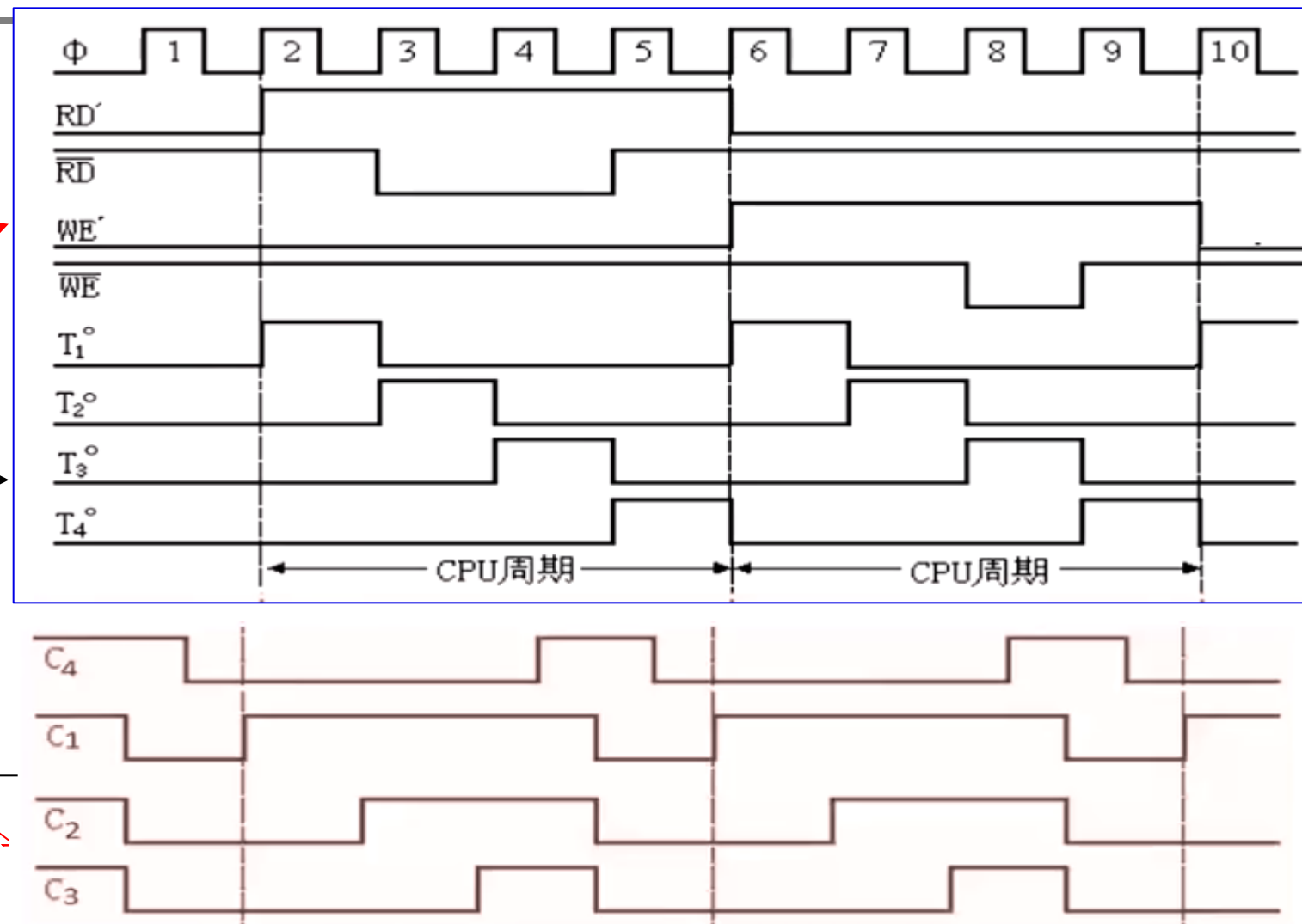
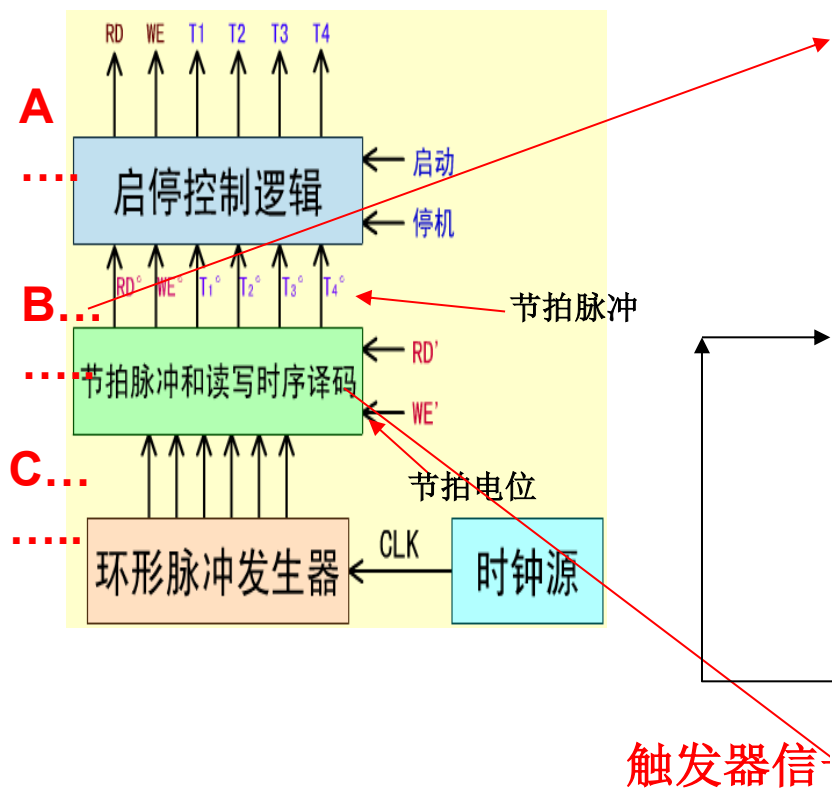
图 5.19 节拍电位与节拍脉冲时序图—存储器读写时序

1、假定在一个CPU周期中产生四个等间隔的**节拍脉冲** $T_1 \sim T_4$ ，每个节拍脉冲的脉冲宽度均为**200ns**，因此一个CPU周期便是**800ns**，在下一个CPU周期中，它们又按固定的时间关系重复。



CPU时钟信号

B 处输出信号图



RD' 、 WE' 信号来自微程序控制器的控制信号，是持续时间为一个CPU周期的节拍电位信号， T_i° 由 C_i 电路组合生成

>>> 5.3.3 控制方式

- 机器指令所包含的**CPU**周期数反映了指令的复杂程度---控制信号多少
- 每个操作信号所需的时间及出现的次序各不相同。
- 形成控制不同操作序列的时序信号的方法，称作**控制器的控制方法为控制方式**。
- 控制方式：实质反映了时序信号的定时方式。
- 分为以下几种：
 - **同步控制方式（时钟）**
 - **异步控制方式（应答）**
 - **联合控制方式**

>>> 5.3.3 控制方式

➤ 同步控制方式（时钟）

- 每条指令每次执行时的CPU周期数和时钟周期数固定不变

➤ 可选取三种方案实现：

- 完全统一的机器周期执行各种不同的指令（对简单指令和简单操作造成浪费）
- 采用不定长机器周期
 - ◆ 将大多数操作安排在一个较短的机器周期内完成。对某些时间紧张的操作则采取延长机器周期的方法解决。
- 中央控制于局部控制的结合。
 - ◆ 大部分指令安排在固定的机器周期内完成（中央控制）。
 - ◆ 少数复杂指令采用另外的时序控制（局部控制）。

>>> 同步控制方式（中央控制方式）

➤ 同步控制方式的特点

- ① 以指令周期最长的指令为标准，确定指令执行的节拍数；指令周期短的指令则不合理。
- ② 控制器产生统一的、顺序固定的、周而复始的节拍电位和工作脉冲；
- ③ 简单指令（指令周期短的指令）可空着一部分节拍不用。也就是说，不管什么指令，实现的时间都是相同的。

同步方式应用场合：

- 用于CPU内部、设备内部、系统总线操作
- 各挂接部件速度相近，传送时间确定，传送距离较近

➤ 优点

- 控制电路简单，因为时钟统一；

➤ 缺点

- CPU有空闲时间，指令的整体运行速度较慢。

>>> 5.3.3 控制方式

➤ 异步控制方式（应答）

- **特点：**每条指令需要多长时间就占多长时间
- **实现：**不存在统一基准时钟信号，每个指令周期包含的机器周期数目由联络信号控制（例如应答方式，各个操作之间的衔接由“结束-起始”信号来实现，额外信号），不固定（每个部件按照自身的速度进行工作）。

➤ 异步式应用场合

- 用于异步总线操作或者不同功能部件之间（CPU—内存）
- 各挂接部件速度差异大，传送时间不确定，传送距离较远

➤ 联合控制方式（两种情况）

- ① **大部分操作序列为固定的机器周期**，对某些时间难以确定的操作则以执行部件的“回答”信号(异步式)作为本次操作的结束（采用异步方式）。
- ② **机器周期的节拍脉冲数固定**，但是各条指令周期的机器周期数不固定（微程序控制器采用）。

➤ 异步控制方式的特点

- ① 每条指令需要多少节拍，就产生多少节拍；
- ② 指令执行完毕，发出回答信号；
- ③ 控制器收到回答信号即开始执行下条指令。

➤ 优点

- ① 异步控制采用不同时序，运行速度快；

➤ 缺点

- ① 控制电路比较复杂。

>>> 5.4 微程序控制器

5.4.1 微程序控制原理

5.4.2 微程序设计技术

>>> 5.4.1 微程序控制原理

➤ 基本思想

- 仿照通常的解题程序的方法，把操作控制信号编成所谓的“微指令”，存放到一个只读存储器里；
- 当机器运行时，一条一条地读出这些微指令，从而产生各种操作控制信号，使部件执行规定的操作。

➤ 要解决的问题：如何将一条机器指令编辑成微指令集合或微操作集合（多个微操作可能对应一条微指令）

➤ 微程序设计技术是利用软件方法来设计硬件的一门技术。

- 具有规整性、灵活性、可维护性等一系列优点；
- 逐渐取代了早期的组合逻辑控制器，已被广泛地应用。

>>> 举例说明微程序控制思想

- 通过对右表中的指令分析，可得以下CPU周期的控制信号

① 取指：PC读、指令Cache读、IR写、PC增量

② 间址：IR读、AR锁存

③ 间址：R3读、AR锁存

④ 执行：R1读、ALU传送控制、ALU输出、DR锁存、R0写

⑤ 执行：数据Cache读、DR锁存、R1写

⑥ 执行：R1读、R2读、ALU加、ALU输出、DR锁存、R2写

⑦ 执行：R2读、数据Cache写

⑧ 执行：IR读、PC写

- 指令LAD R1 , (R3)可经历①③⑤步骤完成；

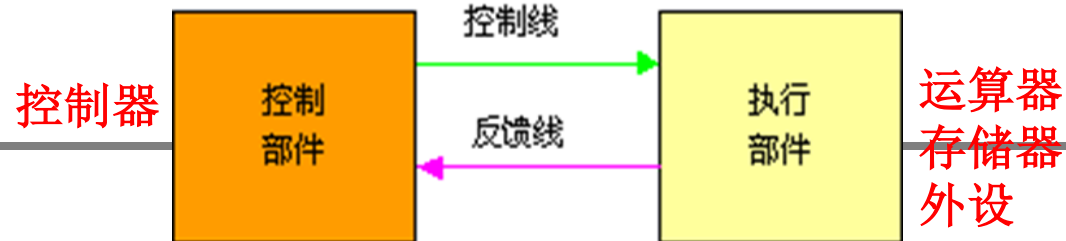
- 指令STO R2 , 6可经历①②⑦步骤完成。

地址	指令
①④	MOV R0 , R1
①②⑤	LAD R1 , 6
①⑥	ADD R1 , R2
①③⑦	STO R2 , (R3)
①⑧	JMP 101

>>> 1、微命令和微操作

➤ 控制部件与执行部件之间存在两种联系

- 控制部件向执行部件发出控制信号（各种控制命令—微命令）；
- 执行部件向控制部件返回状态信息；



控制信号线

反馈信息

➤ 状态测试

- 执行部件通过反馈线向控制部件反映当前操作状态，以使控制部件决定下一步的新微命令；

➤ 微命令

- 控制部件通过控制线向执行部件发出的各种控制命令；它是构成控制序列的最小单位
- 微命令是控制计算机各部件完成某个基本微操作的命令。
 - ◆ 例如：打开或关闭某个控制门的电位信号、某个寄存器的打入脉冲等。

➤ 微操作：是微命令的操作过程。

- 执行部件接受微命令后所执行的操作；
- 微命令和微操作是一一对应的。
- 微操作是执行部件中最基本的操作。
- 微命令是微操作的控制信号，微操作是微命令操作过程。

执行部件中的最基本的操作

>>> 微操作的分类

➤ 由于数据通路的结构关系，微操作可分为相容的和互斥的两种：

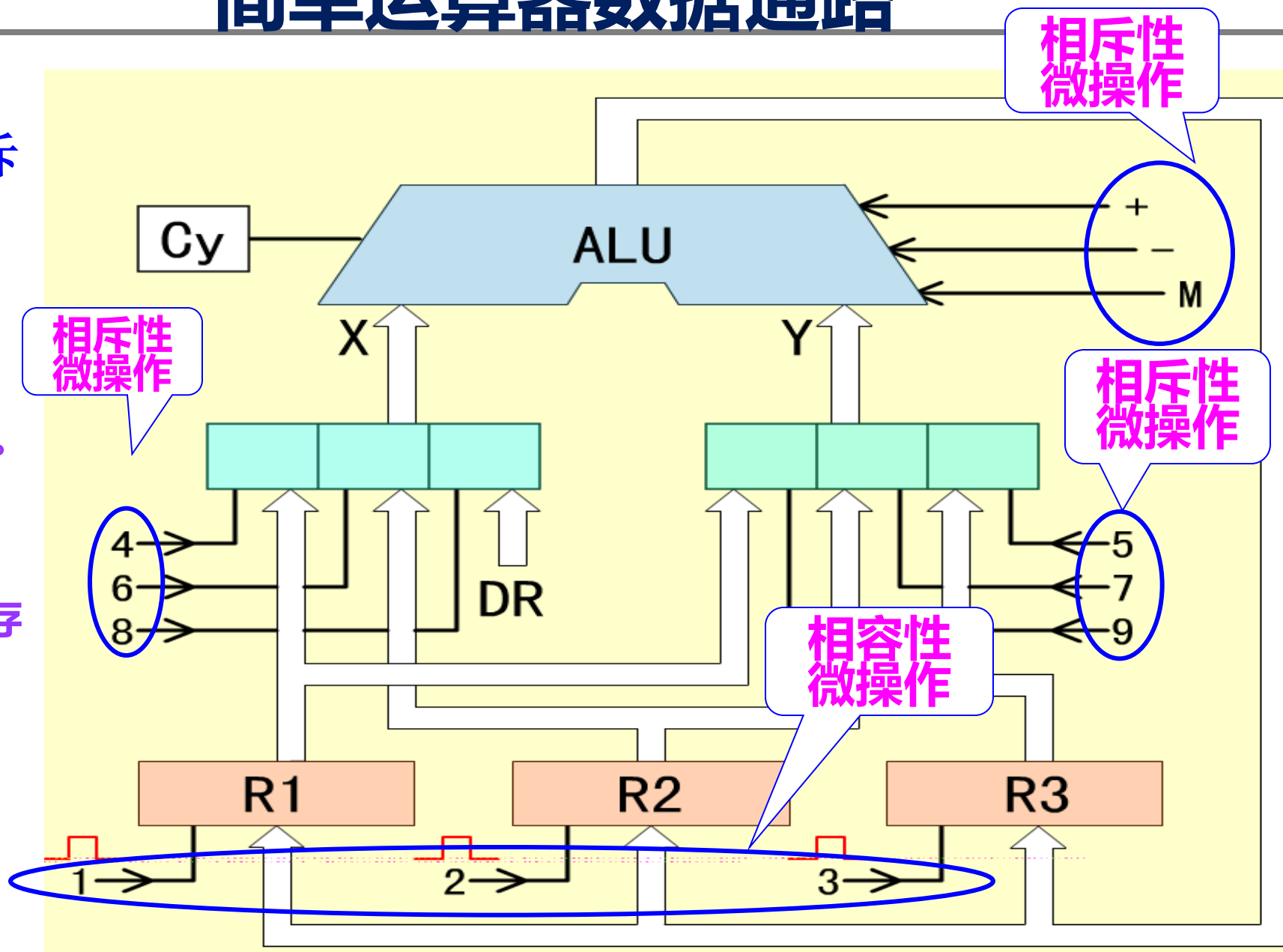
➤ 相斥性微操作

- 是指不能同时或不能在同一个节拍内并行执行的微操作。可以编码
- 例如取指周期中
($(PC) \rightarrow MAR; 1 \rightarrow R$ (内存读命令))

➤ 相容性微操作

- 是指能够同时或在同一个节拍内并行执行的微操作。必须各占一位

简单运算器数据通路





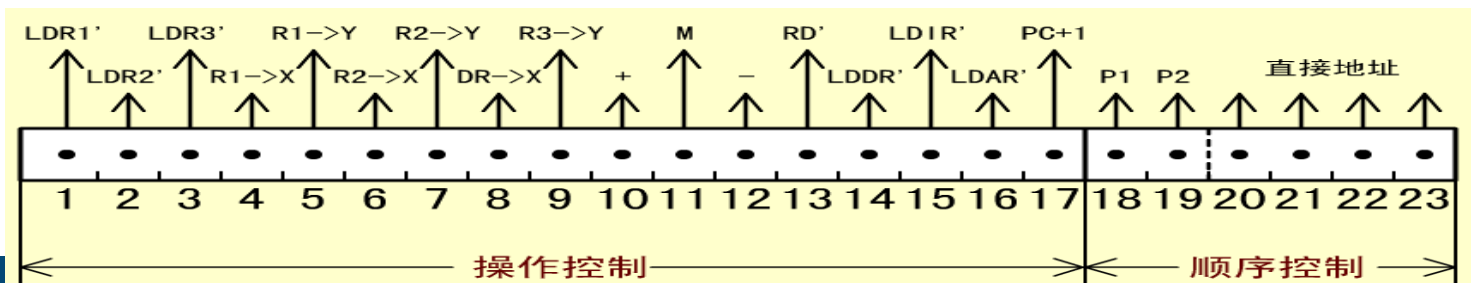
2、微指令和微程序

➤ 微指令

- **一个CPU周期内**，一组实现一定操作功能的微命令的组合，构成一条微指令
- 它是微命令的组合，微指令存储在控制器中的控制存储器

➤ 微指令一般包含操作控制和顺序控制两大部分信息

- **操作控制字段**：又称微操作码字段，用以产生某一步操作所需的各个微操作控制信号；
 - ◇ 某位为1，表明发微指令
 - ◇ 微指令发出的控制信号都是节拍电位信号，持续时间为一个CPU周期
 - ◇ 微命令信号还要引入时间控制
- **顺序控制字段**：又称微地址码字段，用于控制产生下一条微指令的地址；
- 所有的微指令都存放于控制存储器中，使用**地址访问**；



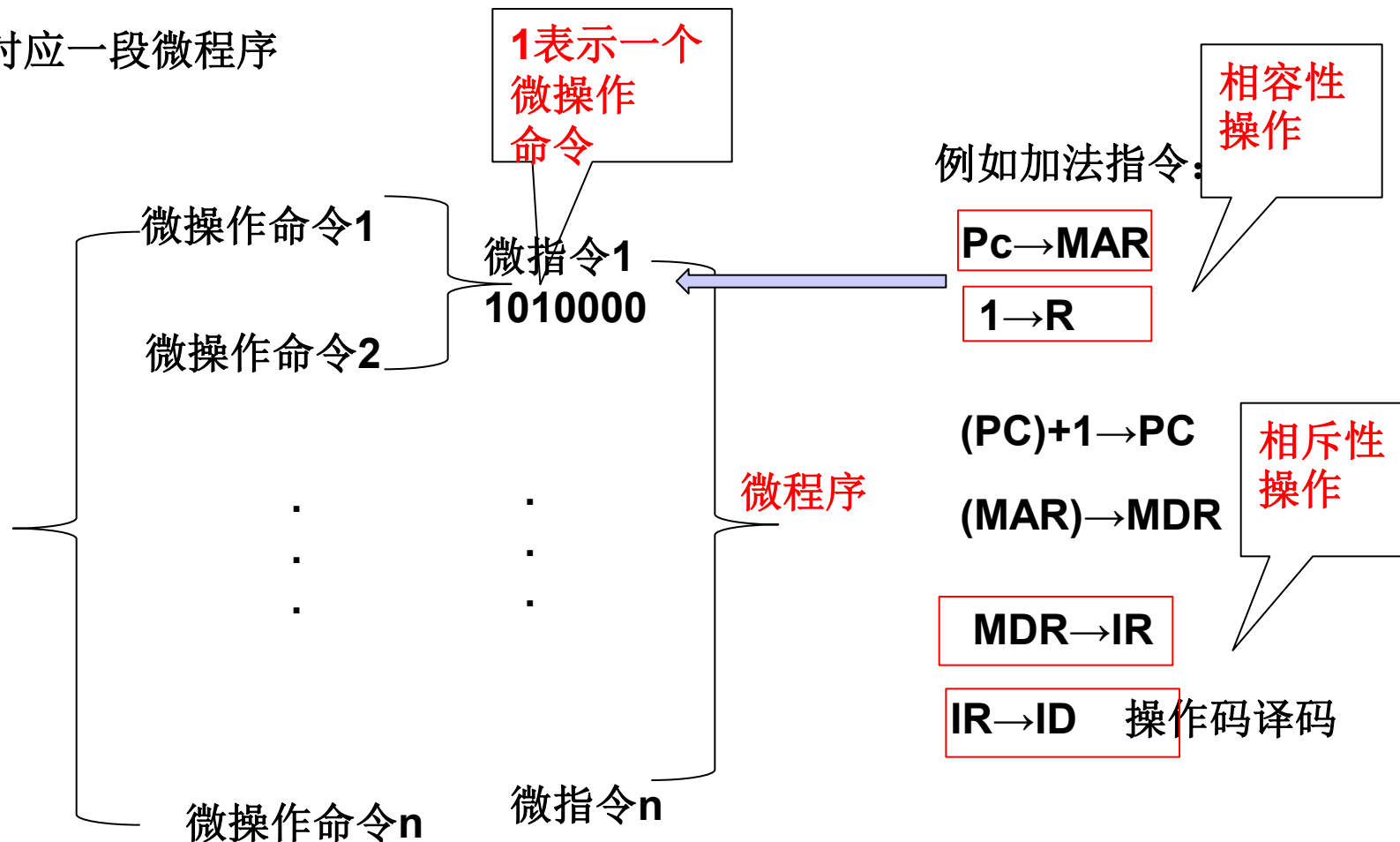
微地址

>>> 机器指令、微程序、微指令、微命令

一条机器指令对应一段微程序

➤ 执行一条机器指令

➤ 微程序

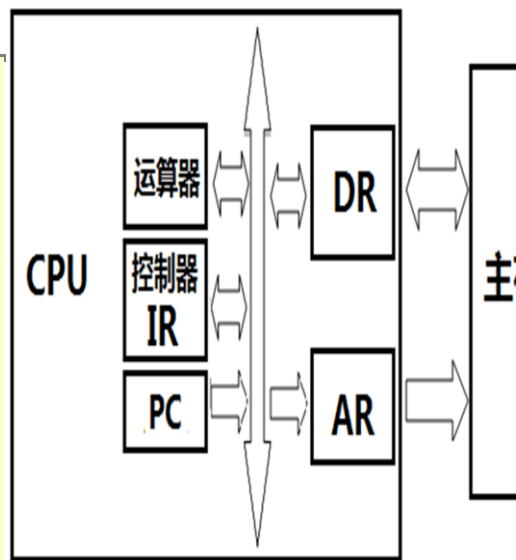
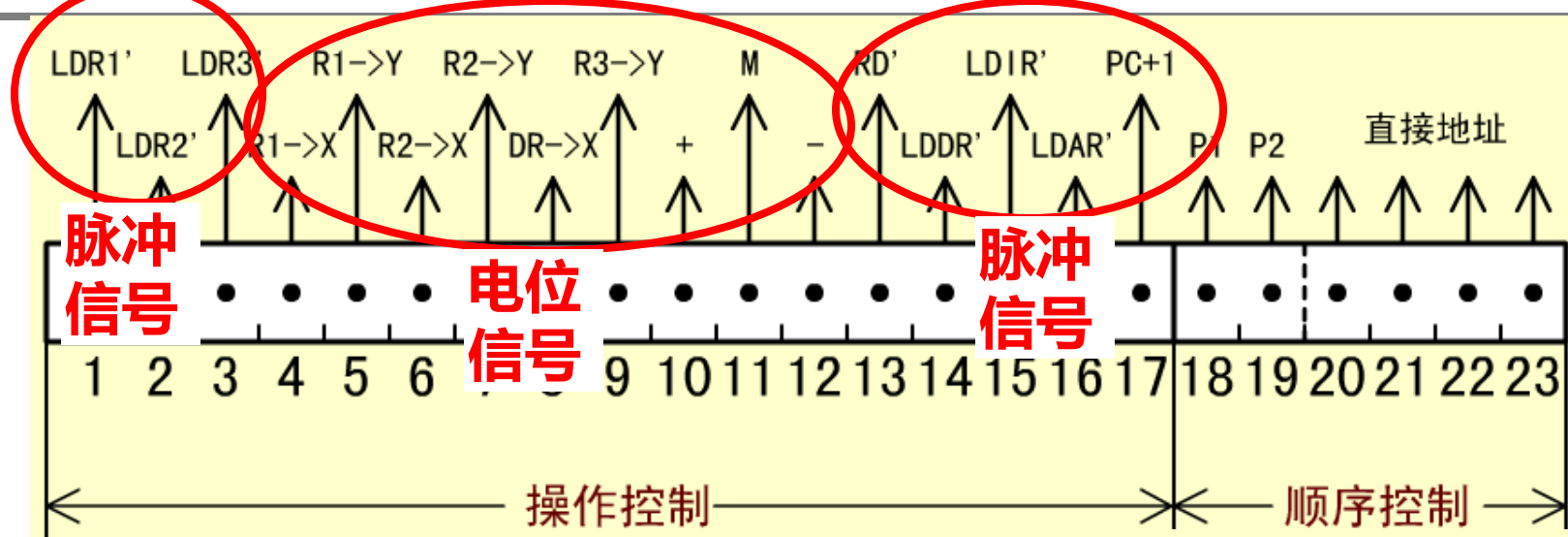


○ 一系列微指令的有序集合就是微程序，每条机器指令都对应着一段微程序；

◆ 微地址：存放微指令的控制存储器的单元地址

◆ 一段微程序对应一条机器指令。

>>> 微指令基本格式



➤ **P₁、P₂为判别测试标志;**

○ 若为00, 则顺序寻址微指令;

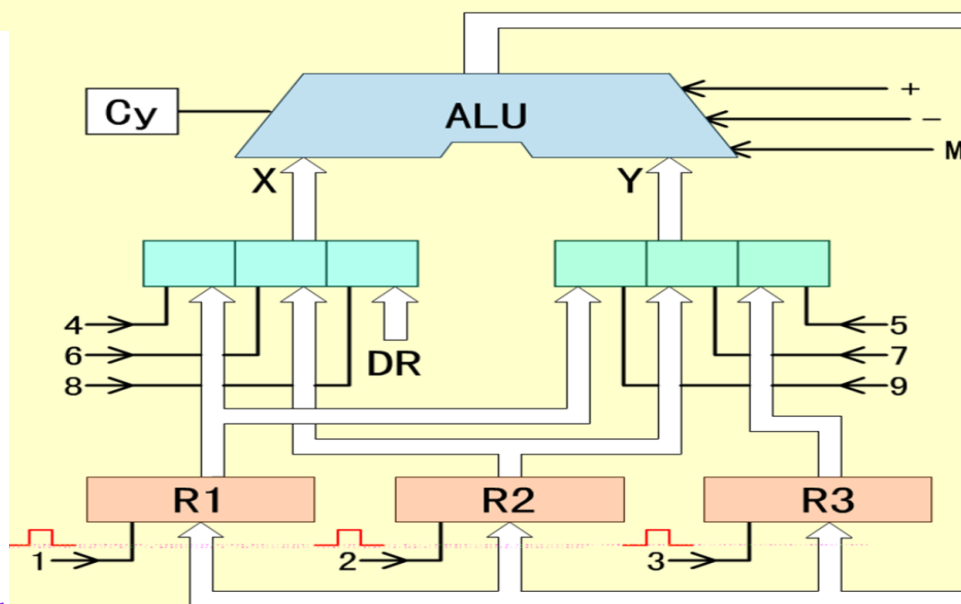
○ 若为01、10, 则跳跃寻址微指令;

◆ 这里P₁、P₂仅是一个例子, 具体的系统判别位不同;

➤ **直接地址20~23位**

○ 顺序寻址时, 直接地址即为下一条微指令地址;

○ 跳跃寻址时, 则以直接或相对寻址方式得到下条微指令地址;



微指令格式对照的通路

>>> 微指令基本格式

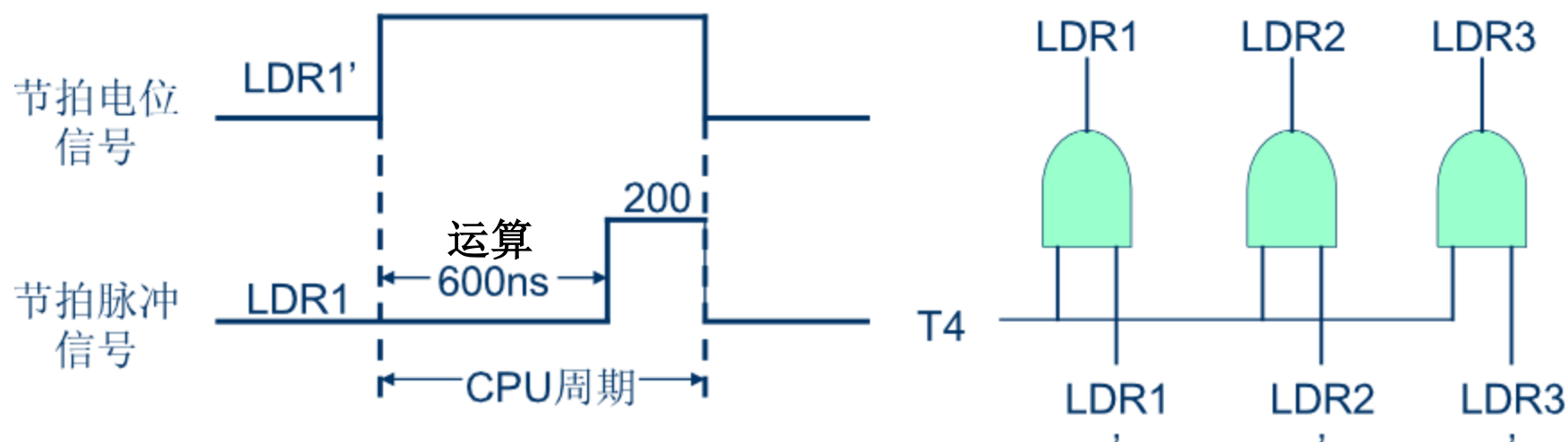
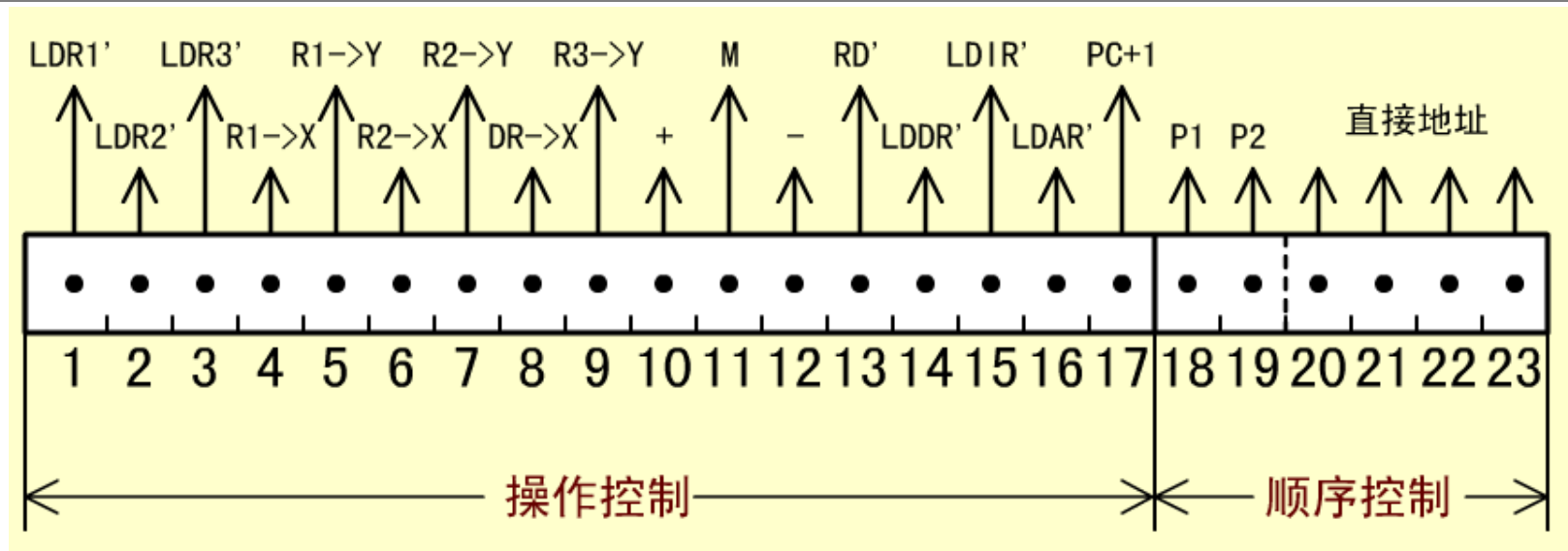


图5.23 运算器操作时序与产生逻辑

>>> 3、微程序控制器原理框图

➤ 控制存储器（ μCM ），微程序控制器的核心部件，存放微程序。

- 存放实现指令系统所需要的全部微指令；
- 由只读存储器构成，要求速度快，读出周期短。

➤ 微指令寄存器(μIR)

- 存放当前正在执行的一条微指令；它的位数同微指令字长相等。
- 由微地址寄存器和微命令寄存器两部分构成；

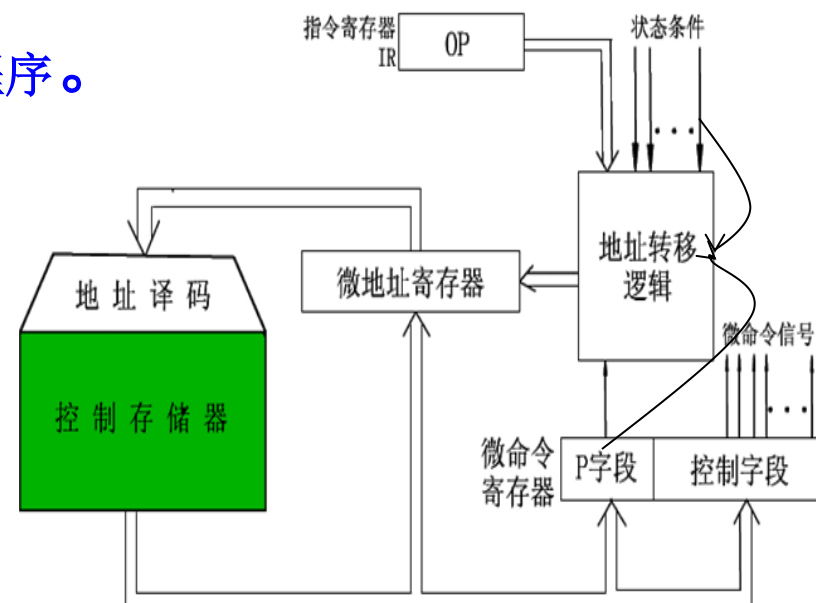
◆ 微地址寄存器：用来产生初始微地址和后继微地址，以保证微指令的连续执行。

◆ 微命令寄存器(μMAR)：保存一条微指令的操作控制字段和判别测试字段的信息，它接受微地址形成部件送来的微地址，为下一步从 μCM 中读取微指令作准备。

◆ 一个CPU周期

➤ 地址转移逻辑

- 跳跃寻址微指令时，负责微地址的修改。比如结束当前指令的最后一条微指令，转向下一条机器指令的第一条取指微指令





微程序控制器的工作过程

- “取指微指令”为所有指令的公用微指令；
 - 通常放在控制存储器中的“0”地址单元；
 - 所有机器指令的最后一条微指令的直接地址都指向0地址单元，用以取下一条微指令；
- 1、机器指令操作码译码，转入执行周期对应微程序（自动）；
 - 操作码译码，通过微地址形成部件产生出该机器指令所对应的微程序的入口地址，即P1测试，将所要执行微指令的地址送入微地址寄存器；
- 2、从μCM逐条读取微指令并执行，每条微指令都能自动产生下一条微指令的地址。
 - 一般顺序寻址，直接使用下址字段，寻址后续微指令；
 - 需要跳跃寻址的，经P2条件测试，修正微指令的下址。
- 3、执行完对应于一条机器指令的微程序后，返回到取指微指令，不断重复，直至程序执行完毕。

>>> 4、微程序举例

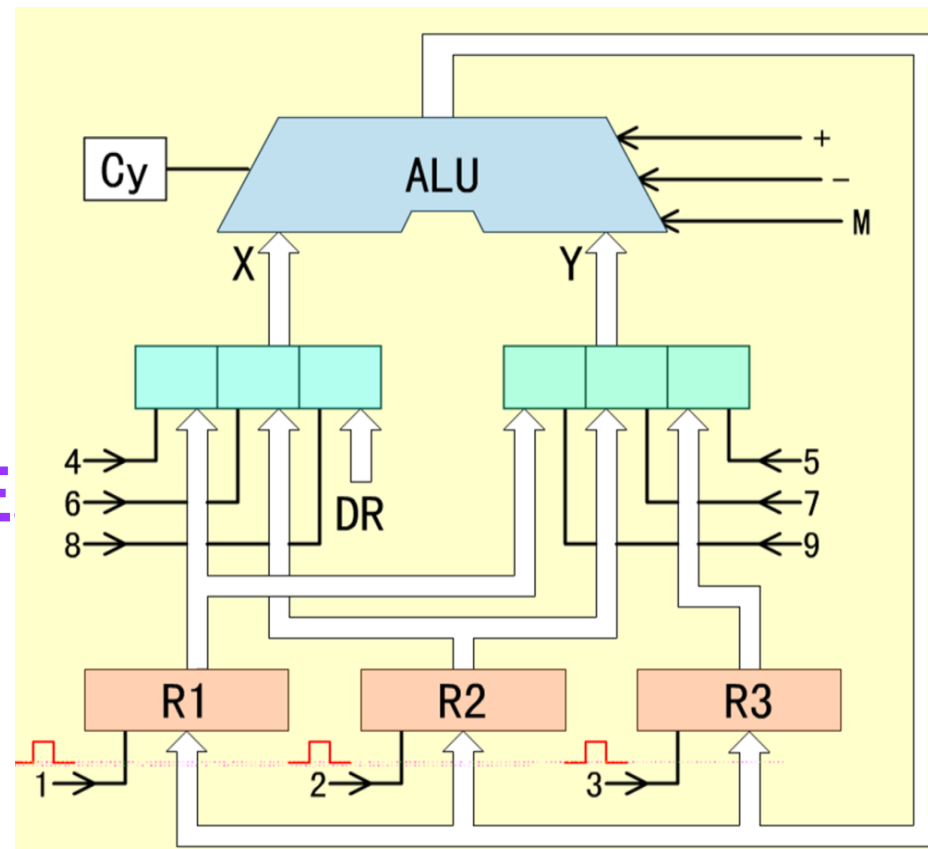
以“十进制加法”指令为例

➤ “十进制加法”指令是用BCD码来完成十进制数的加法运算。

- 当两数位相加小于等于9时，结果正确；
- 当两数位相加大于9时，必须对和数位进行加6的修正

➤ 例如：

- $3+4 = 0011 + 0100 = 0111 = 7$
- $8+7 = 1000 + 0111 = 1111 + \mathbf{0110} = \mathbf{0001\ 0101} = 15$
- $25+36 = 0010\ 0101 + 0011\ 0110$
 $= 0101\ 1011 + \mathbf{0110} = \mathbf{0110\ 0001}$





➤ “十进制加法”指令是用**BCD**码来完成十进制数的加法运算。

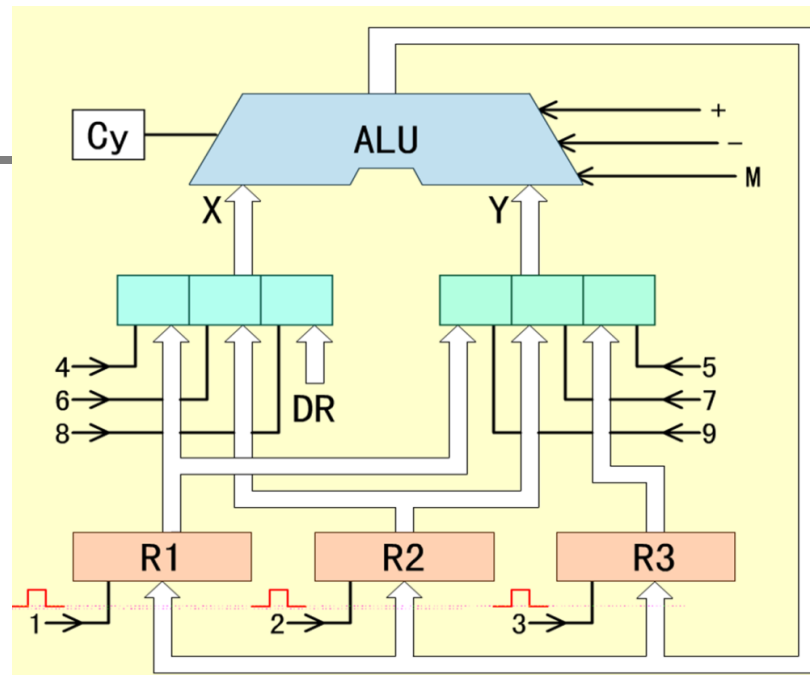
- 当两数位相加小于等于9时，结果正确；
- 当两数位相加大于9时，必须对和数位进行加6的修正。
- **Cy=0，减6，Cy=1正常完成**

$$\begin{array}{r} 4 \\ +5 \\ \hline 9 \end{array} \quad \begin{array}{r} 0100 \\ +0101 \\ \hline 1001 \\ +0110 \\ \hline 1111 \end{array}$$

Cy=0

$$\begin{array}{r} 5 \\ +6 \\ \hline 11 \end{array} \quad \begin{array}{r} 0101 \\ +0110 \\ \hline 1011 \\ +0110 \\ \hline 10001 \end{array}$$

Cy=1



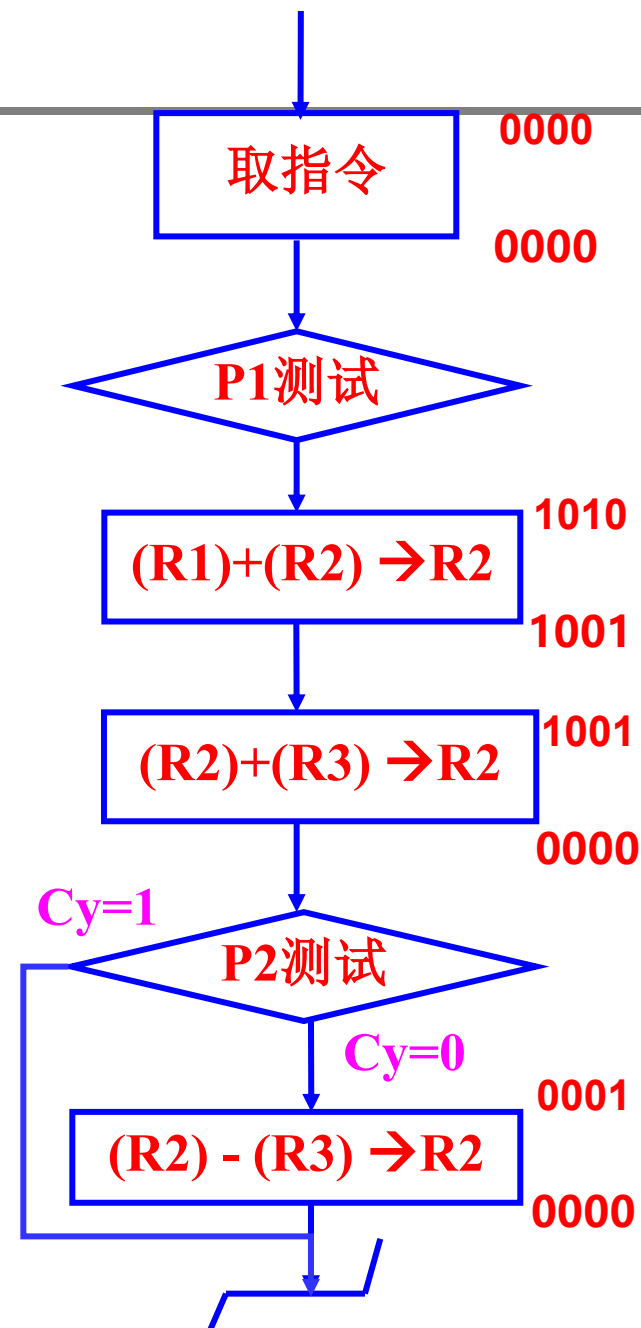
实现过程：

- 1、两个数加完后只管先加6修正
- 2、再判断结果，如果**Cy=0**，等于多加了6，所以还原即减6；反之修正加6是正确的。

>>> 十进制加法指令的微指令执行流程

- 假定(R1)=a, (R2)=b, (R3)=6;
- 第一条微指令——**取指微指令**
 - 从内存中取出该指令, 并修改PC的值;
 - 对操作码进行P1测试, 确定下一条微指令的地址;
- 第二条微指令——**两数相加微指令**
- 第三条微指令——**和数修正微指令**
 - P2测试, 根据辅助进位标志决定下一条微指令的地址
- 第四条微指令——**和数逆修正微指令**
DAA R2, R1

将R1和R2的值作为压缩BCD码相加, 结果送R2



取指微指令

➤➤➤PC→AR

○16号微命令LDAR

➤主存→DR→IR

○13号微命令RD

○14号微命令LDDR

○15号微命令LDIR

➤PC增量

○17号微命令PC+1

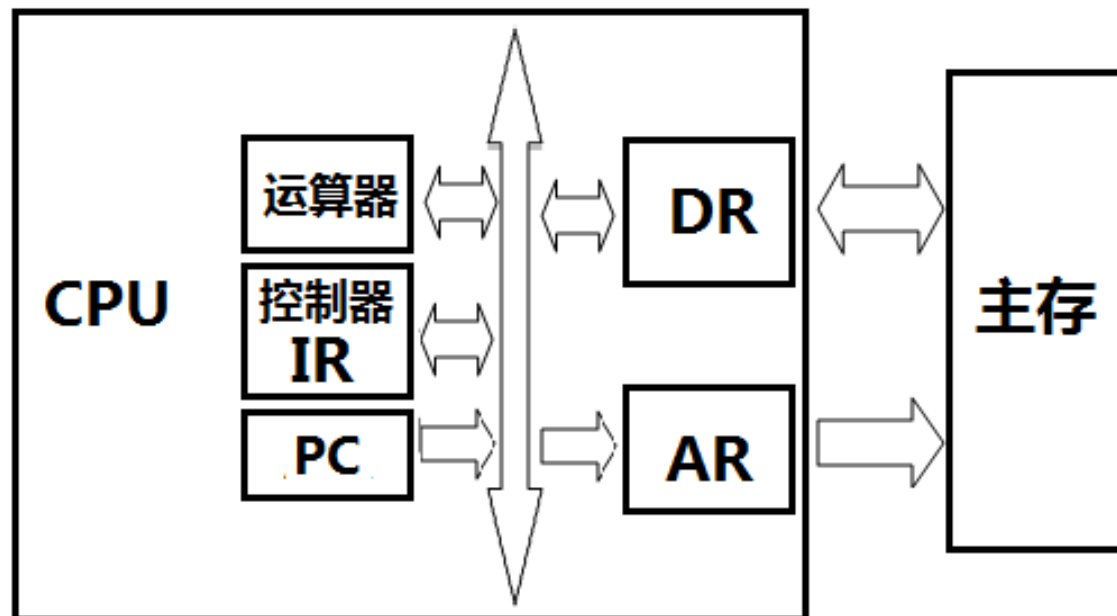
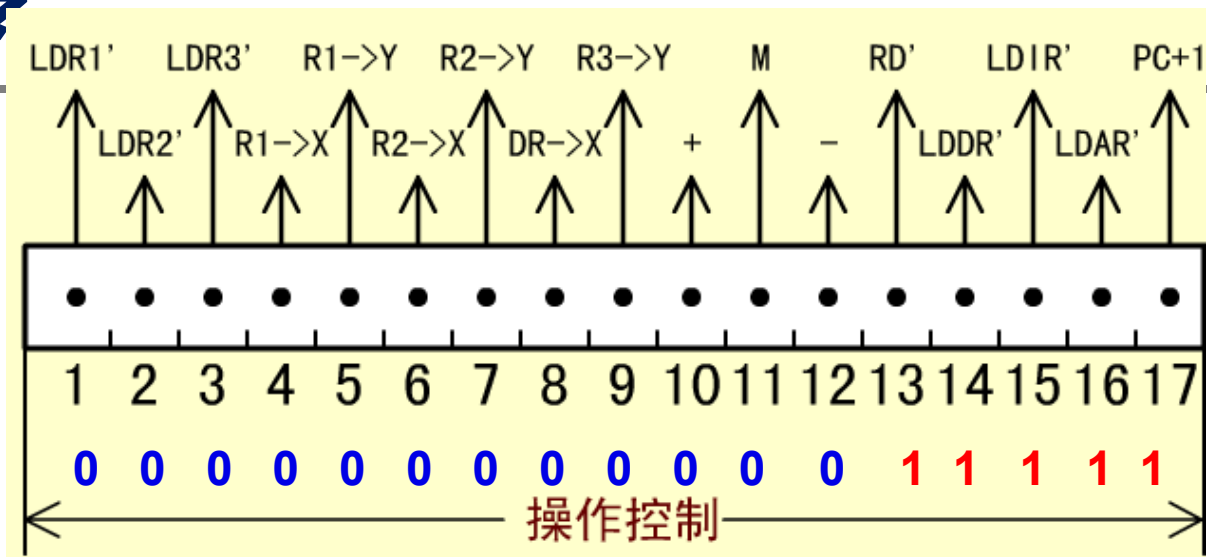
➤操作控制字段为

0000 0000 0000 1111 1

➤顺序控制字段

10 0000

P1根据状态测试的是指令寄存器的操作码，修改地址为1010



控制存储器

0000	取微指令
0001	和数减6
.....	
1001	和数加6
1010	两数相加
.....	

两数相加微指令

➤假定(R1)=a, (R2)=b, (R3)=6

➤两个操作数送入ALU

○4号微命令R1→X

○7号微命令R2→Y

➤ALU执行加法运算

○10号微命令+

➤结果保存在R2中

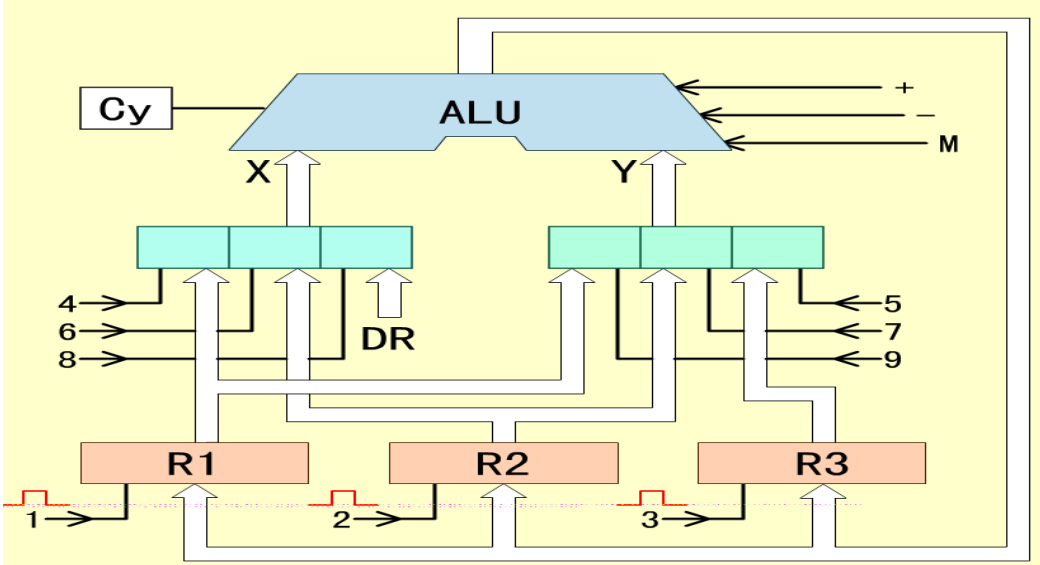
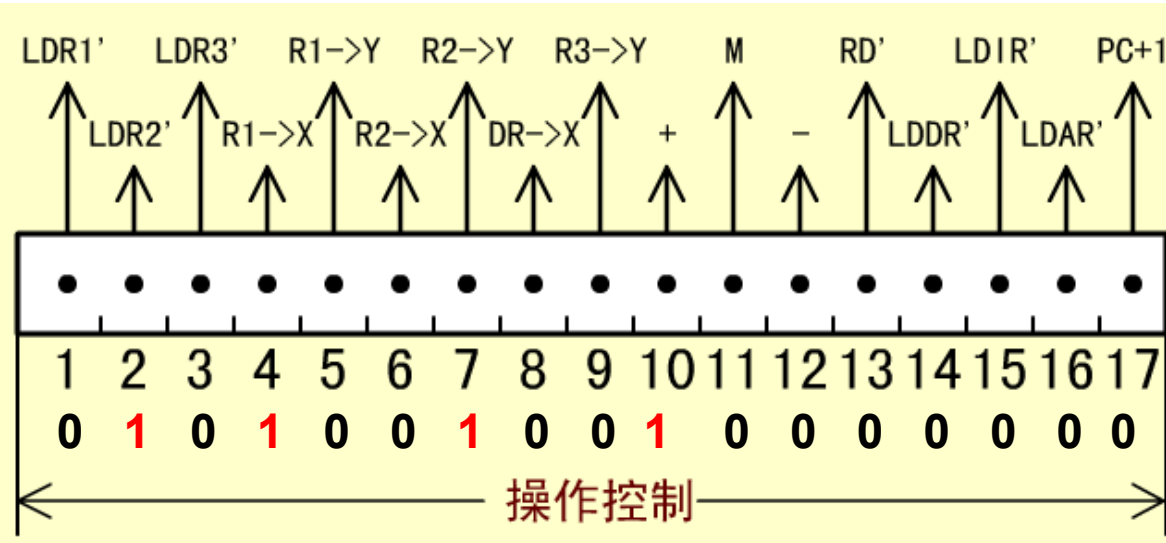
○2号微命令LDR2'

➤微指令的操作控制部分为

0101 0010 0100 0000 0

➤顺序控制字段

00 1001 p1p2=00,下条微指令地址1001（和数加6），不变



控制存储器	
0000	取微指令
0001	和数减6
.....	
1001	和数加6
1010	两数相加
.....	

>>> 和数加6微指令

➤ 假定(R1)=a, (R2)=b, (R3)=6

➤ 两个操作数送入ALU

○ 6号微命令R2→X

○ 9号微命令R3→Y

➤ ALU执行加法运算

○ 10号微命令+

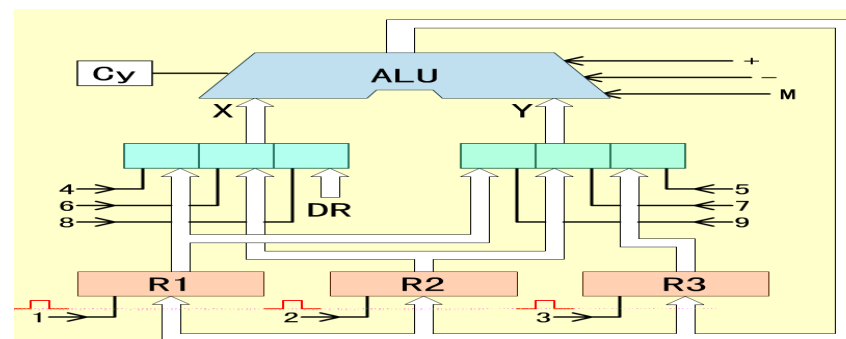
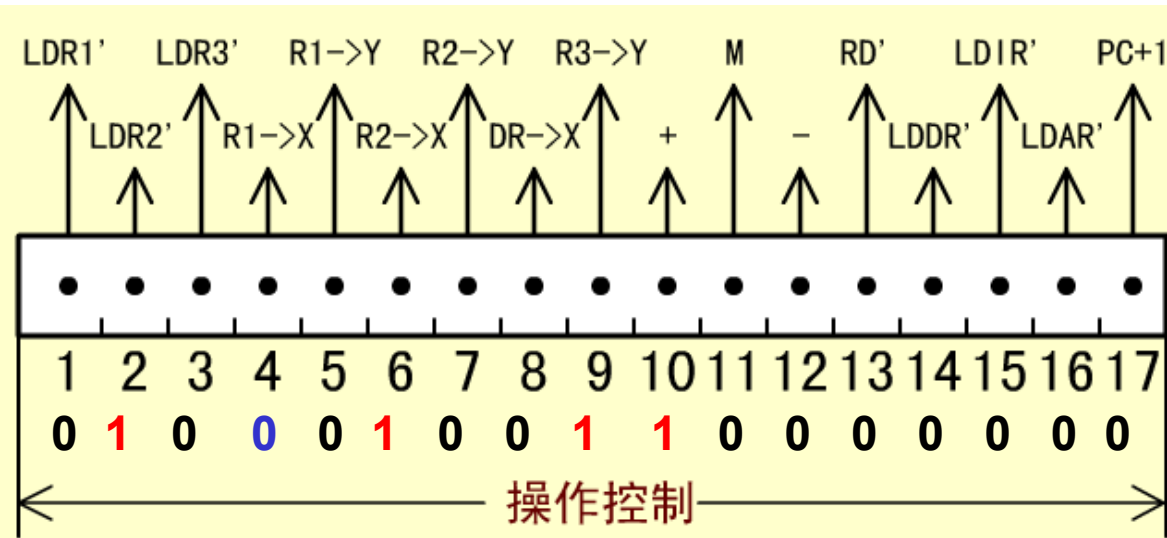
➤ 结果保存在R2中

○ 2号微命令LDR2'

➤ 微指令的操作控制部分为0100 0100 1100 0000 0

➤ 顺序控制字段

01 0000 p2=1, 根据“状态条件”为进位标志位Cy, 若Cy=1, 下一条指令地址为0000 (为什么), 否则修改下条微指令地址为0001(和数减6)



控制存储器

0000	取微指令
0001	和数减6
.....	
1001	和数加6
1010	两数相加
.....	

>>> 和数减6微指令

➤ 假定(R1)=a, (R2)=b, (R3)=6

➤ 两个操作数送入ALU

○ 6号微命令R2→X

○ 9号微命令R3→Y

➤ ALU执行加法运算

○ 12号微命令-

➤ 结果保存在R2中

○ 2号微命令LDR2'

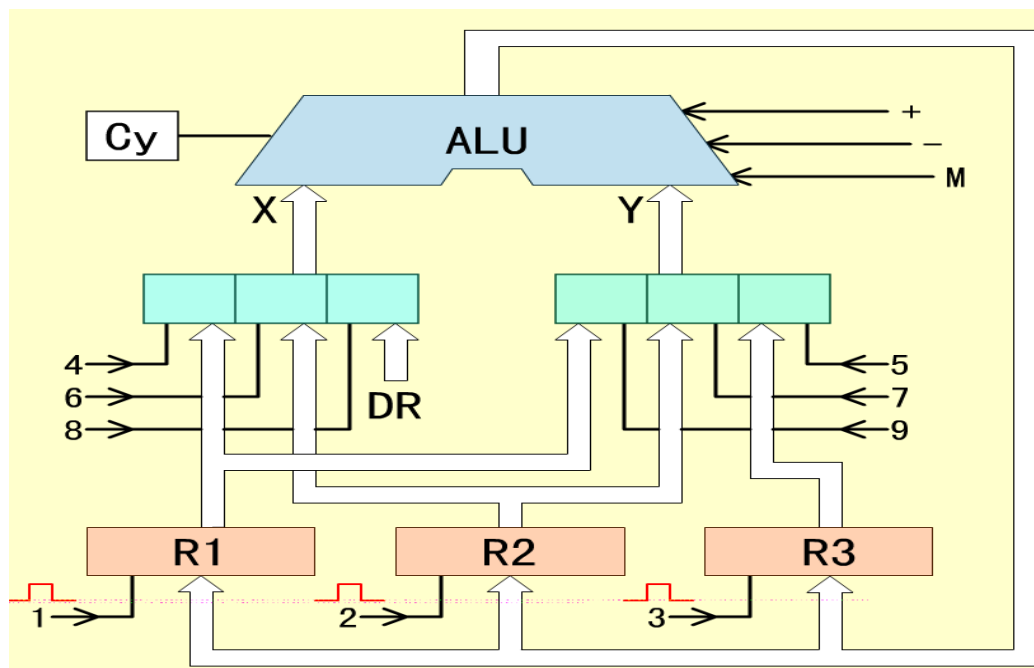
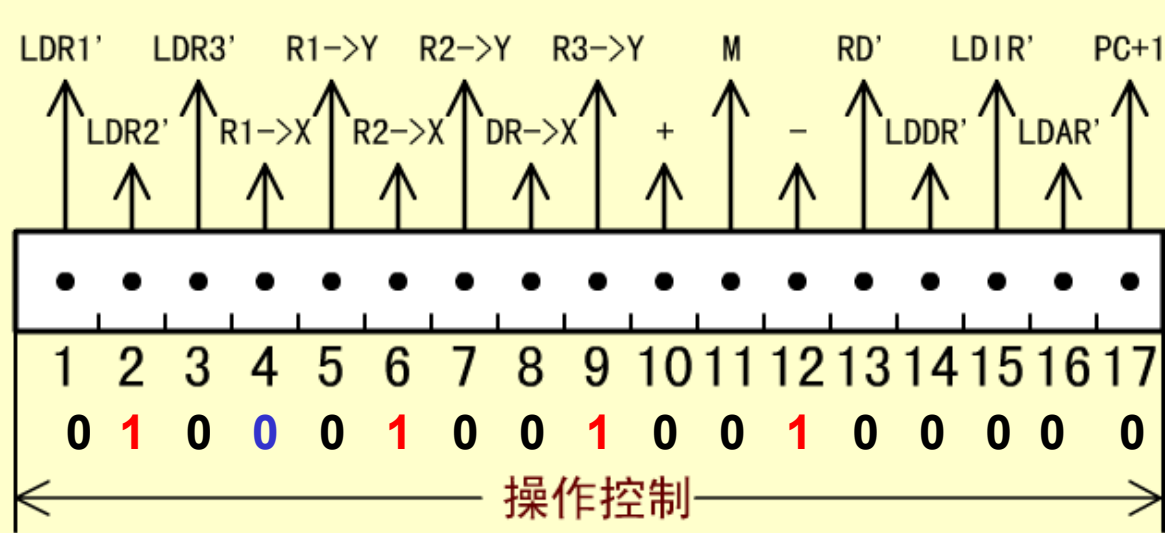
➤ 微指令的操作控制部分为

0100 0100 1001 00000

➤ 顺序控制字段

00 0000

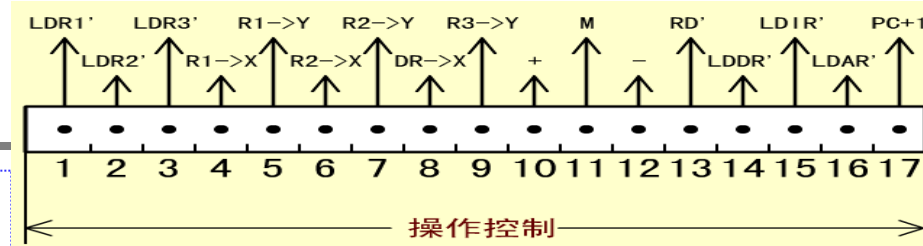
转向取指令微指令



控制存储器

0000	取微指令
0001	和数减6
.....	
1001	和数加6
1010	两数相加
.....	

>>> 十进制加法指令的微指令



➤ 第一条微指令（取指微指令）

○ 000 00 00 00 000 11111 10 0000

RD'
LDAR'
LDDR'
LDIR'
PC+1

➤ 第二条微指令（两数相加微指令）

○ 010 10 01 00 100 00000 00 1001

R1→X
R2→Y
+
LDR2'

➤ 第三条微指令（和数加6微指令）

○ 010 00 10 01 100 00000 01 0000

R2→X
R3→Y
+
LDR2'

➤ 第四条微指令（和数减6微指令）

○ 010 00 10 01 001 00000 00 0000

R2→X
R3→Y
—
LDR2'

控制存储器

0000	取微指令
0001	和数减6
.....	
1001	和数加6
1010	两数相加

..... 微程序转移逻辑

- ① P1测试：操作码译码，微下址由地址映射获得；
- ② P2测试：跳跃寻址时，测试成功则转向微下址单元，否则转向微下址+1单元；
- ③ 顺序寻址：转向微下址单元。



总结

➤ 一台计算机可以划分为：控制部件和执行部件

- 控制部件：控制器；
- 执行部件：相对控制器来说，运算器、存储器、外围设备等

➤ 二者之间的联系：控制线和反馈信息

- 控制器通过控制线发送微命令，执行部件执行微命令对应的操作
- 执行部件通过反馈线向控制部件反映操作情况，为能否执行新的微命令进行“状态测试”

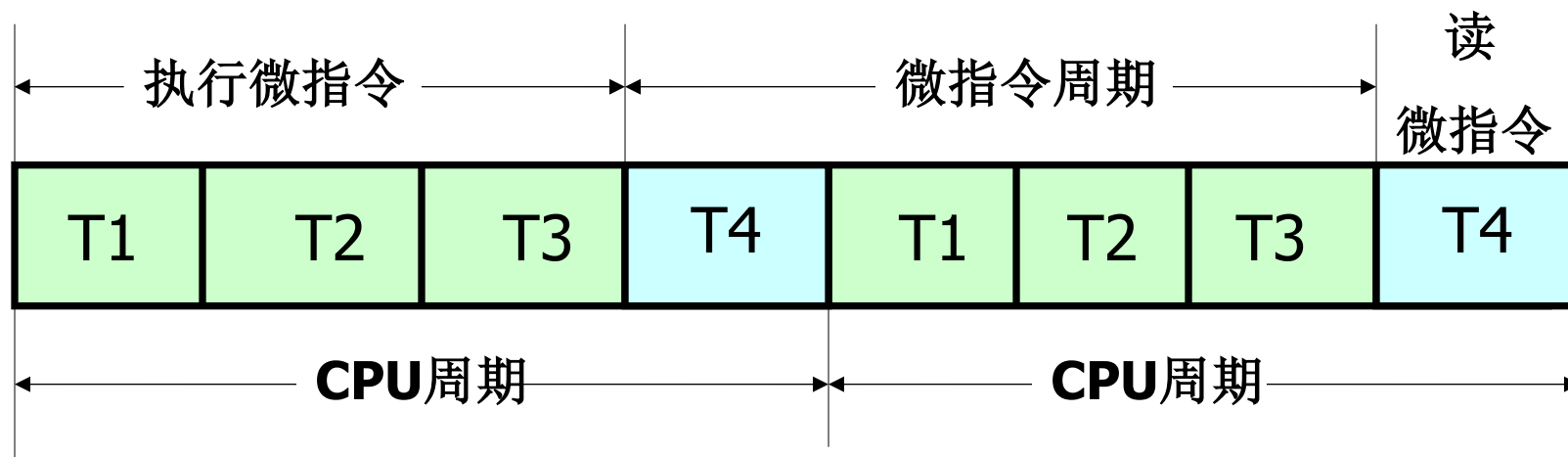
>>> 5、CPU周期与微指令周期的关系

➤ 微指令周期

- 读出微指令的时间加上执行该条微指令的时间；
- 这是串行方式（取微指令与执行微指令串行进行）的微程序控制器中的概念；

➤ 为了确保机器的控制信号的同步，一个微指令周期时间设计得恰好和CPU周期时间相等。

>>> CPU周期与微指令周期关系的例子



➤ **CPU周期为0.8us，包括4个节拍脉冲T1~T4，每个脉冲200ns；**

○ T1—T3的600ns为执行微指令的时间；

○ T4的200ns为取微指令的时间；

➤ **T4的上升沿保存微指令的执行结果，T4期间取微指令。**

➤ **T1的上升沿，将读出的微指令存入微指令寄存器；**

>>> 6、机器指令与微指令的关系

➤ 一条机器指令对应一个微程序，一个微程序是由若干条微指令组成；

- 一条机器指令所完成的操作划分成若干条微指令来完成，每个操作由一条微指令解释和执行。
- 每一个CPU周期对应一条微指令。

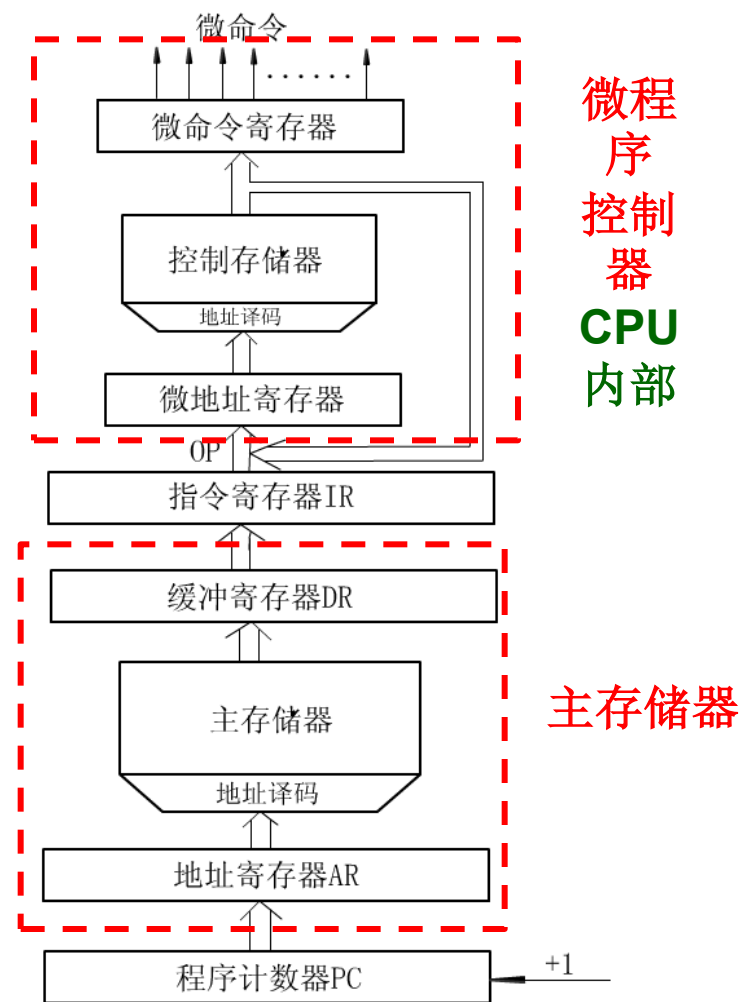
➤ 程序与微程序的关系/机器指令与微指令的关系

○ 使用机器指令编写的程序

- ◆ 存放于系统的主存RAM存储器中，使用存储单元地址访问；
- ◆ 程序员直接使用，面向计算机

○ 每条机器指令对应的微程序

- ◆ 是由保存在控制存储器的微指令构成的，利用微地址访问控存逐条获取；
- ◆ 由机器设计者使用，面向机器指令
- ◆ 一般不可改写



>>> 5.4.2 微程序设计技术

1. 微命令编码
2. 微地址的形成方法
3. 微指令格式
4. 动态微程序设计

微程序设计主要考虑的问题：

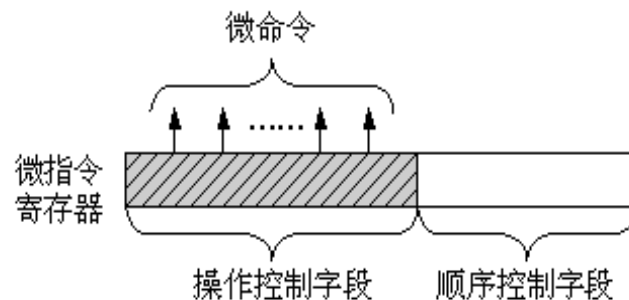
- 如何有效缩短微指令字长；
- 如何有利于缩短微程序，减少所需的控存空间；
- 如何有利于提高微程序执行速度；

>>> 1、微命令编码

➤指微指令中**操作控制字段**的编码表示方法，以及如何把编码翻译成相应的微指令。

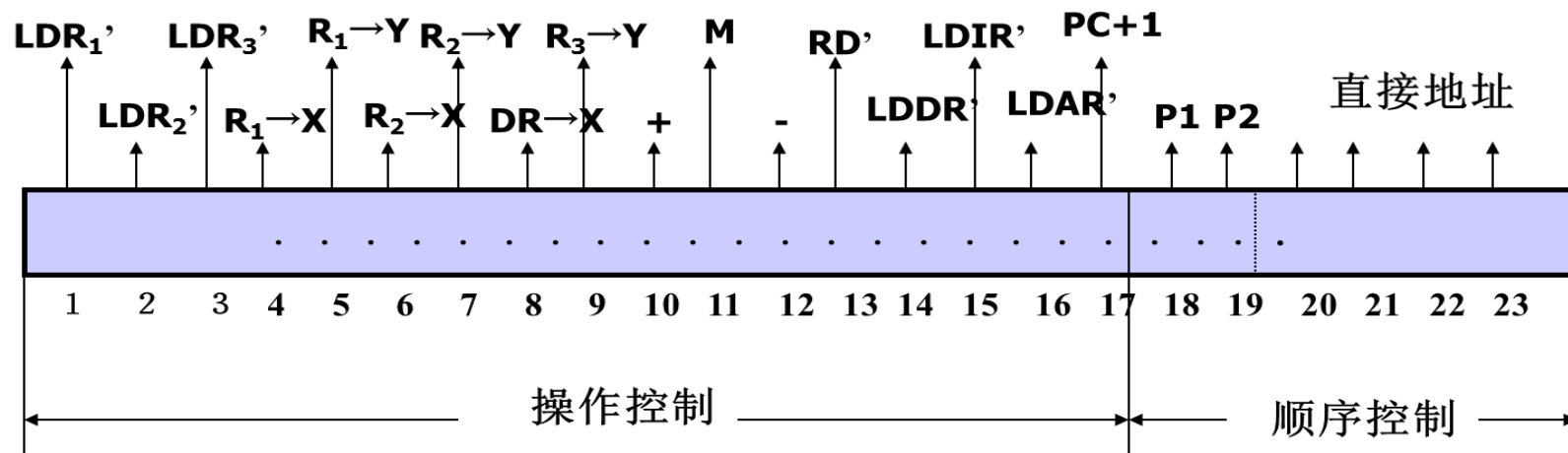
➤微命令编码表示方法

○直接表示法、编码表示法、混合表示法（自学）



>>> 直接表示法

- 操作控制字段中的各位分别可以直接控制计算机，不需要进行译码。
- 操作控制字段中的每一位代表一个微命令，一条微指令能定义多个可并行的微命令；



- **原则：**相容性微命令，可以并行完成的微命令
- **优点**
 - 简单直观，其输出可直接用于控制，执行速度快（同时可进行多个微操作）；
- **缺点**
 - 微指令字较长，因而使控制存储器容量较大。

>>> 编码表示法

← 下 地 址 字 段 →			← 操 作 控 制 字 段 →							
微下址	CI ₃ ~CI ₀	SCC ₃ ~SCC ₀	IOMRW	I ₂ ~I ₀ I ₈ ~I ₃	AB口地址	SST	SSH	SCI	DC ₂	DC ₁
8位	4位	4位	3位	9位	8位	3位	1位	2位	3位	3位

➤ 编码方法

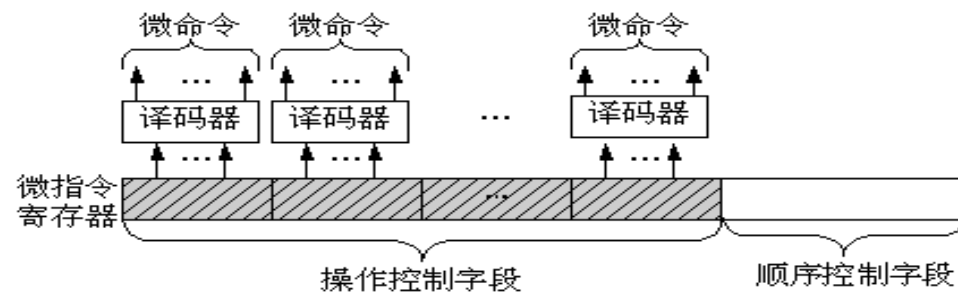
- 将微指令操作控制字段划分为若干个**子字段**；每个字段是一组**相斥微命令**。
- 每个子字段的**不同编码表示不同的微命令**；

➤ 遵循原则：

- 把**相斥的微命令**划分在同一个字段中，**相容的微命令**划分在不同字段；
- 字段的划分应与数据通路结构相适应；
- 每个子字段应留出一个**空操作状态**，即可选择不发出该子字段对应的所有微命令；例如**3位二进制位译码**后可表示**7个微命令**。
- 每个子字段所定义**的微命令不宜太多**；

➤ 优点:可大大缩短微指令字长；

➤ 缺点:需要微命令译码，增加译码电路。故微程序的执行速度稍稍减慢。



>>> 编码表示法

➤ 例如，某机器指令系统总共需要**256个微命令**

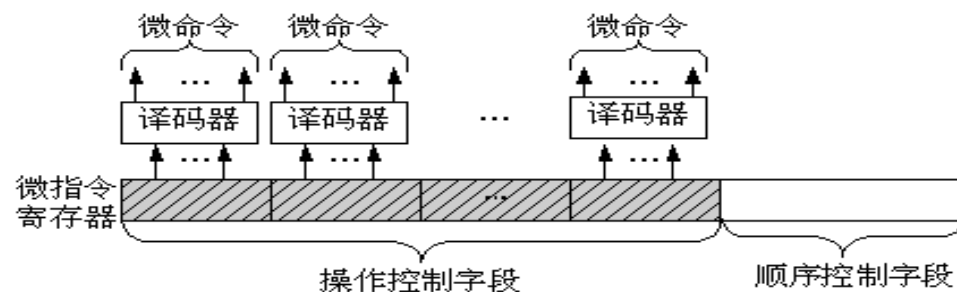
○ 采用直接编码法

◆ 微指令的操作控制字段需**256位**；

○ 采用编码表示法

◆ 若采用4位一个子字段，则每个子字段可编码16个微命令；

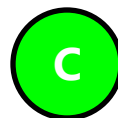
◆ 256个微命令需16个子字段，即微指令的操作控制字段仅**64位**。



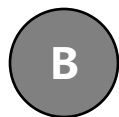
某计算机的控制器采用微程序控制方式，微指令中的操作控制字段采用字段直接编码法，共有33个微命令，构成5个互斥类，分别包含7、3、12、5和6个微命令，则操作控制字段至少有（ ）



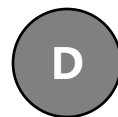
A 5位



C 15位



B 6位



D 33位

7---3, 3---2, 12---4, 5---3, 6---3

至少: $3+2+4+3+3=15$

提交

>>> 2、微地址的形成方法

- 微指令执行的顺序控制问题，实际上就是如何确定下一条微指令的地址问题。
- 微程序的入口地址
 - 微程序的第一条微指令所在控存单元的地址；
 - ◆ 确切的说，应该是**机器指令执行周期**的第一条微指令的地址；
 - ◆ 当公用的取指微程序从主存中取出机器指令之后，由机器指令的操作码字段指出各段微程序的入口地址，这是一种多分支(或多路转移)的情况。
- 产生后继地址的两种方法
 - 计数器的方式
 - 多路转移的方式

>>> 计数器方式

- 与机器指令顺序控制方法类似。
 - 设置微程序计数器 μPC ;
 - 顺序寻址, 则 $\mu PC+1$;
 跳跃寻址, 则以某种方式为 μPC 赋值;
- 优点
 - 微指令的顺序控制字段较短 (不必有微地址字段), 微地址产生机制简单。
- 缺点
 - 多路并行转移功能较弱, 速度较慢, 灵活性较差。

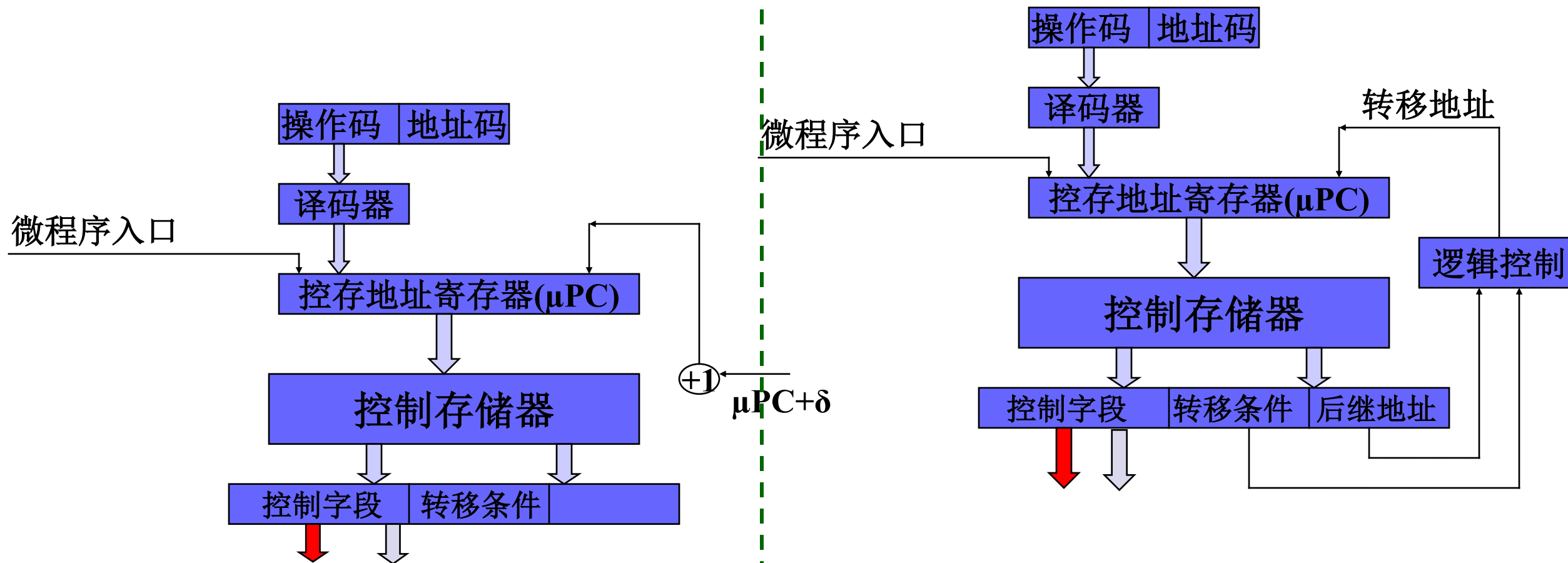
多路转移方式

- 根据“判别测试”标志和“状态条件”信息 (操作码+状态条件测试+微指令中下地址), 由判断逻辑确定后继微地址。
 - 判别测试条件可选用直接表示或编码表示方法;
 - 状态条件若 n 位标志, 则可 2^n 路转移
- 特点
 - 能以较短的顺序控制字段配合, 实现多路并行转移, 灵活性好, 速度较快;
 - 但转移地址逻辑需要用组合逻辑方法设计, 硬件成本较高。



顺序执行微指令时：现行微地址加上一个增量产生后继微地址

非顺序执行微指令时：由转移微指令直接或间接给出转移微地址

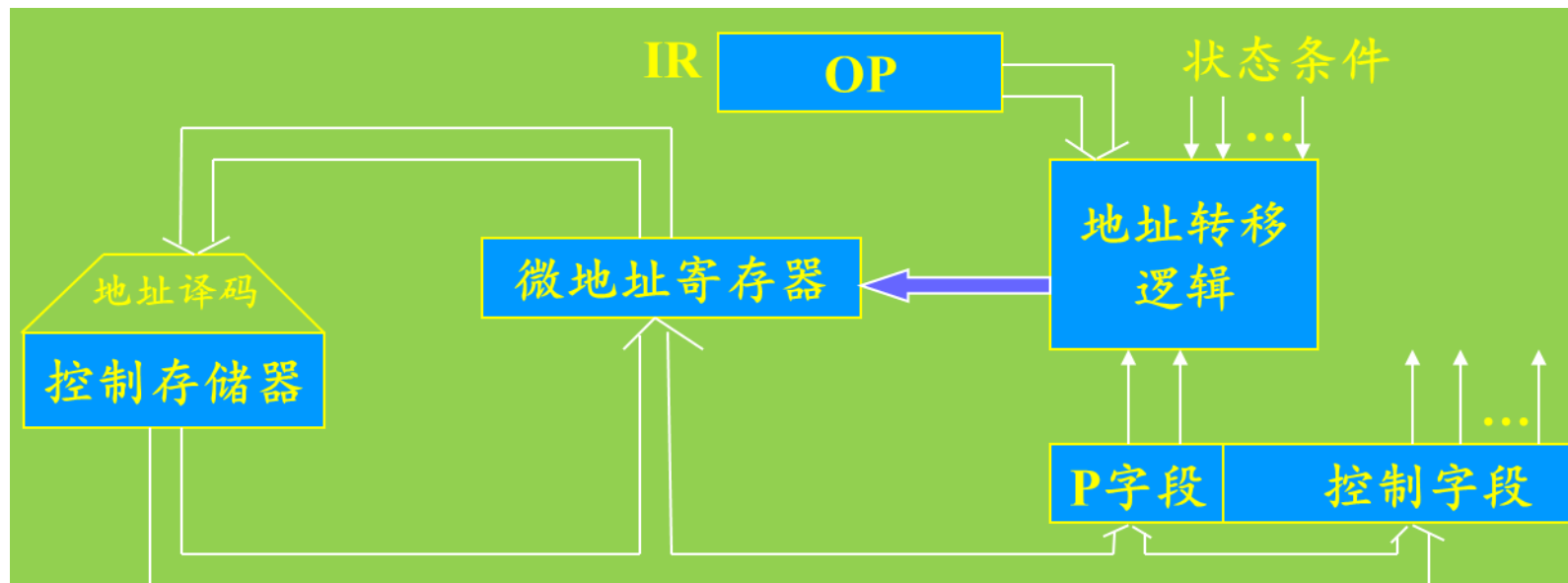


微程序的执行顺序控制：计数器方式

>>> 微程序的执行顺序控制：多路转移方式

➤ 严格按照微指令中的微地址字段取下一条微指令

- 微程序不产生分支时：后继微地直接由微指令的顺序控制字段给出
- 微程序出现分支时：按顺序控制字段的“判别测试”标志和“状态条件”信息通过运算或判别测试选择若干候选微地址中的一个微地址



>>> 多路转移方式

- 一条微指令具有多个转移分支的能力成为多路转移。
- 可根据条件信息选定某一个候选微地址的方法。
 - 当微程序不产生分支时，后继地址就是顺序控制字段的地址
 - 当微程序产生分支，按照顺序控制字段的“判别标志”和“状态条件”信息来选择其中一个微地址
 - ◆ 状态条件有 n 位标志位，则有 2^n 路转移
- 特点
 - 能以较短的顺序控制字段配合，实现多路并行转移，灵活性好，速度较快；
 - 但转移地址逻辑需要用组合逻辑方法设计。

➤➤➤ 【例】已知某计算机采用微程序控制方式，其控存容量为 512×32 位，微程序可以在控存中实现转移，可控制微程序转移的条件有6个，采用直接编码方式，后继微指令地址采用多路转移方式。

➤ 微指令字长32位，格式如下，请说明微指令中3个字段分别应为多少位。

微操作编码	测试指令	下址
-------	------	----

- ① 由控存单元数可知共512个，则每个控存单元地址位为9位，则微指令中的后继微指令地址位数为9位；
- ② 可控制微程序转移的条件为6个，且按照直接控制编码，则测试条件位数为6位；
- ③ 剩下的为操作控制字段可用位数 $32 - 9 - 6 = 17$ 位；

>>> 3、微指令格式

➤影响微指令格式的主要因素是微指令的编译方法，通常考虑速度、成本等原因，设计时采用不同的编译方法，因此编译方式也不同

➤**两类：**水平型微指令和垂直型微指令

➤水平型微指令(例如本书的例子)

控制字段	判别 测试字段	下地址字段
------	------------	-------

○一次能定义并执行多个并行操作微命令的微指令；

◆一般由操作控制字段、判别测试字段、下地址字段三部分构成；

○根据控制字段编码方式的不同，可分为全水平型、字段译码法水平型、直接和译码相混合的水平型三种微指令。

>>> 3、微指令格式

- 垂直型微指令（类似机器指令结构），采用编码方式。
 - 微指令中设置微操作码字段，采用微操作码编译法，由微操作码规定微指令的功能；
 - ◆ 例如共有63种微操作，则6位编码即可
 - 设置微操作控制字段时，一次只能执行一到二个微命令的微指令称为垂直型微指令。
 - 结果较长的微程序换取较短的微指令结构

>>> 垂直型指令举例

- 微指令字长**16**位，微操作码**3**位
- 寄存器-寄存器传送型微指令



○ 功能：把源寄存器数据送目标寄存器

- 运算控制型微指令



○ 功能：选择ALU的左、右两端的输入源，按ALU字段所指定的运算功能进行处理，并将结果送入暂存器中

>>> 水平型微指令与垂直型微指令比较

1. 水平型微指令**并行操作能力强**，效率高，灵活性强。垂直型微指令则较差。
2. 水平型微指令**执行一条指令的时间短**，垂直型微指令执行时间长。
3. 由水平型微指令解释指令的微程序，具有**微指令字比较长，但微程序短**的特点。
垂直型微指令则相反，微指令字比较短而微程序长。
4. 水平型微指令用户**难以掌握**，而垂直型微指令与指令比较相似，相对来说，比较容易掌握（因为可编程）。

>>> 5.4.2 微程序设计技术

➤4、微程序设计

- 对应于一台计算机的机器指令只有一组微程序，这一组微程序设计好之后，一般无须改变而且也不好改变，这种微程序设计技术称为**静态微程序设计**。
- 采用E²PROM作为控制存储器，可以通过改变微指令和微程序来改变机器的指令系统，这种微程序设计技术称为**动态微程序设计**。

>>> 5.5 硬连线控制器

➤ 基本思想

○ 实现方法

- ◆ 通过逻辑电路直接连线而产生的，又称为组合逻辑控制方式

○ 设计目标

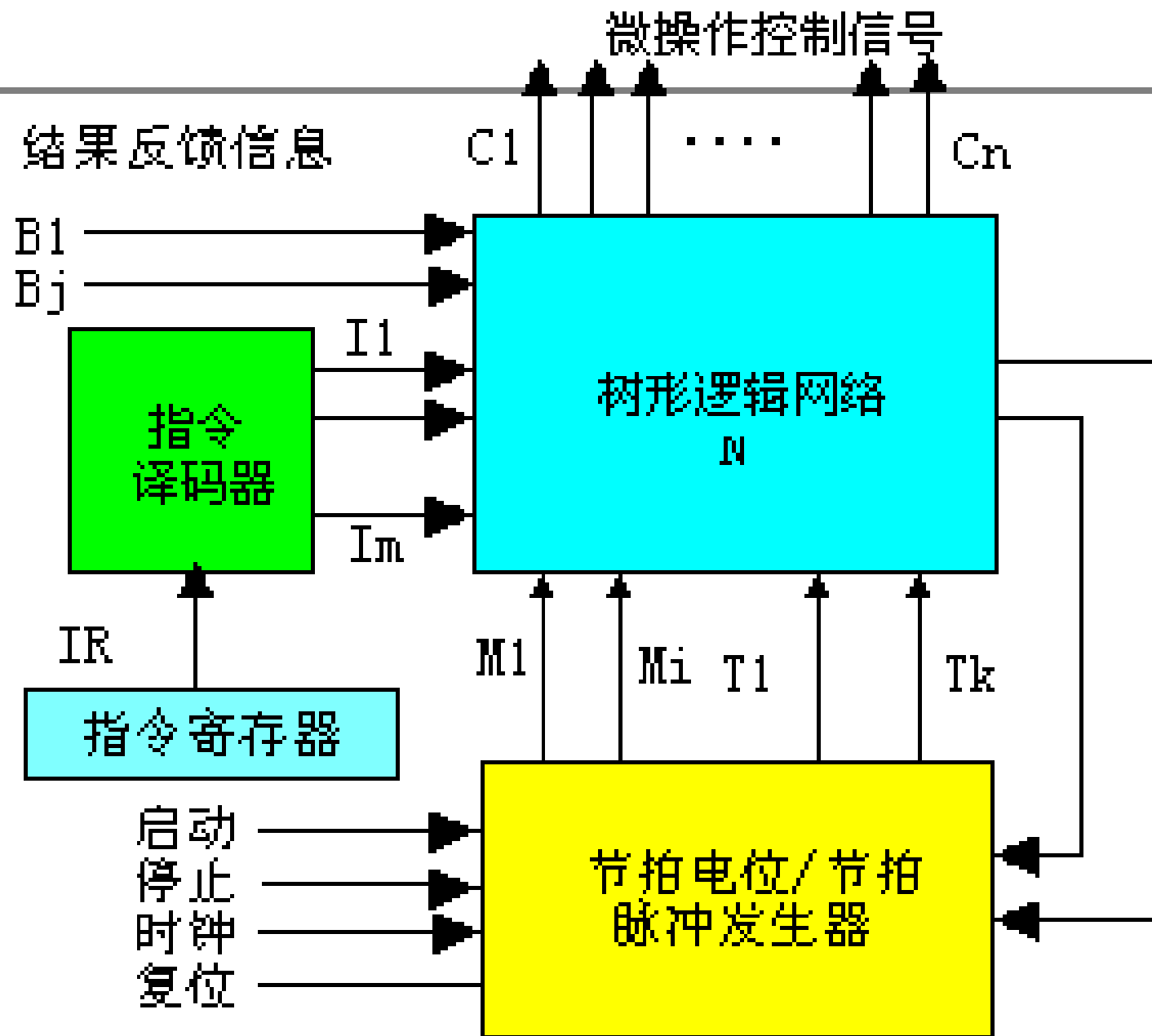
- ◆ 使用最少元件（复杂的树形网络）
- ◆ 速度最高

>>> 5.5 硬连线控制器

- 硬布线控制器是一种由门电路和触发器构成的复杂树形网络。
- 硬布线控制器是早期设计计算机的一种方法；
 - 随着新一代机器及VLSI技术的发展，硬布线逻辑设计思想又得到了重视。
- 与微程序控制相比，硬布线控制的速度较快；
 - 微程序控制中每条微指令都要从控制存储器中读取一次，影响了速度，而硬布线控制主要取决于电路延迟；
- 缺点：
 - 设计不规整，设计效率较低；
 - 控制器核心结构零乱，不便于检查和调试；
 - 不易修改、扩展指令系统功能；



硬连线控制器结构方框图



>>> 硬布线控制器与微程序控制器的比较

➤ 指令功能的实现

- **组合逻辑控制器**：一条指令的功能是直接由硬布线控制器或者组合逻辑控制器解释实现的
- **微程序控制器**：一条指令对应一段微程序，指令的功能由微程序解释实现的，微命令是硬件完成的。

➤ 速度影响

- **组合逻辑控制器**：硬件实现指令，速度较快
- **微程序控制器**：多次访存，速度相对慢

下列关于主存储器(MM)和控制存储器(CS)的叙述中，错误的是

Control Store，简称CS

- ☐ A MM在CPU外，CS在CPU内
- ☒ B MM按地址访问，CS按内容访问
- ☐ C MM存储指令和数据，CS存储微指令
- ☐ D MM用RAM和ROM实现，CS用ROM实现

提交

2009年考研408真题 第19题

19. 相对于微程序控制器，硬布线控制器的特点是（ ）

- ☐ A 指令执行速度慢，指令功能的修改和扩展容易
- ☐ B 指令执行速度慢，指令功能的修改和扩展难
- ☐ C 指令执行速度快，指令功能的修改和扩展容易
- ☒ D 指令执行速度快，指令功能的修改和扩展难

提交

5.6 流水线技术与流水CPU

5.6.1 流水线原理

5.6.2 流水线的应用

5.6.3 指令流水线设计中的若干问题

>>> 5.6.1 流水线原理

➤ 标准冯·诺依曼体系CPU

- 采用串行处理技术，一个时刻只能进行一个操作；

➤ 使用流水线的目的

- 利用时间和空间的**并行处理技术**，加快计算机的处理速度；
- 在任一时刻，各工序均同时并行工作，但每个工序完成不同的部分

➤ 流水线的处理方法

- 引入工厂生产流水线技术原理，将输入任务分割成**可并发执行的多个子任务**；
- 每个子任务的工作时间基本相当，分别由固定的部件完成；
 - ◆ 例如，指令可分成：取指IF、译码ID、执行EX、写回WB；
- 也可同时设置多条流水线，并行执行不同的指令；

>>> 5.6.1 流水线原理

➤ 并行性的两种含义：

① **同时性**：两个以上事件在同一时刻发生；

◆ 如多机系统中，同一时刻多个进程在运行。

② **并发性**：两个以上事件在同一间隔内发生。

◆ 如并发程序，某一时刻CPU中只有一个进程在运行，而在一个时间段内，多个进程同时运行。

➤ 并行性的三种形式：

○ **时间并行（重叠）**：让多个处理过程在时间上相互错开，轮流使用同一套硬件设备的各个部件，以加快硬件周转而赢得速度，实现方式就是采用流水处理部件。

○ **空间并行（资源重复）**：以数量取胜，设置重复资源，同时工作。

◆ 它能真正的体现同时性；LSI和VLSI为其提供了技术保证

○ **时间并行 + 空间并行**：时间重叠和资源重复的综合应用。

>>> 流水CPU的时空图

➤实现流水：输入任务分割成一系列子任务，子任务在流水线的各个阶段并发地执行。

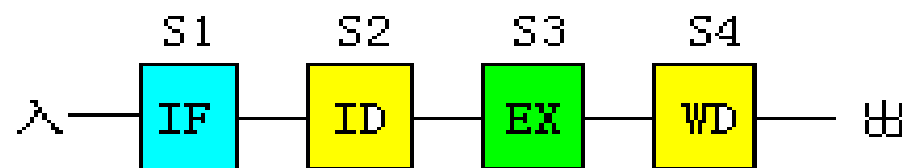
➤流水线CPU时空图

○IF (Instruction Fetch取指)

○ID (Instruction Decode指令译码)

○EX (Execution执行或计算有效地址)

○WB (Write Back写回)



(a) 一个指令流水线过程段

➤指令执行过程划分为不同阶段，占用不同资源，就能使多条指令同时进行。

>>> 5.6.1 流水线原理 ——时空图

➤ 非流水CPU的执行

- 假定指令执行包括4个阶段：取指IF、译码ID、执行EX、写回WB；
- **串行**执行完成一条指令，需要4个周期；第1条指令执行完才能执行第2条指令；

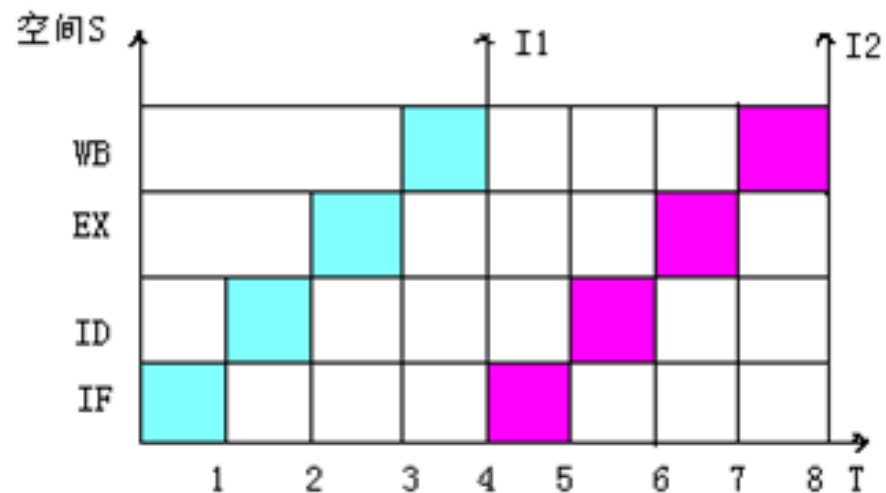
取指1	译码1	执行1	写回1	取指2	译码2	执行2	写回2	...
-----	-----	-----	-----	-----	-----	-----	-----	-----

➤ 流水CPU的执行

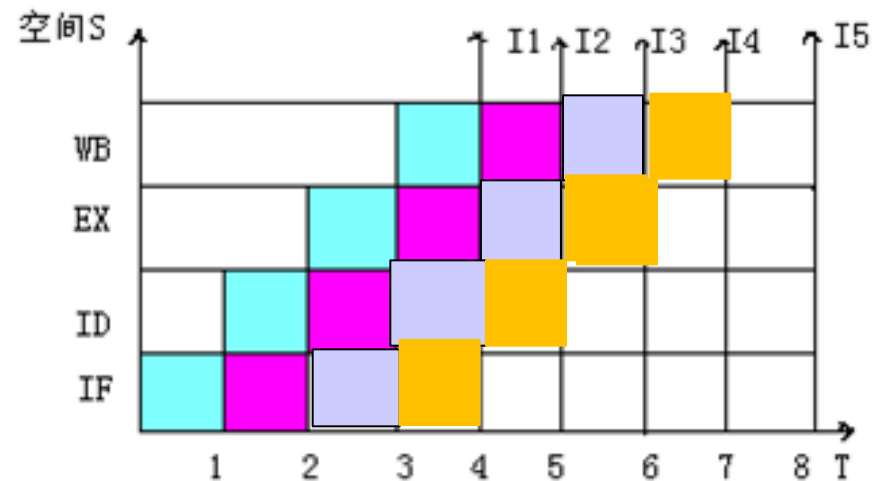
- 流水线上设置4个独立部件，分别负责每个指令阶段的执行；
- 每个部件工作时长一致，以4个阶段的最长执行时间为准；
- 理想情况下，流水线满负荷后，多条指令并行执行，每个周期完成1条指令的执行。



>>> 流水CPU的时空图



(b) 非流水线时空图



(c) 标量流水线时空图

当流水线满载时，每一个时钟周期就可以输出一个结果

每隔4个机器时钟周期才有一个输出结果

>>> 流水计算机的时空图

➤ 标量流水线（pipeline）

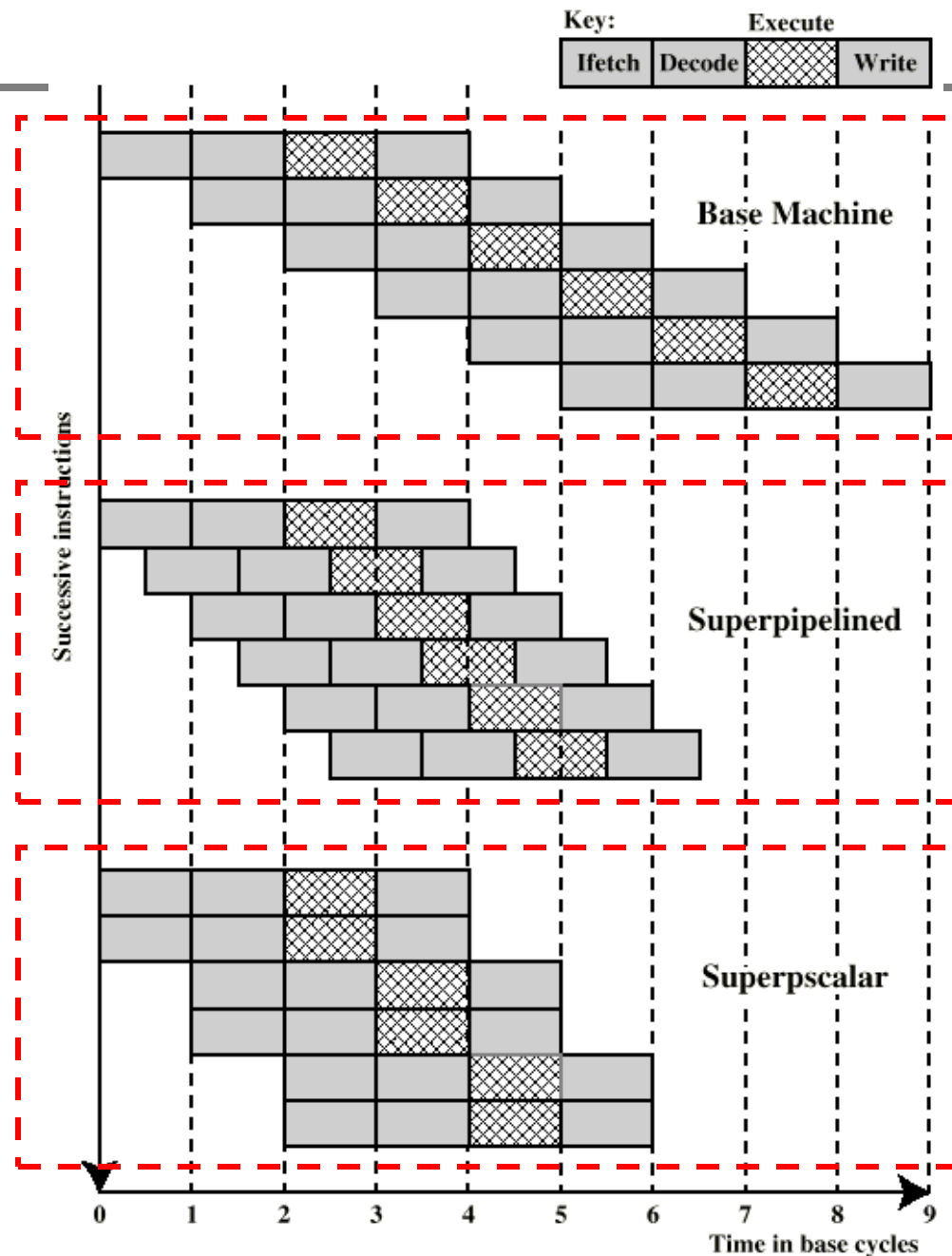
- 只有一条指令流水线；

➤ 超流水线（superpipeline）

- 多级流水线；
- 每个阶段内部再划分步骤；
- 奔腾Pro的流水线长达14步；

➤ 超标量（superscalar）流水线

- 具有两条以上的指令流水线；
- 奔腾以上CPU才具有这种结构；



(2017年) 下列关于超标量流水线特性的叙述中, 正确的是_____

- I. 能缩短流水线功能段的处理时间
- II. 能在一个时钟周期内同时发射多条指令
- III. 能结合动态调度技术提高指令执行并行性

A 仅II

C 仅II、III

B 仅I、III

D I、II和III

>>> 流水线分类 (1/2)

➤ 指令流水线：处理器级流水

- 指令执行的并行处理；
- 例如，指令流可划分为取指、译码、取数、执行、写回等阶段；

➤ 算术流水线：部件级流水

- 运算操作步骤的并行处理；
- 现代微机中大多采用流水的算术运算器；

➤ 处理机流水线：系统级流水

- 程序步骤的并行处理；
- 将每一阶段的处理分散在不同的机器上，应用于多机系统中；

>>> 流水线分类 (2/2)

➤ 按流水线功能的强弱划分

- 单功能流水线
- 多功能流水线

◆ 按连接方式划分

静态流水线、动态流水线

➤ 按流水线是否有反馈回路划分

- 线性流水线
- 非线性流水线

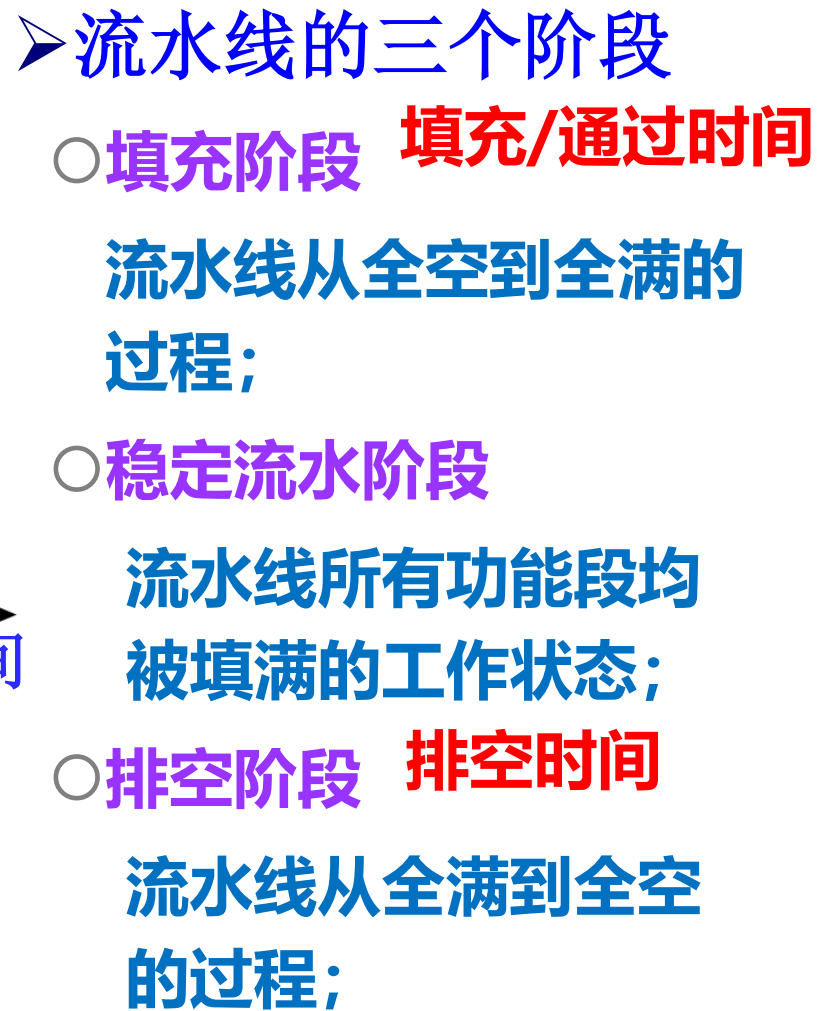
➤ 按流水线任务调度顺序划分

- 顺序流动流水线
- 异步流动流水线

➤ 按流水线一次处理对象的数量划分

- 标量流水线
- 超标量流水线
- 向量流水线
- 超长指令字流水线

示例：流水线阶段数为k，每阶段耗时 Δt_0
通过时间=排空时间= $(k-1)\Delta t_0$



>>> 流水CPU的加速比

- 若某CPU指令执行，可划分为K个阶段，每个阶段1个周期；
- 非流水线CPU处理n个任务时，所需时钟周期数

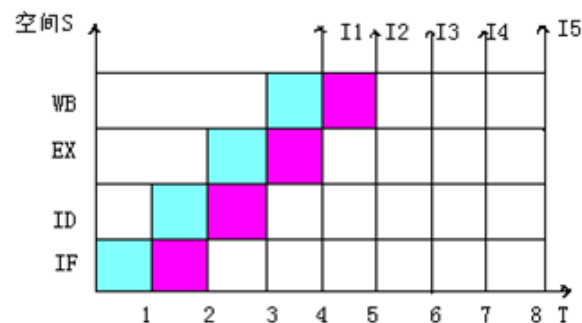
$$T_1 = n \times k$$

- 一个具有k级过程段的流水CPU处理这n个任务时，需要的时钟周期数

$$T_k = k + (n - 1)$$

- k个时钟周期用于处理第一个任务；
 - k个周期（即第一个任务）后，流水线被装满，剩余的n - 1个任务只需n - 1个周期即可完成（即每个任务只需要一个周期，剩余共n-1个任务）。
- 将 T_1 和 T_k 的比率定义为k级线性流水处理器的加速比：

$$C_k = T_1 / T_k$$



(c) 标量流水线时空图

>>> 流水线的吞吐率 (Throughput Rate)

➤ 流水线的吞吐率

- 单位时间内流水线所完成的任务数量或输出的结果数量;
- 指令流水线的吞吐率, 即单位时间内完成指令的条数, 单位: MIPS;

➤ 最大吞吐率 $TP_{\max}$: 流水线达到稳定状态后的吞吐率;

➤ 实际吞吐率 TP : m 阶段流水线完成 n 个任务的吞吐率;

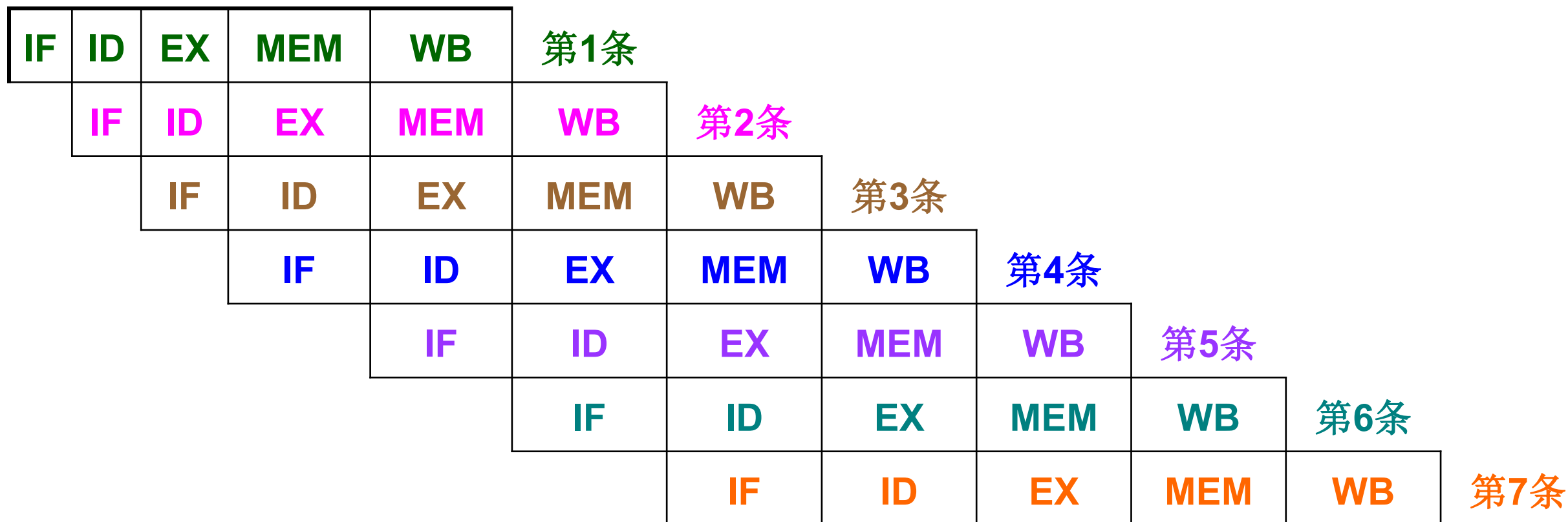
➤ 假定一个具有 k 级过程段的流水 CPU 处理 n 个任务时, 流水线的吞吐率为:

$$TP = \frac{n}{T_k \cdot \Delta t} = \frac{n}{(k - 1 + n) \Delta t}$$

Δt : 指令流水线的时钟周期长

以上计算仍为理想状态。

>>>【例】指令流水线有取指(IF)、译码(ID)、执行(EX)、访存(MEM)、写回寄存器堆(WB)五个过程段，共有7条指令连续输入此流水线，时钟周期为100ns，请画出流水处理的时空图。



➤➤➤【例1】指令流水线有取指(IF)、译码(ID)、执行(EX)、访存(MEM)、写回寄存器堆(WB)五个过程段，共有7条指令连续输入此流水线，时钟周期为100ns。

➤求流水线的实际吞吐率（单位时间里执行完毕的指令数）；

○流水线执行完7条指令的时间是

$$5 + (7 - 1) = 11 \text{ 个时钟周期}$$

○故实际吞吐率为

$$7 / (11 \times 100\text{ns}) \approx 64 \times 10^5 \text{ 条指令/s}$$

➤求流水处理器的加速比。

○K级流水线处理器的加速比为

$$C_k = \text{非流水时间} T_1 / \text{K级流水时间} T_k = n \times k / (k + (n - 1))$$

○本题中，已知数据 $n = 7$ ， $k = 5$ ，则

$$C_k = 7 \times 5 / (5 + 6) = 35 / 11 \approx 3.18$$

>>> 5.6.2 流水CPU的应用

流水线技术是Intel在486芯片中首次使用

1、流水计算机的系统组成

➤ 流水CPU的组成

○ 指令部件

- ◆ 指令部件本身构成一个流水线，由**取指令**、**指令译码**、**计算操作数地址**、**取操作数**等过程段组成。

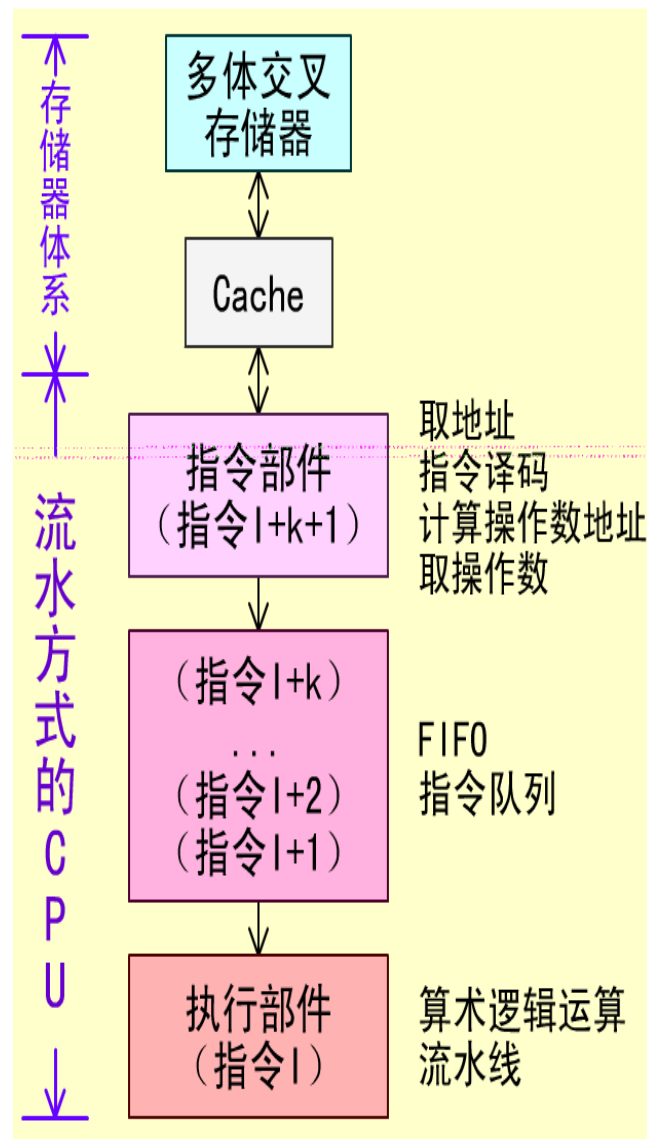
○ 指令队列

- ◆ 指令队列是一个先进先出的寄存器栈，用于存放经过译码的指令和取来的操作数。

○ 执行部件

- ◆ 执行部件可以具有多个算术逻辑运算部件，这些部件本身又用流水线方式构成。

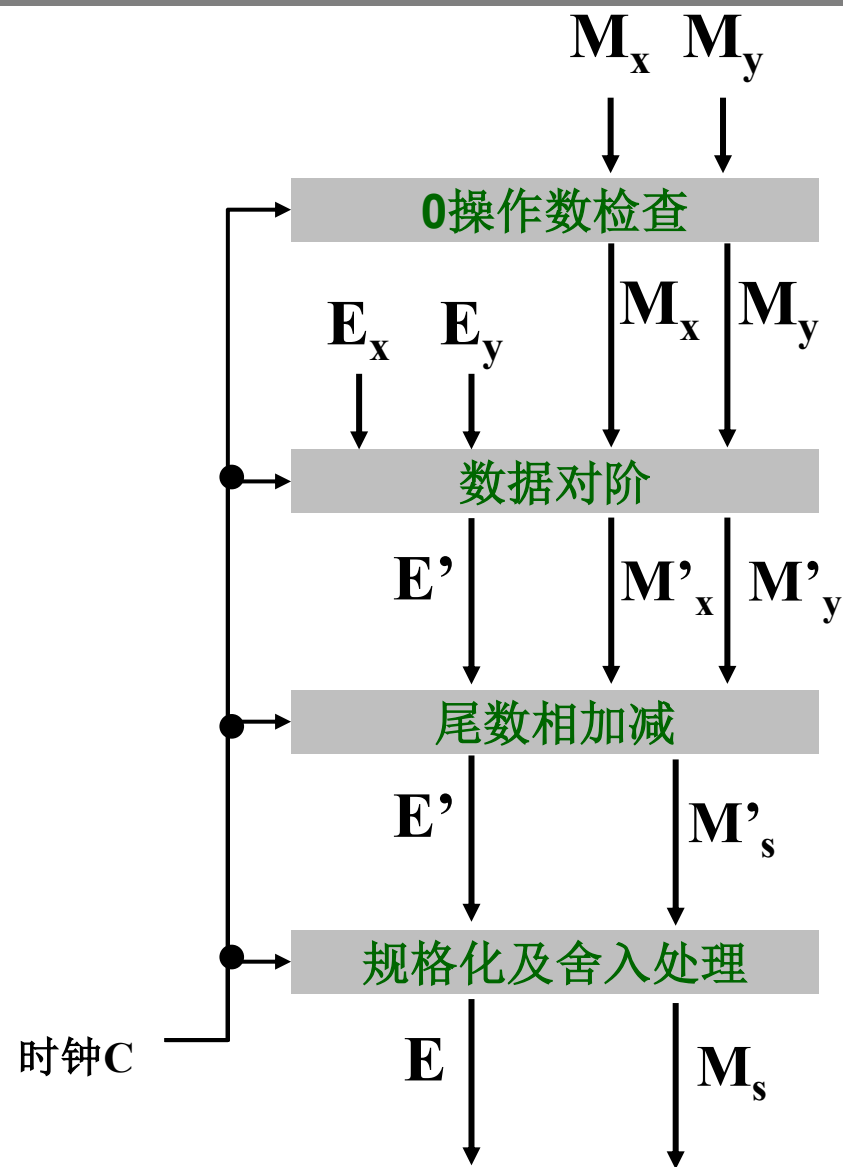
- 主存采用多体交叉存储器，Cache，以提高访问速度。



流水计算机系统组成原理示意图

>>> 2、浮点运算流水线

- 运算器按定点数和浮点数类型，将执行部件分为定点执行部件和浮点执行部件，分别处理相应的运算指令；
 - 浮点数运算又可分为加减法和乘法两部分；
- 每部分运算执行部件均可以流水线方式组织；
 - 通常，浮点加减法运算流水线阶段划分4阶段；
 - ◆ 0操作数检查、数据对阶、尾数加减、规格化及舍入处理；



>>> 5.6.3 指令流水线设计中的若干主要问题

➤ 流水线的相关

○ 由于指令间的依赖关系，导致流水线无法按设计好的周期执行的现象；

◆ 主要的相关类型：资源相关、数据相关、控制相关；

➤ 流水线的冒险/冲突

○ 相关问题导致流水线无法得到正确执行结果的现象；

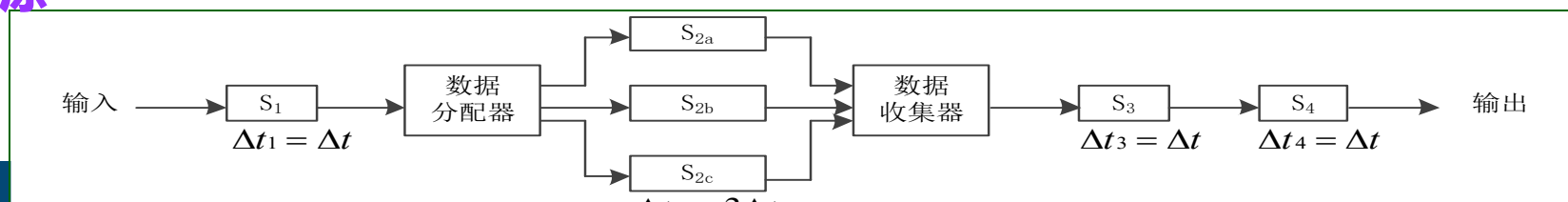
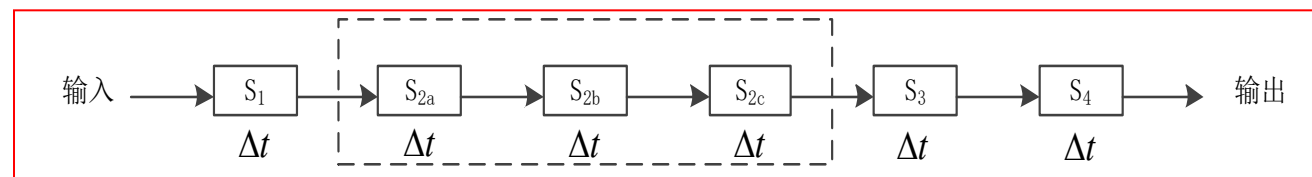
○ 如何划分流水线中过程段是解决问题的一个关键因素（因素诸多）

➤ 流水线拥挤问题的解决

○ 希望过程段时间相近即速度匹配（否则产生拥挤））

○ 方案一：瓶颈阶段细分

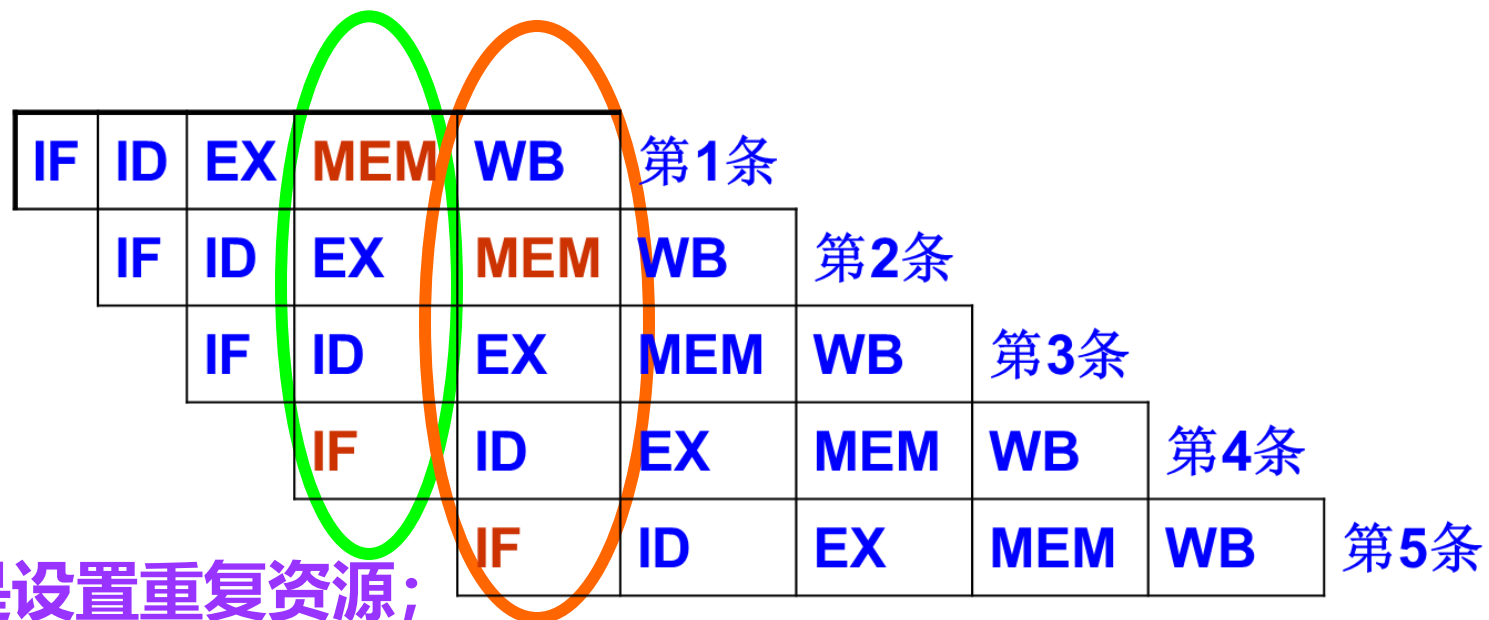
○ 方案二：设置重复资源



>>> 流水线的资源相关问题

➤ 资源相关

- 多条指令进入流水线后，在同一机器周期内争用同一个功能部件所发生的冲突；



- **解决方法：**指令推迟执行，或是设置重复资源；

➤ 例如：第1、2条指令的MEM与第4、5条指令的IF，都需要访存；

➤ 若数据与指令在同一存储器中，且单端口访问，则争用资源；

➤ 解决方法

- ① 第4条指令推迟执行（至少推迟3个周期）；
- ② 将数据与指令分离存储；或采用双端口存储器；

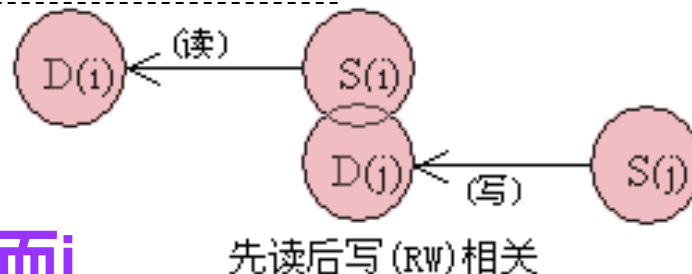
>>> 数据相关的类型

设指令i、j依次流入流水线

➤ “先读后写” WAR相关

- 正常顺序是：i先读，j再写；
- 由于异步流动下，j可能先于指令i被解释，从而i读到j写后的数据，不正确。

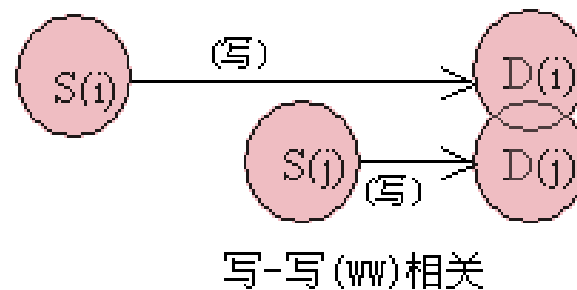
i是读数单元是j写数单元



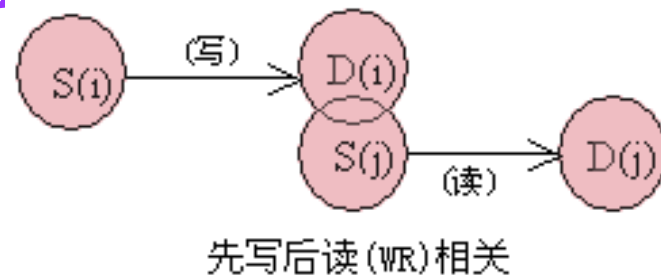
➤ “写-写” 相关

i, j写同一单元

- 正常情况下，最后单元中保存j写的数据；
- 由于异步流动，j可能先于i写数据，使该单元最后结果不正确。



➤ “先写后读” RAW相关类似



>>> 流水线的数据相关问题

➤ 数据相关

ADD R1, R2, R3 ; $(R2) + (R3) \rightarrow R1$
SUB R4, R1, R5 ; $(R1) - (R5) \rightarrow R4$
AND R6, R1, R7 ; $(R1) \wedge (R7) \rightarrow R6$

○ 后一条指令需使用前一条指令的执行结果;

○ **解决方法**: 指令推迟执行, 或是定向传送技术。

➤ 设某程序段如右所示, 则这3条指令的流水执行如下:

○ ADD指令在第5个周期 (WB) 才会更新R1;

○ 但SUB、AND指令需要更早读取R1;

➤ 解决方法

① 数据相关指令推迟若干时钟周期执行;

② 数据定向 (旁路) 技术, 直接采用运算器的结果;

③ 改变程序的执行顺序, 将无关指令提前执行;

IF	ID	EX	MEM	WB	ADD		
	IF	ID	EX	MEM	WB	SUB	
		IF	ID	EX	MEM	WB	AND

> 课本P191 【例5.6】

流水线中有三类数据相关冲突：**写后读(RAW)相关**；**读后写(WAR)相关**；**写后写(WAW)相关**。判断以下三组指令各存在哪种类型的数据相关。

① I1 ADD R1,R2,R3 ; $(R2+R3) \rightarrow R1$

写后读(RAW)相关

I2 SUB R4,R1,R5 ; $(R1-R5) \rightarrow R4$

② I3 STA M(x), R3; $(R3) \rightarrow M(x)$, M(x)是存储单元

读后写(WAR)相关

I4 ADD R3,R4,R5; $(R4+R5) \rightarrow R3$

③ I5 MUL R3,R1,R2; $(R1) \times (R2) \rightarrow R3$

写后写(WAW)相关

I6 ADD R3,R4,R5; $(R4+R5) \rightarrow R3$

>>> 控制相关

➤ 控制相关 转移控制类指令执行导致的

- 引起原因：当执行转移指令时，根据转移条件是否发生来控制指令的执行顺序；
- 根据当前的状态条件，或顺序取下一条指令执行，或要转移到新的目标地址去取指令执行。

➤ 例如：当执行指令A的时候，紧跟A之后的几条指令已经被放到了指令流水线中且进行了预处理

- 当执行完转移指令，应跳转到新的目标地址去取指令执行，只能先把已经进入到流水线的指令清空，然后才能把转移跳转后所需的指令取过来放在流水线上，但此时流水线就会出现断流

>>> 流水线的控制相关问题

➤ 控制相关解决方法

○ 冻结或者排空流水线

- ◆ 一旦发现分支指令就暂停流水线中分支指令之后的指令执行，直到分支指令确定了新的PC值为止；

○ 为了减少转移指令对流水线性能的影响可采用两种技术

- ◆ 延迟转移法：编译程序重排指令序列。不排空流水线—先执行再转移

- ◆ 转移预测法

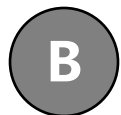
- ◇ 依据指令过去的行为预测将来的行为，通过使用转移取和顺序取两路指令预取队列以及目标指令缓冲器，可将转移预测提前到取指阶段进行；

2009年考研408真题 第18题

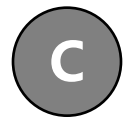
18.某计算机的指令流水线由四个功能段组成，指令流经各功能段的时间（忽略各功能段之间的缓存时间）分别是90ns、80ns、70ns和60ns，则该计算机的CPU时钟周期至少是（ ）

**A**

90ns

**B**

80ns

**C**

70ns

**D**

60ns

提交

2010年考研408真题 第19题

19.下列不会引起指令流水阻塞的是（ ）

- ☒ A 数据旁路
- ☐ B 数据相关
- ☐ C 条件转移
- ☐ D 资源冲突

提交

2013年考研408真题 第18题

18.某CPU主频为1.03GHz，采用4级指令流水线，每个流水段的执行需要1个时钟周期。假定CPU执行了100条指令，在其执行过程中，没有发生任何流水线阻塞，此时流水线的吞吐率为（ ）

A 0.25×10^9 条指令/秒

B 0.97×10^9 条指令/秒

C 1.0×10^9 条指令/秒

D 1.03×10^9 条指令/秒

提交

2016年考研408真题 第18题

18.在无转发机制的五段基本流水线中，下列指令序列存在数据冒险的指令对是（ ）。

L1: ADD R1, R2, R3; $(R2)+(R3) \rightarrow R1$

L2: ADD R5, R2, R4; $(R2)+(R4) \rightarrow R5$

L3: ADD R4, R5, R3; $(R5)+(R3) \rightarrow R4$

L4: ADD R5, R2, R6; $(R2)+(R6) \rightarrow R5$

- ☐ A L1和L2
 ☒ B L2和L3
 ☐ C L2和L4
 ☐ D L3和L4

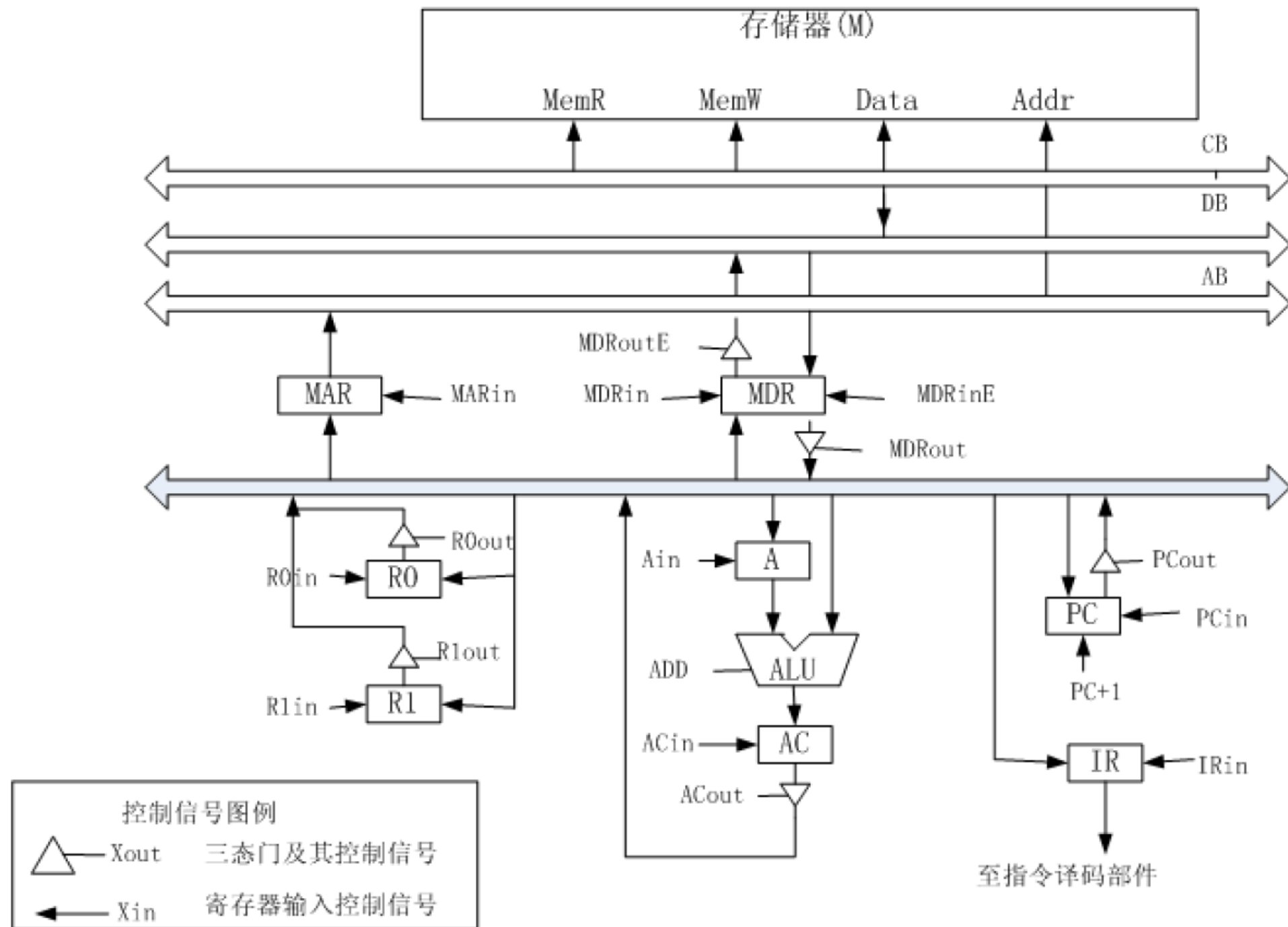
提交

>>> 2009年考研408真题第44题

44. (13分) 某计算机字长16位，采用16位定长指令字结构，部分数据通路结构如下图所示，图中所有控制信号为1时表示有效，为0时表示无效。

例如，控制信号MDRinE为1表示允许数据从DB打入MDR，MDRin为1表示允许数据从内部总线打入MDR。

假设MAR的输出一直处于使能状态。加法指令“**ADD (R1) , R0**”的功能为 $(R0) + ((R1)) \rightarrow (R1)$ ，即将R0中的数据与R1的内容所指主存单元的数据相加，并将结果送入R1的内容所指主存单元中保存。



>>> 2009年考研408真题第44题

➤ 下表给出了上述指令取指和译码阶段每个节拍（时钟周期）的功能和有效控制信号；

➤ 请按照表中描述方式用表格列出指令执行阶段每个节拍的功能和有效控制信号。

时钟	功能	有效控制信号
C1	$MAR \leftarrow (PC)$	PCout, MARin
C2	$MDR \leftarrow M(MAR)$ $PC \leftarrow (PC) + 1$	MemR, MDRinE, PC +1
C3	$IR \leftarrow (MDR)$	MDRout, IRin
C4	指令译码	无



2 个 解 答

时钟	功能	有效控制信号
C5	$MAR \leftarrow (R1)$	R1out, MARin
C6	$MDR \leftarrow M(MAR)$	MemR, MDRinE
C7	$A \leftarrow (R0)$	R0out, Ain
C8	$AC \leftarrow (MDR) + (A)$	Add, MDRout, ACin
C9	$MDR \leftarrow (AC)$	ACout, MDRin
C10	$M(MAR) \leftarrow MDR$	MDRoutE, MemW

时钟	功能	有效控制信号
C5	$MAR \leftarrow (R1)$	R1out, MARin
C6	$MDR \leftarrow M(MAR)$	MemR, MDRinE
C7	$A \leftarrow (MDR)$	MDRout, Ain
C8	$AC \leftarrow (R0) + (A)$	Add, R0out, ACin
C9	$MDR \leftarrow (AC)$	ACout, MDRin
C10	$M(MAR) \leftarrow MDR$	MDRoutE, MemW

>>> 2014年考研408真题第44题

44. (12分) 某程序中有如下循环代码段P:”for (i=0;i<N;i++) sum+=A[i];”
，假设编译时变量sum和i分别分配在寄存器R1和R2中，常量N在寄存器R6中，数组A的首地址在寄存器R3中。程序段P起始地址为0804 8100H，对应的汇编代码和机器代码下表所示。

编号	地址	机器代码	汇编代码	注释
1	0804 8100H	00022080H	loop: sll R4, R2, 2	(R2)<<2→R4
2	0804 8104H	00832020H	add R4, R4, R3	(R4)+(R3)→R4
3	0804 8108H	8C850000H	load R5, 0(R4)	((R4)+0)→R5
4	0804 810CH	00250820H	add R1, R1, R5	(R1)+(R5)→R1
5	0804 8110H	20420001H	addi R2, R2, 1	(R2)+1→R2
6	0804 8114H	1446FFFAH	bne R2, R6, loop	If(R2)!=(R6) goto loop

>>> 2014年考研408真题第44题

➤ 执行上述代码的计算机M采用32位定长指令字，其中分支指令bne采用如下格式。

31	26	25	21	20	16	15	0
OP				Rs		Rd	OFFSET

OP为操作码；Rs和Rd为寄存器编号；OFFSET为偏移量，用补码表示。请回答下列问题，并说明理由。

(1) M的存储器编址单位是什么？

如表所示，每条指令占4个存储单元，故存储器按字节编址；

(2) 已知sll指令实现左移功能，数组A中每个元素占多少位？

由表中指令序列可知，R2存放数组A元素序号，元素地址由 $R2 \times 4$ 得到，故每个元素占4个存储单元，即4字节，32位。

>>> 2014年考研408真题第44题

➤执行上述代码的计算机M采用32位定长指令字，其中分支指令**bne**采用如下格式。

31	26	25	21	20	16	15	0
OP				Rs		Rd	OFFSET

(3) 上表中**bne**指令**OFFSET**字段的值是多少？

Bne指令机器码为1446FFFAH，由指令格式可知，

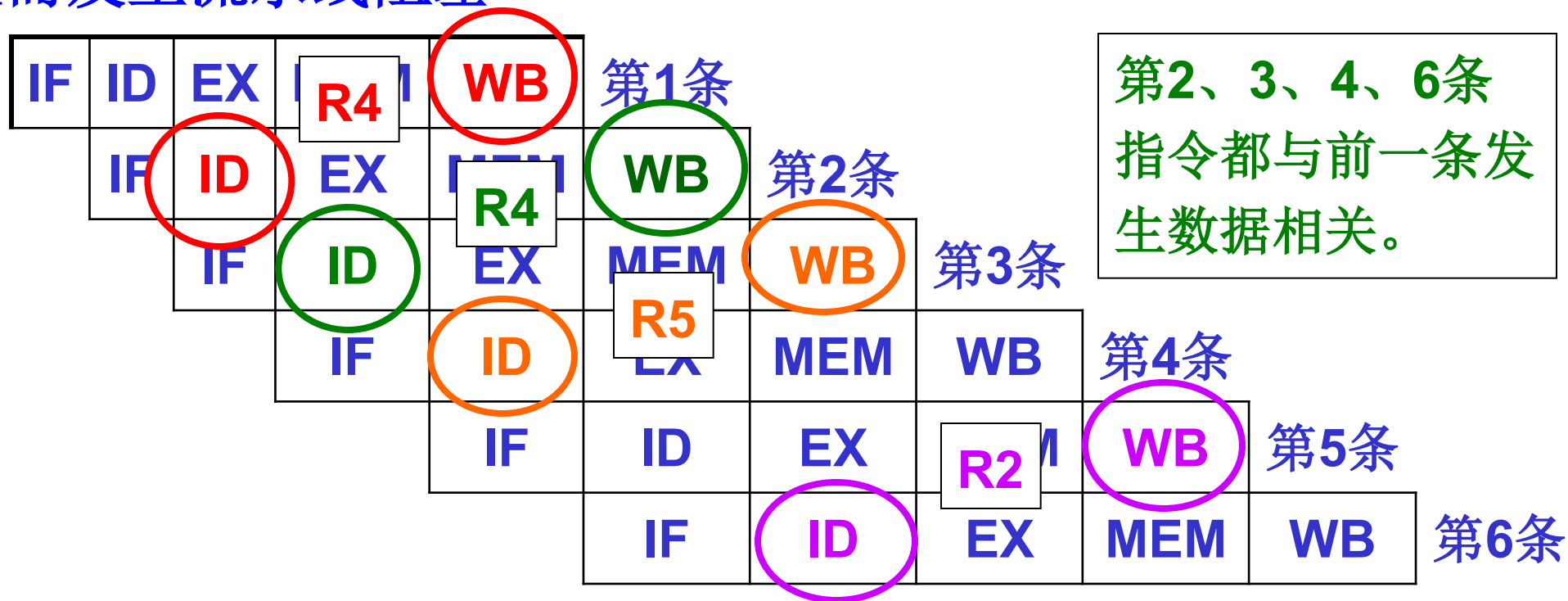
OFFSET的值为0FFFAH。

已知**bne**指令采用相对寻址方式，当前PC内容为**bne**指令地址，通过分析指令地址和**bne**指令内容，推断出**bne**指令的转移目标地址计算公式。

由于目标地址为0804 8100H，当前PC为0804 8114H，偏移量为-6，故地址计算公式为： $(PC)+4+OFFSET*4$

>>> 2014年考研408真题第44题

➤若M采用5级指令流水线：IF（取指）、ID（译码及取数）、EXE（执行）、MEM（访存）、WB（写回寄存器），且硬件不采用任何转发措施，分支指令的执行均引起3个时钟周期的阻塞，则P中哪些指令的执行会由于数据相关而发生流水线阻塞？



>>> 2014年研究生统考试题 第44题

IF	ID	EX	MEM	WB	I1					
	IF	ID	EX	MEM	WB	I2				
		IF	ID	EX	MEM	WB	I3			
			IF	ID	EX	MEM	WB	I4		
				IF	ID	EX	MEM	WB	I5	
					IF	ID	EX	MEM	WB	I6

➤哪条指令的执行会发生控制冒险？为什么指令1的执行不会因为与指令5的数据相关而发生阻塞？

- 第6条指令bne指令会发生控制冒险；
- 第5条指令与下次循环的第1条指令有数据相关，但第6条指令与第5条指令有数据相关，会推迟3个周期，因而可消除该数据相关；

>>> 本章小结

- 理解**CPU**的功能与结构（运算器、控制器必备部件）；
- 理解**CPU**执行指令过程中的数据通路建立概念；
- 给定**CPU**结构和指令功能，能够分析指令周期，并能写出每个**CPU**周期所需的控制信号；
- 理解微程序控制器的设计思想和工作原理
 - 以TEC-XP为例，能够理解微指令格式、微程序中微指令的寻址方式；
- 理解流水**CPU**的基本原理，能够分析流水线结构、计算流水线的性能指标，清楚各种相关问题的产生原因和解决方案；



某CPU主频为1.03GHz，采用4级指令流水线，每个流水段的执行需要1个时钟周期。假定CPU执行了100条指令，在其执行过程中，没有发生任何流水线阻塞，此时流水线的吞吐率为（ ）

解答：采用4级流水执行100条指令，在执行过程中共用 $4 + (100 - 1) = 103$ 个时钟周期，CPU的主频1.03GHz，也就是每秒钟有1.03G个时钟周期。流水线的吞吐率 $= 1.03G \times 100 / 103 = 1.0 \times 10^9$ 条指令/秒

提交